Assignment 2 - Fresh test results

English report based on 17 completed fresh benchmark runs, functional verification and supplied CloudWatch evidence. Repository publication and student AI-reflection review remain.

This report uses only the new session 20261108T135433Z. Earlier reports and tuning experiments are excluded. Original records are under results/a2-fresh/20261108T135433Z.

Verified findings and limitations

The 17 runs sent and processed 9,000,000 events, with zero final failed requests, zero additional duplicates and 269 application retries. Every recorded final Ready/in-flight/outbox count was zero. The 4-ingest 500K/512 run improved main HTTPthroughput by 25.80% versus one ingest, while end-to-end throughput fell by 5.16%. The 1M/512 run had Ready peak 377,327, mean Consumer lag 49,377.24 ms, 110.256 seconds of drain/verification and 77 retries. These overload runs did not meet the queue-depth or lag targets.

The 1M ingest-counter deltas cover only 3,716 requests, rather than the full 1,000,077 client attempts. The cause of the mismatch is unresolved, so that distribution chart is not used to claim whole-run balance. The 4-ingest 500K/512 counters record 124,998 / 125,000 / 125,000 / 125,002 attempts and provide a complete balanced-distribution observation.

The 1M Consumer mean I/O wait was 41.39%, with mean OS-observed writes of 4,842.87 IOPS; RabbitMQ mean CPU was 54.74%. These support downstream storage/processing pressure without proving an exclusive EBS IOPS bottleneck. Dynamic scaling was disabled for the fixed-worker sweep, so that experiment does not validate its adaptive behavior.

Corrected two- and four-ingest CloudWatch captures are included. Per-run Consumer sweeps retain automatic broker/system evidence but have no separately supplied ALB CloudWatch captures. The single-target console image shows the deregistration transition; routing probes provide separate evidence of the active single-ingest topology. AI reflection text is a factual draft requiring student review; local A2 publication to the configured Git remote has not been verified.

Coverage of the assignment requirements

Requirement	Status	Where in this report
Client 64 / 128 / 256 / 512 threads	Met: every level at 1, 2 and 4 ingests (12 runs), plus a 1M run at 512	Benchmark results
Consumer thread sweep	Met, in two parts. This session ran 5 / 10 / 15 / 20 workers (the sweep recorded in `REQUIREMENTS_A2.md`) at 4 ingests and 128 client threads. A supplementary test of 10 / 20 / 40 / 80 workers (forward then reverse, 8 runs, 2026-11-09 04:06 to 04:51 UTC) is in `TUNING_PARAMETERS.md` section 2: no measurable throughput change above ten workers, and active workers never exceeded 10 because only ten ordered resort lanes exist	Benchmark results (last four rows), Consumer sweep note, `TUNING_PARAMETERS.md` section 2
Single ingest, 500K swipes: throughput, latency, queue metrics	Met	Benchmark results, per-run evidence
Two ingests, same load: throughput and distribution	Met	Benchmark results, scaling comparison, `distribution.png` per run
Four ingests, 500K then 1M stress; find bottlenecks	Met	Scaling comparison, 1M run, performance improvement analysis

Requirement	Status	Where in this report
Client metrics (runtime, throughput, failures, retries)	Met	Per-run tables
Queue metrics (peak/average depth, consumer and producer rate)	Met	Per-run tables, `queue.png`
System metrics (CPU, memory, network)	Met	`system.png` per run, per-host CSVs
Subscribers receive real, swipe-driven leaderboard updates	Met	Subscribers receive real, swipe-driven leaderboard updates
Client output for 1, 2, 4 instances	Met	Complete client output linked in every run
Queue console screenshots	Met: the final RabbitMQ Management capture of every run is embedded (queued messages over time and message rates, last ten minutes) and all other captures are linked	Per-run evidence
ALB metrics showing request distribution	Met with a limit: CloudWatch captures are for the whole load balancer (RequestCount and TargetResponseTime). Per-target distribution comes from the ingest counters and routing probes, not from an ALB per-target chart. The 1M counter delta is incomplete (3,716 of 1,000,077 attempts) and is not used for a balance claim	Included AWS console evidence
Performance improvement analysis	Met	Performance improvement analysis

Method and configuration

Four existing ingest EC2 instances are registered in stages: one, two, then four. Each stage is checked through fresh ALB health requests before load. The client runs on the Mac through HTTP/WebSocket ALB endpoints; Consumer metrics use a private SSH tunnel. Every benchmark uses ten resorts, Top-N = 20, random seed 6650, persistent official lift values, prefetch 128, maximum database batch 64,10 ms flush trigger, ACK batch 16 and publisher channel pool 64. Main scaling runs use ten fixed consumer workers; the separate sweep uses 5/10/15/20 fixed workers. SQLite FULL durability is retained.

Existing durable data is retained, not deleted or reset. Database size/cache state can change over sequential runs, so this is a measured deployment comparison rather than a perfectly isolated capacity experiment. Health routing probes identify responding servers; console target screenshots separately establish registered-target state.

For each run, monitoring starts before the client and continues through final drain. RabbitMQ UI screenshots are captured from the actual Management console in an ephemeral UTC browser; generated queue/resource graphs are separately labeled. No absent samples are replaced with zero.

Recorded infrastructure

Instance metadata captured at 2026-11-08T14:25:30.151283+00:00. RAM below is Linux MemTotal, not the advertised instance capacity.

Role	EC2 type	vCPUs	Linux RAM GiB	Private IPv4
ingest1	t3.micro	2	0.89	172.31.45.254
ingest2	t3.micro	2	0.89	172.31.24.108
ingest3	t3.micro	2	0.89	172.31.45.75

Role	EC2 type	vCPUs	Linux RAM GiB	Private IPv4
ingest4	t3.micro	2	0.89	172.31.44.196
consumer	t3.medium	2	3.75	172.31.36.151
rabbitmq	t3.medium	2	3.75	172.31.32.233

Fresh clock tracking is preserved in results/a2-fresh/20261108T135433Z/clock-synchronization.json. This is a point-in-time check, not continuous clock calibration; the Mac client is not calibrated by the EC2 check.

`TargetResponseTime` describes the ALB-to-target response phase, while client HTTP latency includes the client-side round trip. WebSocket snapshot-to-receipt wall-clock differences are not calibrated one-way latencies. AWS metric definitions.

Benchmark results

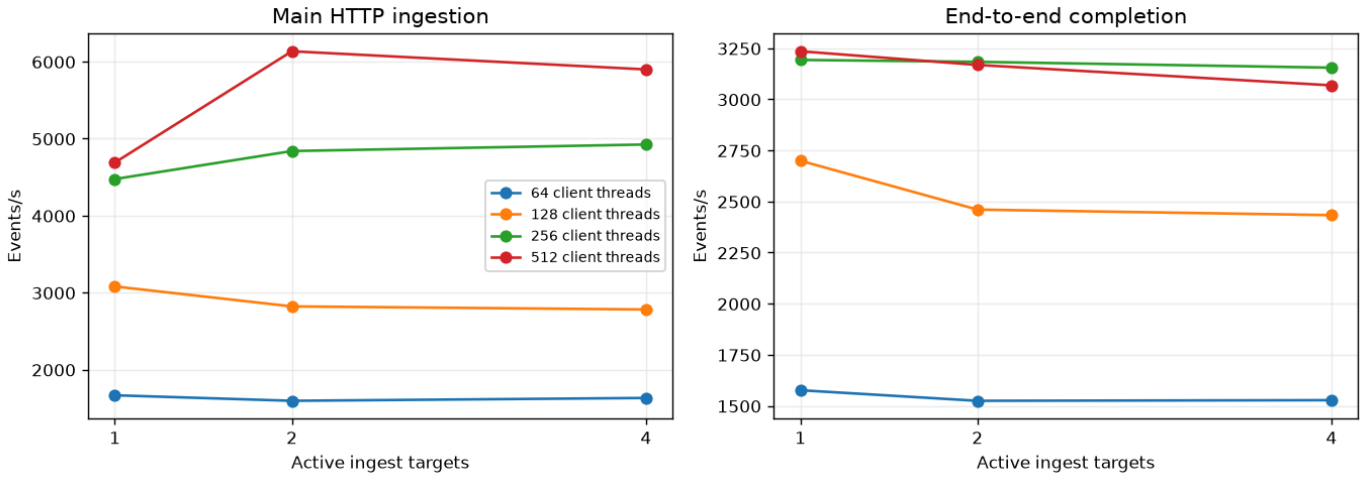
Ingests	Events	Client threads	Workers	HTTP main/s	End-to-end/s	HTTP p99 ms	Retries	Ready peak	Mean lag ms
1	500000	64	10	1,666.74	1,576.41	93.47	0	0	19.66
1	500000	128	10	3,080.16	2,699.50	134.59	0	0	25.18
1	500000	256	10	4,475.42	3,192.84	275.29	6	55,463	5,177.43
1	500000	512	10	4,689.51	3,235.22	469.87	131	59,967	7,135.63
2	500000	64	10	1,594.86	1,524.64	153.19	0	0	18.22
2	500000	128	10	2,821.24	2,460.44	191.59	0	29	29.39
2	500000	256	10	4,839.15	3,183.52	272.08	0	68,120	9,306.91
2	500000	512	10	6,135.42	3,168.37	314.39	55	145,164	16,665.74
4	500000	64	10	1,631.83	1,527.74	155.48	0	0	18.87
4	500000	128	10	2,781.22	2,433.20	202.31	0	26	24.43
4	500000	256	10	4,924.34	3,154.89	194.77	0	80,309	7,641.57
4	500000	512	10	5,899.50	3,068.38	331.42	0	141,495	13,798.40
4	1000000	512	10	6,551.14	3,323.33	312.63	77	377,327	49,377.24
4	500000	128	5	2,604.84	2,275.86	241.01	0	138	42.40
4	500000	128	10	3,155.08	2,650.72	92.10	0	446	34.96
4	500000	128	15	3,032.48	2,643.07	93.09	0	1,059	38.50
4	500000	128	20	3,036.31	2,620.63	99.31	0	111	39.38

Scaling comparison and interpretation

The following compares the latest completed scaling observation for each node/thread configuration with the corresponding single-ingest run. All attempts remain listed in this report; observations are not selected by best throughput. Persistent database/cache state and time of day differ, so percentage changes are descriptive rather than statistically established causal effects.

Client threads	Ingests	Main HTTP change vs1	End-to-end change vs1
64	2	-4.31%	-3.28%
64	4	-2.09%	-3.09%
128	2	-8.41%	-8.86%
128	4	-9.71%	-9.86%
256	2	+8.13%	-0.29%
256	4	+10.03%	-1.19%
512	2	+30.83%	-2.07%

Client threads	Ingests	Main HTTP change vs1	End-to-end change vs1
512	4	+25.80%	-5.16%



If main HTTP throughput rises while end-to-end throughput changes little and queues/drain grow, the improvement is at ingestion rather than completed processing. Adding ingest servers increased HTTP ingestion throughput, but did not improve end-to-end throughput. All events were eventually processed, with additional time required to drain the backlog. Conversely, little scaling at low client concurrency may reflect the admitted workload rather than a saturated ingest. Neither high average I/O wait nor a brief broker CPU peak alone identifies an exclusive bottleneck.

Fixed-worker comparisons must consider both lag and queue depth alongside throughput. Ten ordered resort queues cap useful aggregation parallelism at ten; configuring 15/20 workers does not create15/20 independent resort lanes. Sampled bounds and mean lag do not prove per-event limits. Consumer throughput plateaued at approximately ten workers, consistent with the ten ordered resort queues. Under higher input rates, downstream processing could not keep pace with ingestion, causing backlog and increased lag. Storage-related waiting contributed to the pressure, but the measurements do not isolate a single cause.

Consumer sweep note. The four sweep rows above (5/10/15/20 workers, 4 ingests, 128 threads) are one original observation each. A repeat after an EC2 stop/start is kept separately in output/assignment2-repeat-20261108T201347Z/comparison.md. The repeat does not reproduce the original ordering: 5 workers gave +12.0% end-to-end throughput, 10 workers -11.8%, 15 workers -5.9% and 20 workers -3.3%, and the 5-worker repeat had a Ready peak of 8,848 and mean lag of 1,269 ms against 138 and 42 ms originally. Device reads fell from about 1,073 MiB (first run) to 4 MiB (last), which is consistent with cache warming. The sweep therefore does not show a reliable benefit of more than five workers on throughput; what it does show is that 15 and 20 workers add nothing over 10, which matches the ten ordered resort lanes. 40 and 80 workers were run later and are reported in the supplementary section below.

Performance improvement analysis

Within this session (all runs use the final configuration). The comparisons are descriptive observations: database size and cache state change between sequential runs.

Observation	Result
End-to-end throughput versus client threads, one ingest	1,576/s at 64 threads, 2,700/s at 128, 3,193/s at 256, 3,235/s at 512. It stops rising above 256 threads, while main HTTP throughput still rises from 4,475/s to 4,690/s
More ingests at 512 threads	Main HTTP throughput 4,690/s (1 ingest), 6,135/s (2), 5,900/s (4): +30.8% and +25.8% over one ingest, but end-to-end throughput stays near 3,100 to 3,250/s
Ready peak at 512 threads	59,967 (1 ingest), 145,164 (2), 141,495 (4), and mean consumer lag 7.1 s, 16.7 s, 13.8 s
1M run, 4 ingests, 512 threads	3,323/s end-to-end, Ready peak 377,327, mean lag 49.4 s, 77 retries, no failed requests
Consumer I/O (1M run)	Mean I/O wait 41.4%, 4,843 write IOPS observed by the OS, RabbitMQ CPU mean 54.7%

Reading. Ingestion scales with added ingest servers and client concurrency, but completed processing does not: from about 256 threads the end-to-end rate settles at about 3,100 to 3,300 swipes per second regardless of the number of ingests, and the extra admitted requests appear as queue depth and lag. At 64 and 128 threads (no queue build-up) the queue target of under 1,000 and mean lag under 100 ms are met; at 256 and above they are not. The bottleneck in these runs is therefore downstream of the ingest tier, in the Consumer's SQLite and EBS writes and in RabbitMQ CPU. High I/O wait and a brief broker CPU peak are consistent with this, but these measurements do not isolate one resource.

What produced the improvement (separate sessions, summarized from `TUNING_PARAMETERS.md`). The bigger gains came before this session, from the Consumer's storage path rather than from the ingest tier. They are single-ingest observations and each was a controlled change of one thing:

Change	Before	After
Grouping database writes (100K events, 64 threads, 3,000 IOPS)	1,148/s end-to-end, Ready peak 15,516, drain 13.2 s, mean lag about 3.8 s	1,253/s (+9%), Ready peak 19, drain 0.74 s, mean lag 28 ms
Grouping database writes (500K events)	1,193/s (419 s)	1,503/s (331.8 s), about +26%
EBS 3,000 to 6,000 IOPS (500K, 128 threads, same application settings)	2,198/s, Ready peak 94,252, drain 44.3 s, I/O wait 57%	2,874/s (+31%), Ready peak 0, drain 0.76 s, I/O wait 21%
ACK batch 1 to 16		+4% to +6% end-to-end, lower Consumer CPU at 256 threads
Prefetch 64 to 512, channel pool 32 to 64		No measurable end-to-end change (see the tuning report for the pool and its tail-latency effects)

This session ran after those changes (it started about seven hours after the 6,000-IOPS step), so its numbers show the system near the new ceiling rather than the gains themselves. No Assignment 1 versus Assignment 2 throughput comparison is claimed, because Assignment 1 was not re-run in this session.

Supplementary consumer sweep: 10 / 20 / 40 / 80 workers

The assignment text lists consumer thread counts of 10, 20, 40 and 80. This session's sweep used 5 / 10 / 15 / 20, so the listed values were run afterwards on 2026-11-09, forward then in reverse (10, 20, 40, 80, 80, 40, 20, 10). The same data, with the original evidence paths, are in `TUNING_PARAMETERS.md` section 2.

Setup. 4 ingests behind the ALB, 500K events, 128 client threads, seed 6650, Top-N 20; prefetch 128, database batch 64, ACK batch 16, ingest channel pool 64, `CONSUMER_DYNAMIC_SCALING=false`; pipeline drained at the start of each run; the Consumer was restarted at each worker setting with `monitoring/set_consumer_threads.py` and the monitored `configuredThreads` matched. This follows an EC2 restart.

Run	Workers	Evidence directory (`results/a2-ec2/`)	Client window (UTC)	End-to-end /s	Main HTTP /s	HTTP P99 ms	Ready peak	Mean lag ms	Consumer CPU mean / max %	Consumer I/O wait %	Max active workers
1	10	`instrumented-500000-c128-20261109T040639Z`	04:06:42 to 04:10:00	2,604	3,208	77.3	31,061	5,924.8	32.8 / 99.8	55.1	10
2	20	`instrumented-500000-c128-20261109T041528Z`	04:15:31 to 04:18:35	2,768	3,208	80.1	608	34.1	39.7 / 100.0	20.9	7
3	40	`instrumented-500000-c128-20261109T042706Z`	04:27:09 to 04:30:18	2,701	3,150	83.5	1,199	35.0	38.9 / 99.9	19.1	6
4	80	`instrumented-500000-c128-20261109T043211Z`	04:32:14 to 04:35:25	2,682	3,144	85.8	648	35.2	39.6 / 100.0	18.1	8
5	80	`instrumented-500000-c128-20261109T043641Z`	04:36:44 to 04:39:57	2,649	3,063	99.5	1,552	37.9	38.9 / 100.0	17.5	10
6	40	`instrumented-500000-c128-20261109T044106Z`	04:41:10 to 04:44:21	2,651	3,073	94.2	0	37.2	39.2 / 99.8	17.7	8
7	20	`instrumented-500000-c128-20261109T044456Z`	04:45:00 to 04:48:12	2,641	3,111	93.7	1,035	34.4	38.7 / 100.0	18.0	6

Run	Workers	Evidence directory (`results/a2-ec2/`)	Client window (UTC)	End-to-end /s	Main HTTP /s	HTTP P99 ms	Ready peak	Mean lag ms	Consumer CPU mean / max %	Consumer I/O wait %	Max active workers
8	10	`instrumented-500000-c128-20261109T044828Z`	04:48:31 to 04:51:42	2,654	3,127	83.6	0	39.4	38.5 / 99.1	18.8	8

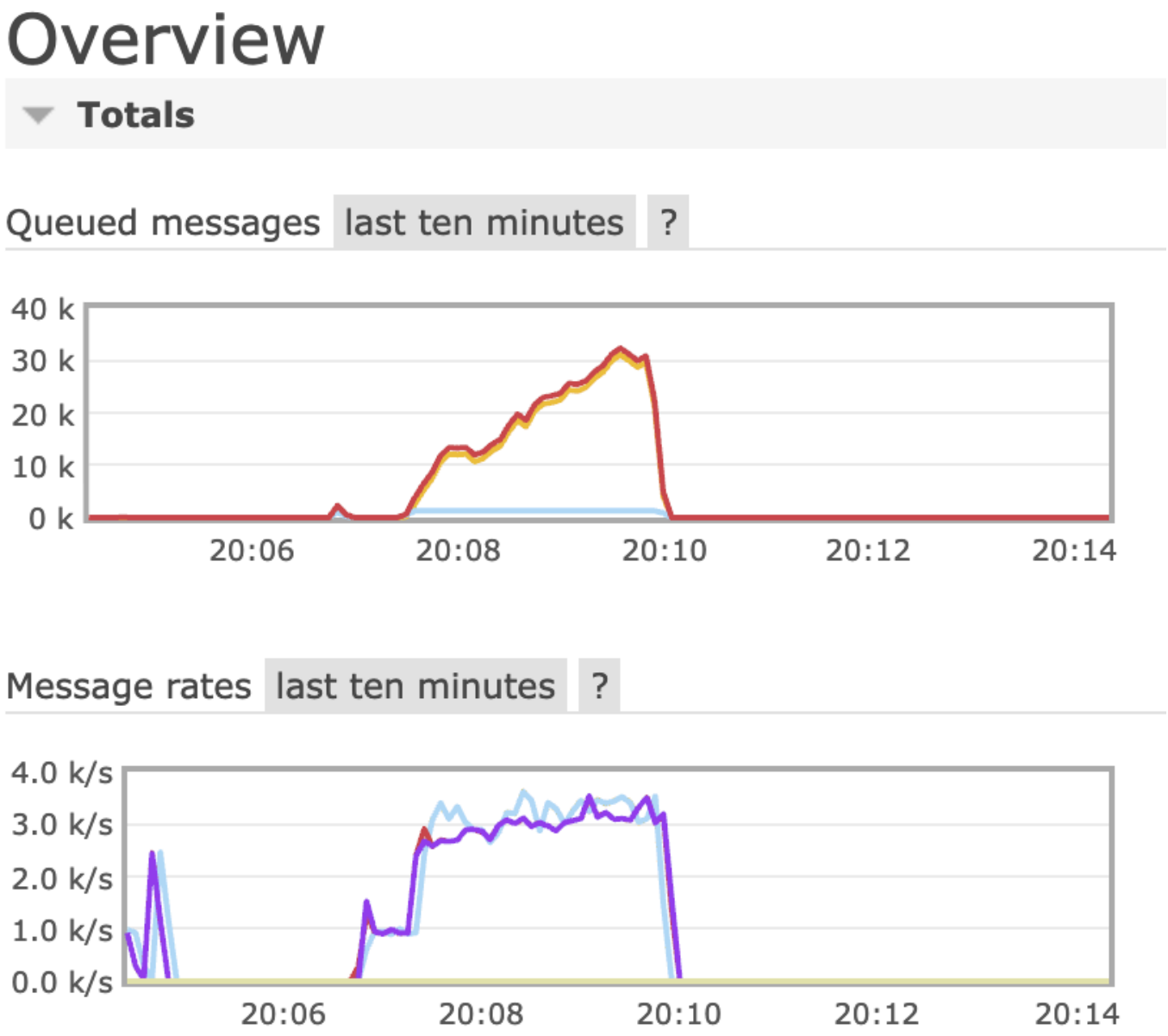
All eight runs processed 500,000 events with 0 duplicates, 0 failed requests, 0 application retries and final Ready, in-flight and outbox 0.

Finding. Excluding run 1, end-to-end throughput was 2,641 to 2,768/s for 10, 20, 40 and 80 workers; two runs of the same setting differ by up to 4.8%, as much as the settings differ from each other, so more workers gave no measurable benefit or cost. Sampled active workers never exceeded 10 even with 80 configured, which matches the ten ordered resort lanes. The second run of each setting was 0 to 5% lower, in the same direction for every setting, which points to drift over the session. Run 1 is a cold-cache observation (I/O wait 55% against 17 to 21% in the others, Ready peak 31,061, mean lag 5,925 ms) and is not a 10-worker result: the same setting in run 8 had Ready 0 and 39 ms lag. Limits: two runs per setting, one load level, and a cold first run.

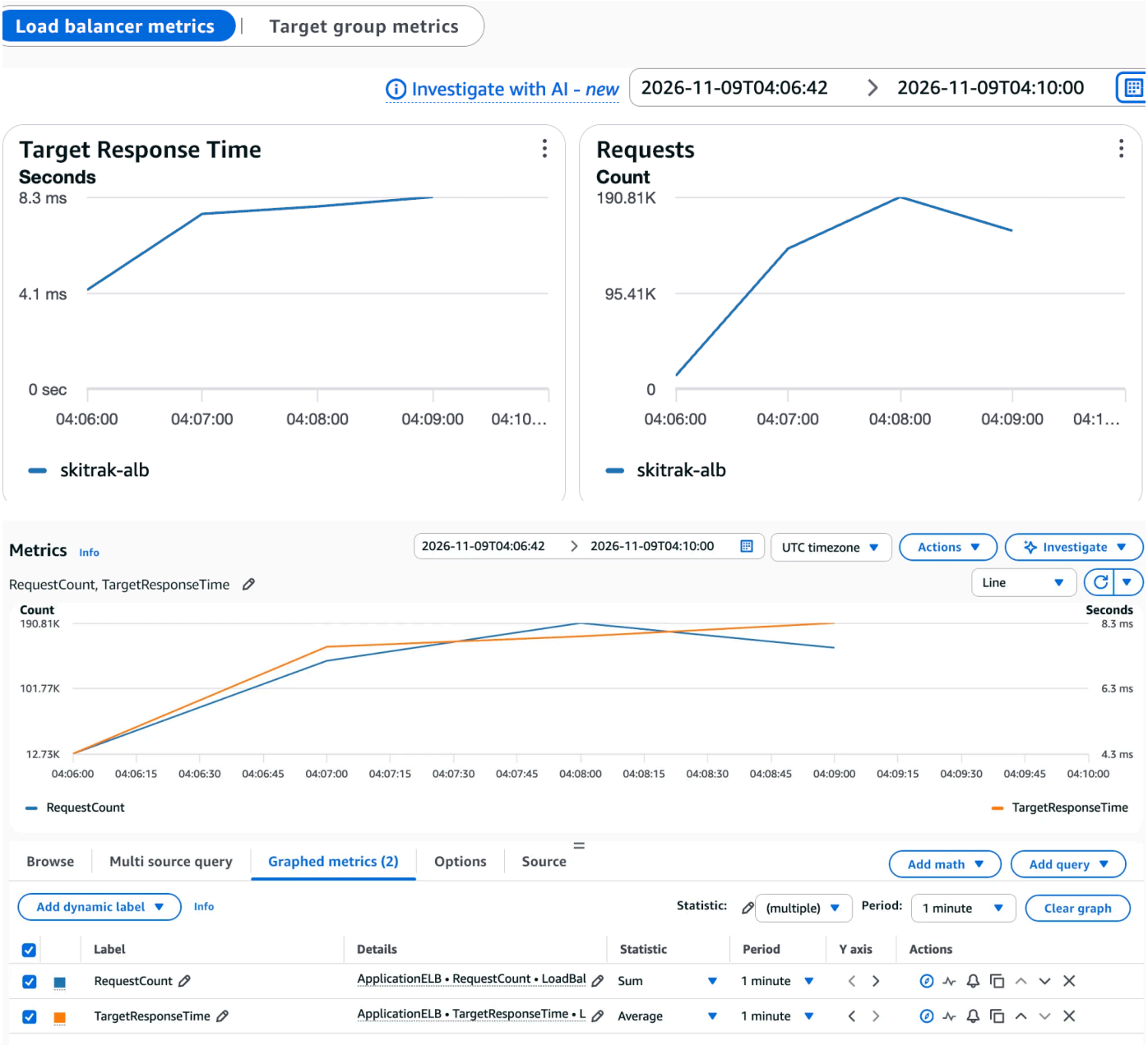
ALB view (runs 1 to 8). The CloudWatch captures use each run's client window with 1-minute buckets (first and last points are partial minutes, lines between points are interpolation, and the y-axis range differs per chart). As read from the chart axes, the peak one-minute request count was 180K to 194K and the peak one-minute average TargetResponseTime 8.3 to 10.5 ms in every run, with no pattern by worker count. That is expected: the worker count acts after the ingest tier, so ingest-side metrics should not move. For run 8 the Metrics chart covers runs 7 and 8 together (04:45:00 to 04:51:42 UTC), so it is not specific to run 8.

Screenshots

Run 1: 10 workers, 04:06:42 to 04:10:00 UTC



RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). Queued messages rise to about 32K during the run and drain by about 20:10 (cold-cache run; Ready peak 31,061 by the Consumer monitor).

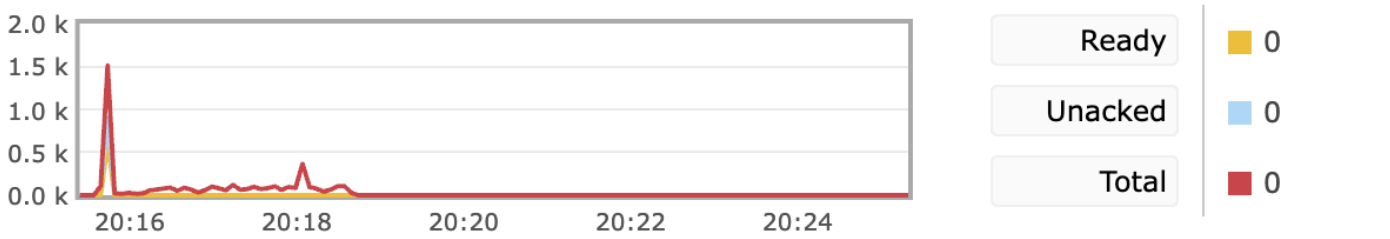

Load balancer metrics
Target group metrics
Investigate with AI - new
2026-11-09T04:06:42
2026-11-09T04:10:00
Target Response Time
Seconds
8.3 ms
4.1 ms
0 sec
04:06:00
04:07:00
04:08:00
04:09:00
04:10...
skitrak-alb
Requests
Count
190.81K
95.41K
0
04:06:00
04:07:00
04:08:00
04:09:00
04:1...
skitrak-alb

Metrics
Info
2026-11-09T04:06:42
2026-11-09T04:10:00
UTC timezone
Actions
Investigate
RequestCount, TargetResponseTime
Line
Count
190.81K
101.77K
12.73K
Seconds
8.3 ms
6.3 ms
4.3 ms
04:06:00
04:06:15
04:06:30
04:06:45
04:07:00
04:07:15
04:07:30
04:07:45
04:08:00
04:08:15
04:08:30
04:08:45
04:09:00
04:09:15
04:09:30
04:09:45
04:10:00
RequestCount
TargetResponseTime

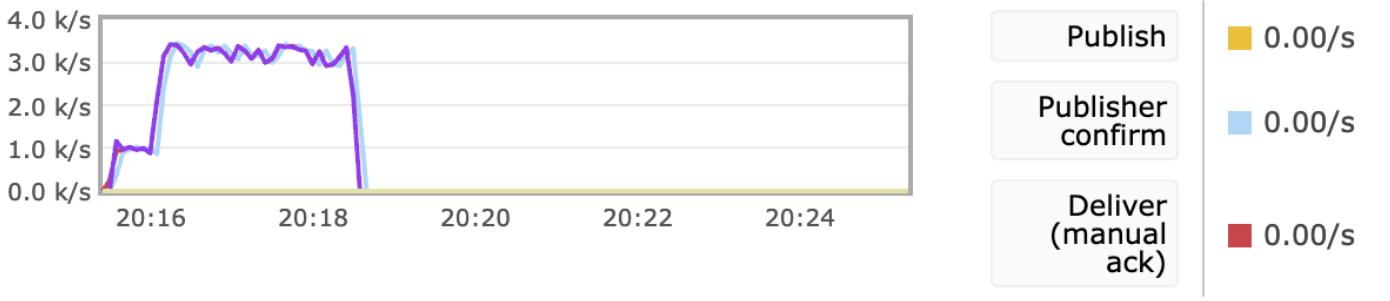
Browse
Multi source query
Graphed metrics (2)
Options
Source
Add math
Add query
Add dynamic label
Info
Statistic:
(multiple)
Period:
1 minute
Clear graph
Label
Details
Statistic
Period
Y axis
Actions
RequestCount
ApplicationELB • RequestCount • LoadBal
Sum
1 minute
TargetResponseTime
ApplicationELB • TargetResponseTime • L
Average
1 minute

Run 2: 20 workers, 04:15:31 to 04:18:35 UTC

Queued messages last ten minutes ?



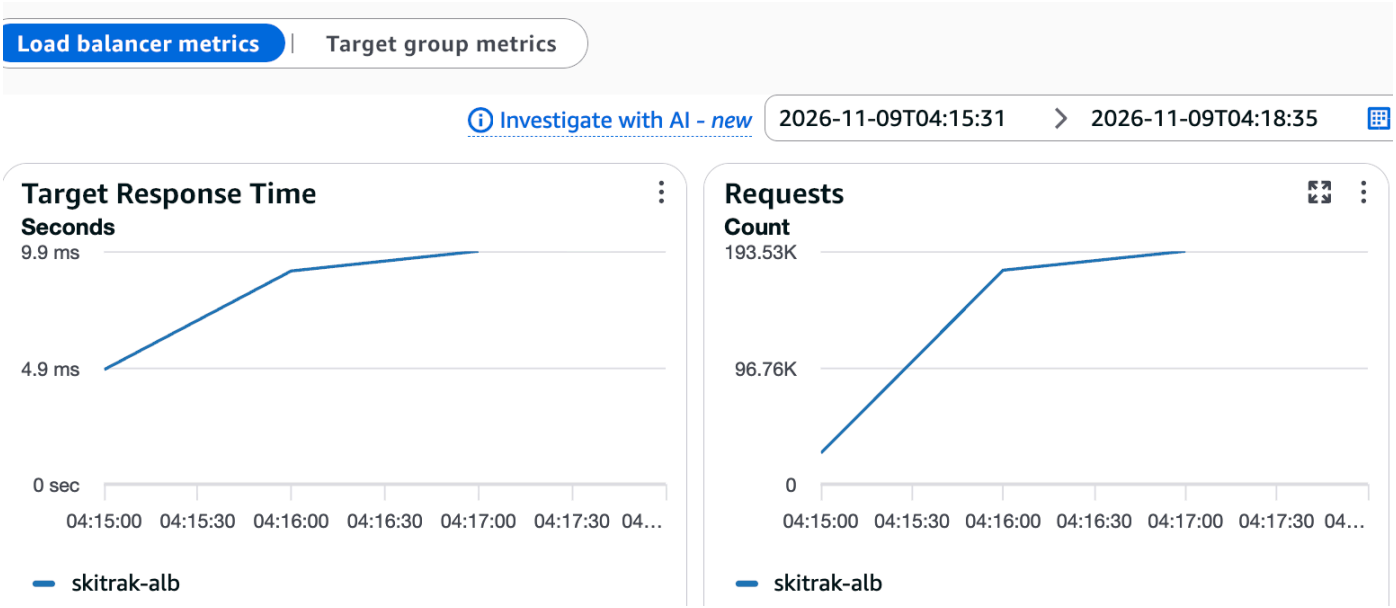
Message rates last ten minutes ?

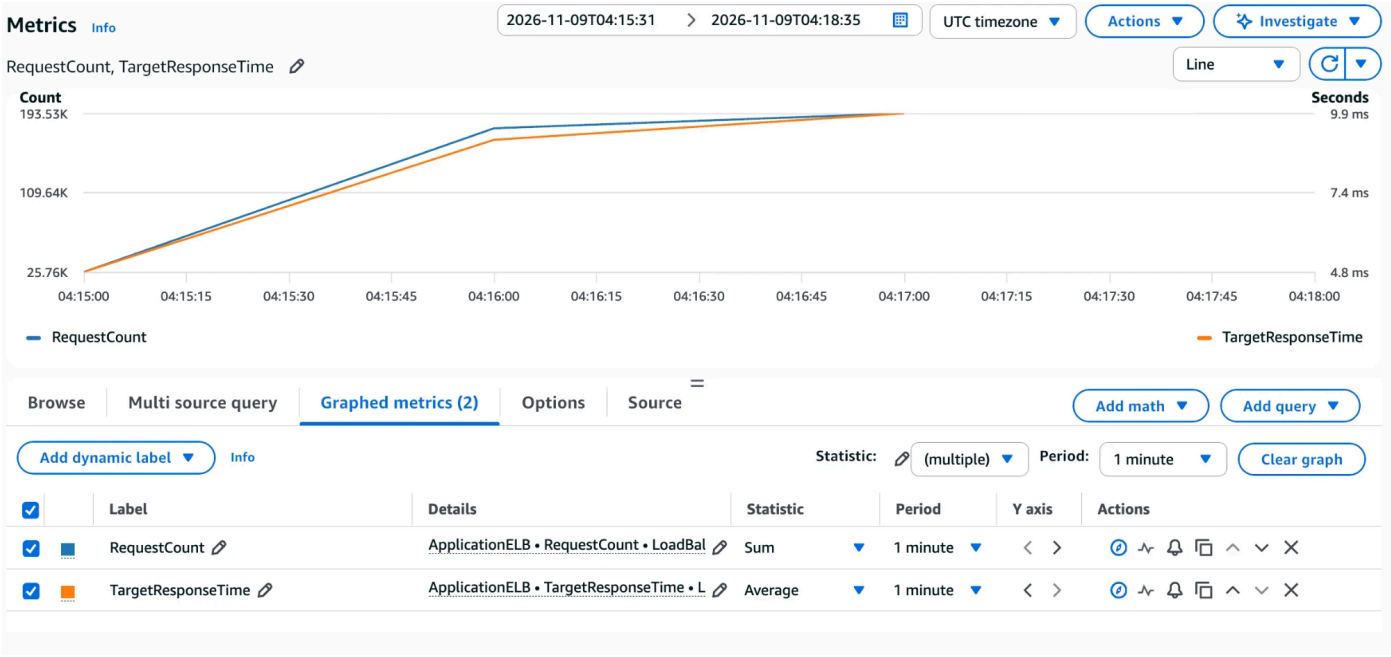


Global counts ?

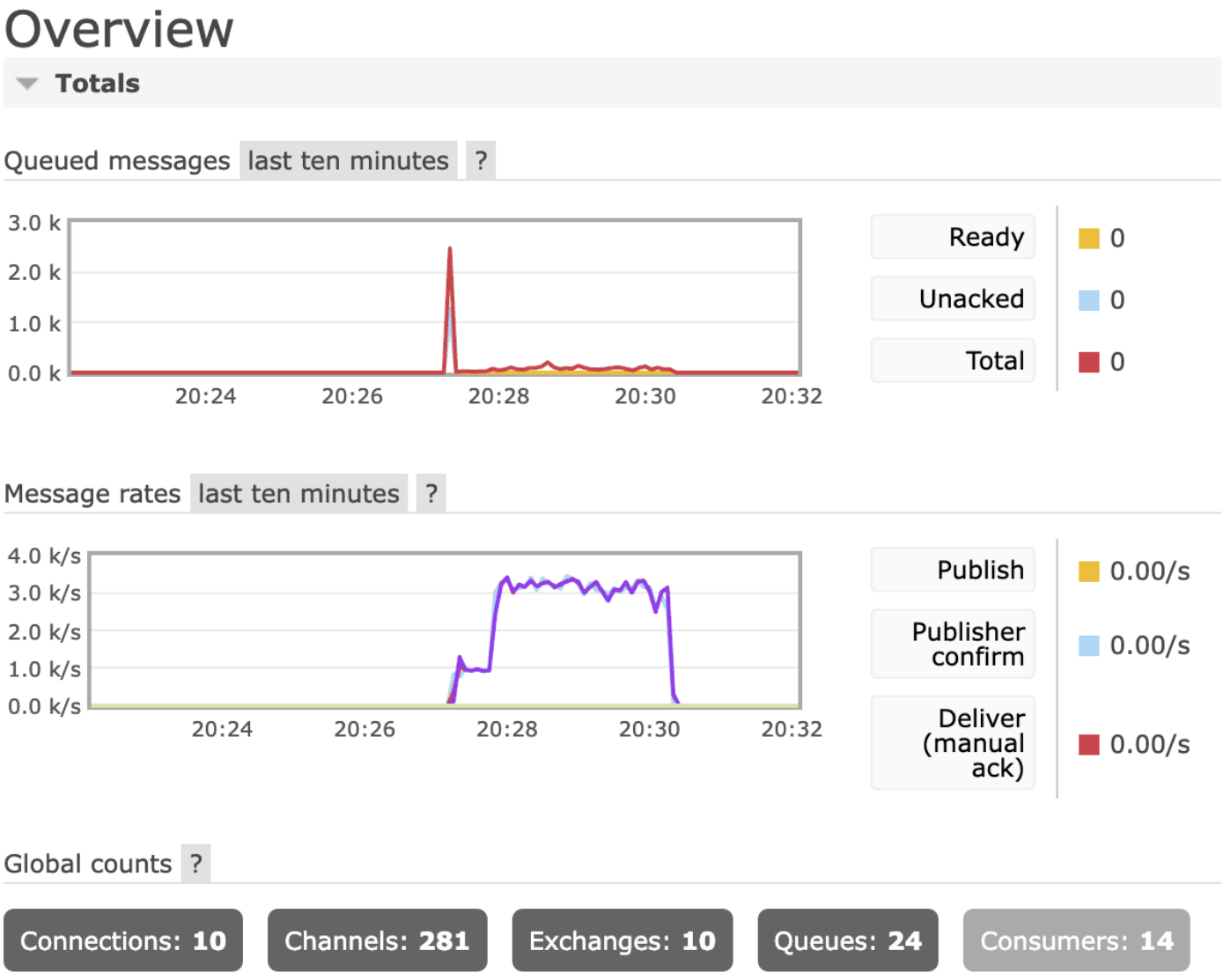
Connections: **10** Channels: **281** Exchanges: **10** Queues: **24** Consumers: **14**

RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). A short rise to about 1.5K at the start, then a few hundred (Ready peak 608).

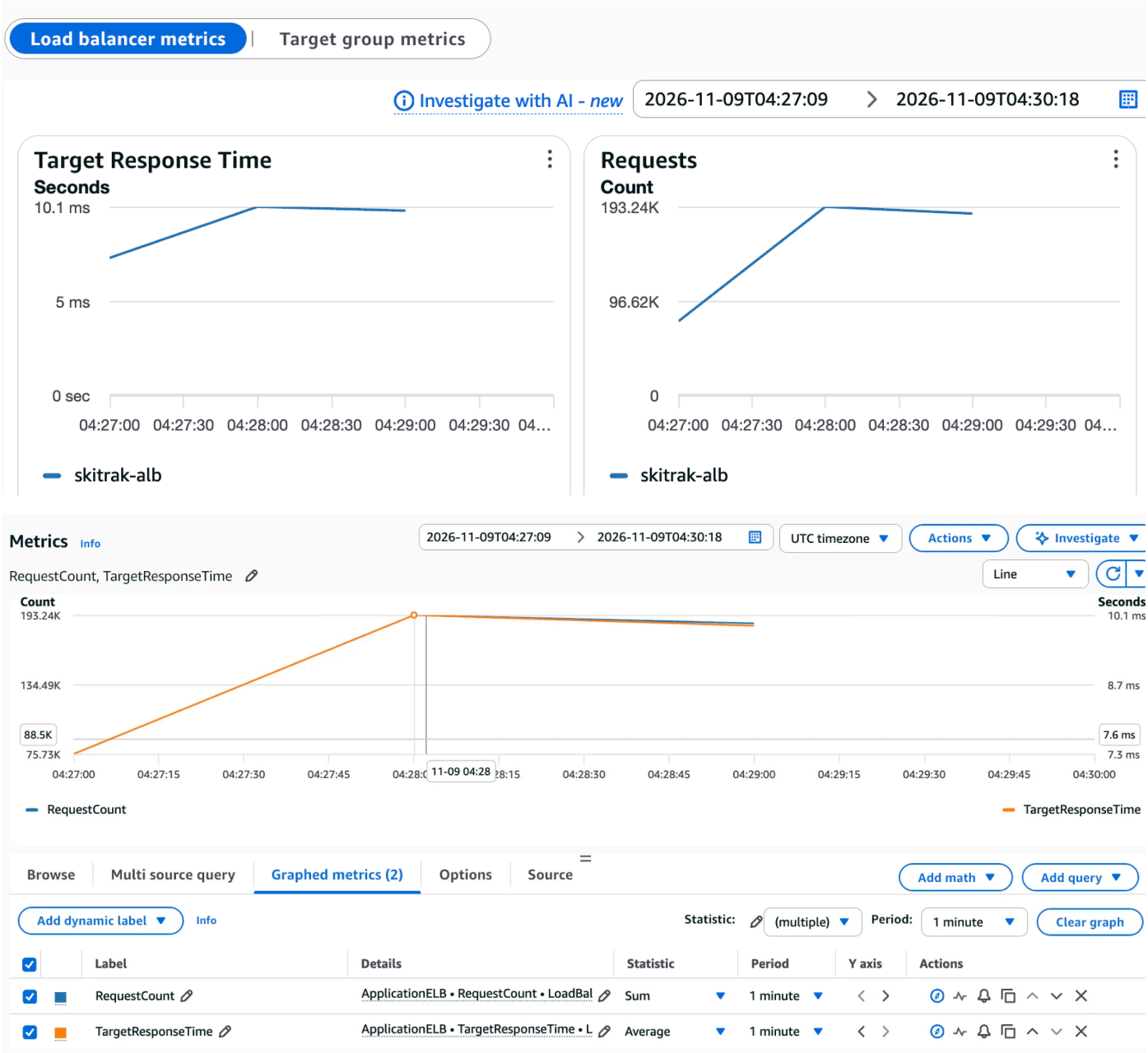




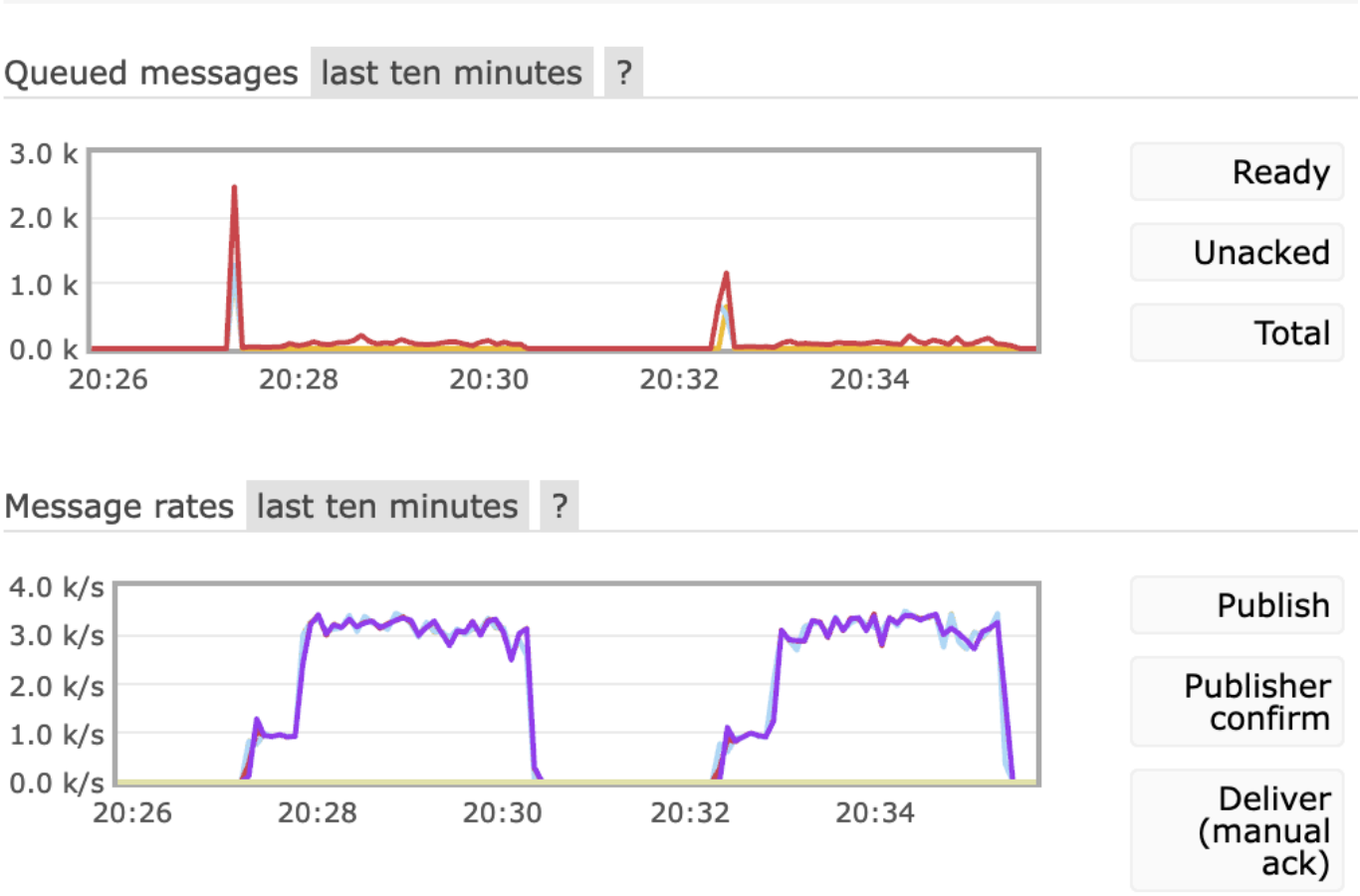
Run 3: 40 workers, 04:27:09 to 04:30:18 UTC



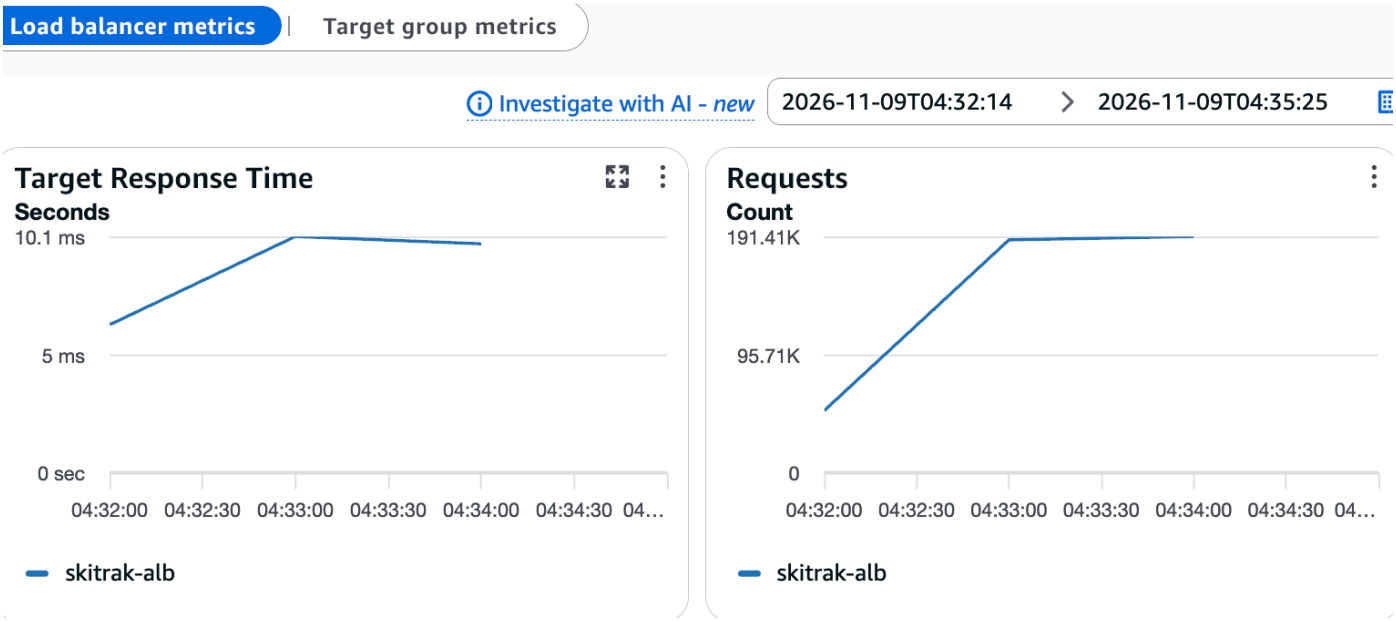
RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). A spike of about 2.5K at 20:27 at the start, then a few hundred (Ready peak 1,199).

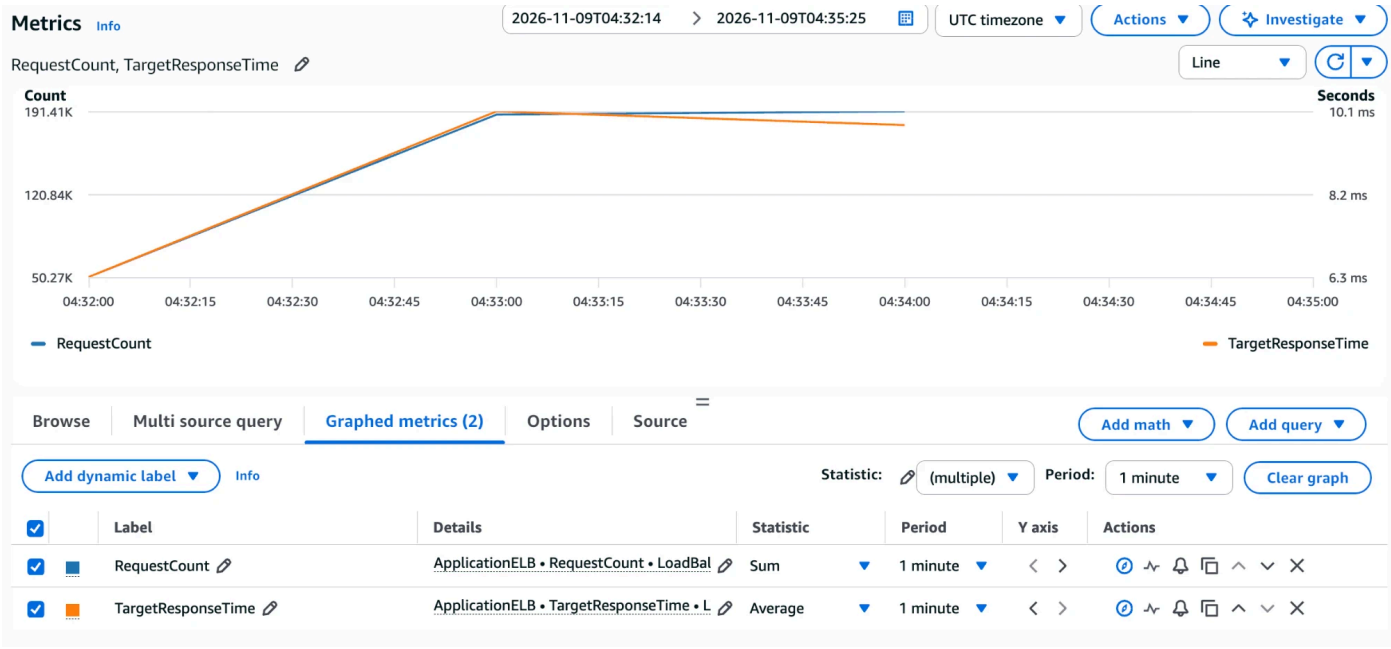


Run 4: 80 workers, 04:32:14 to 04:35:25 UTC

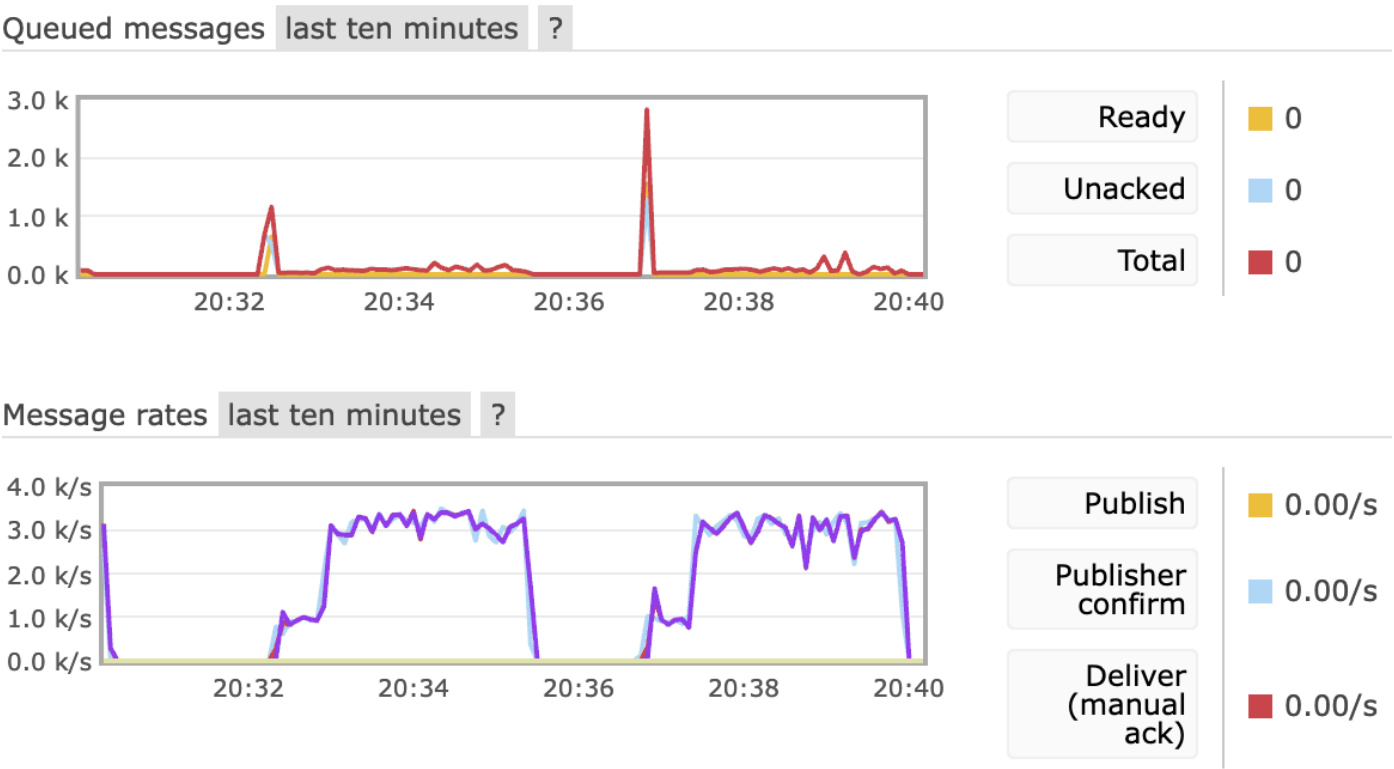


RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). The chart also holds run 3: its 2.5K spike is on the left and this run's spike of about 1.1K at 20:32 on the right.

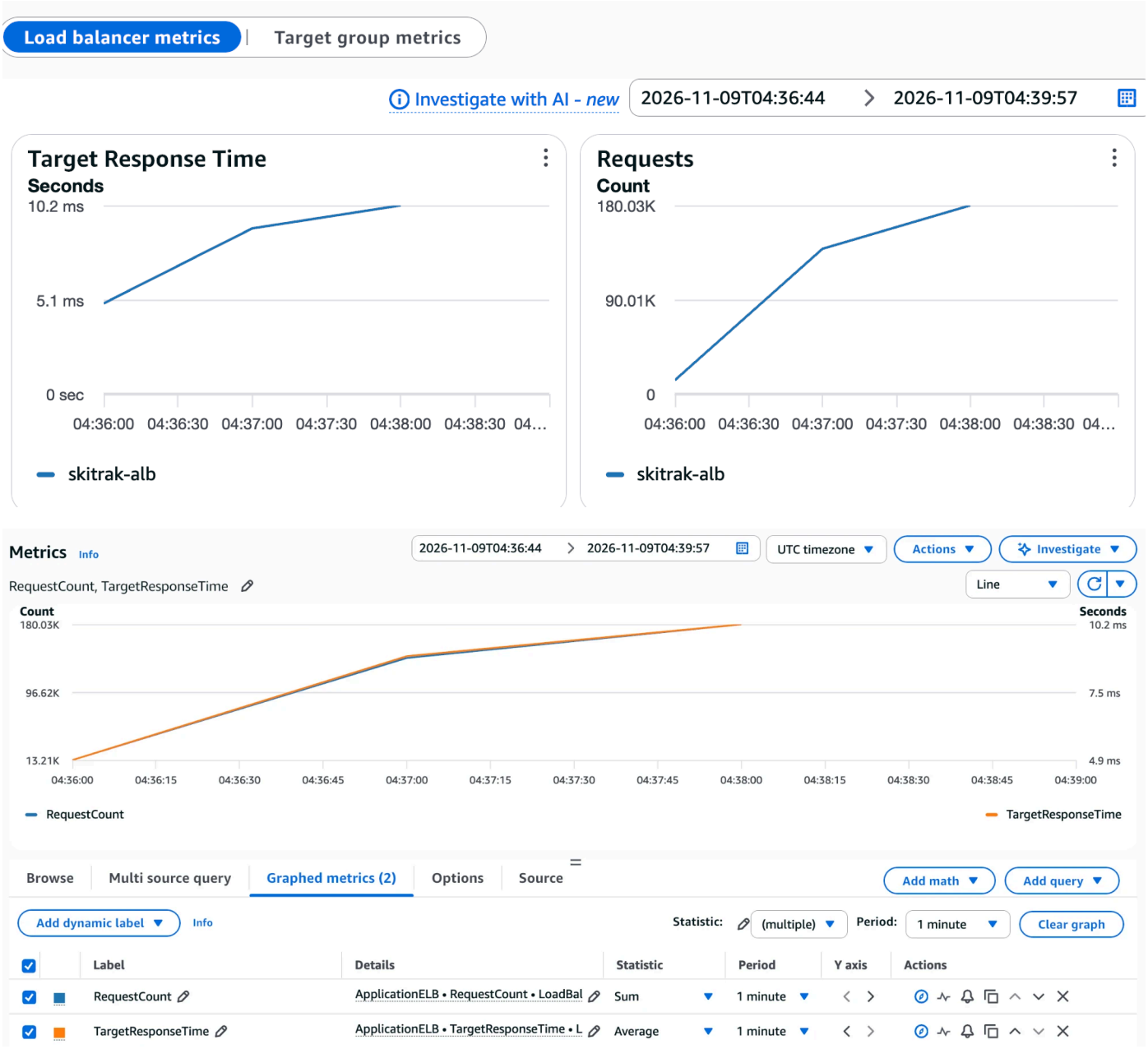




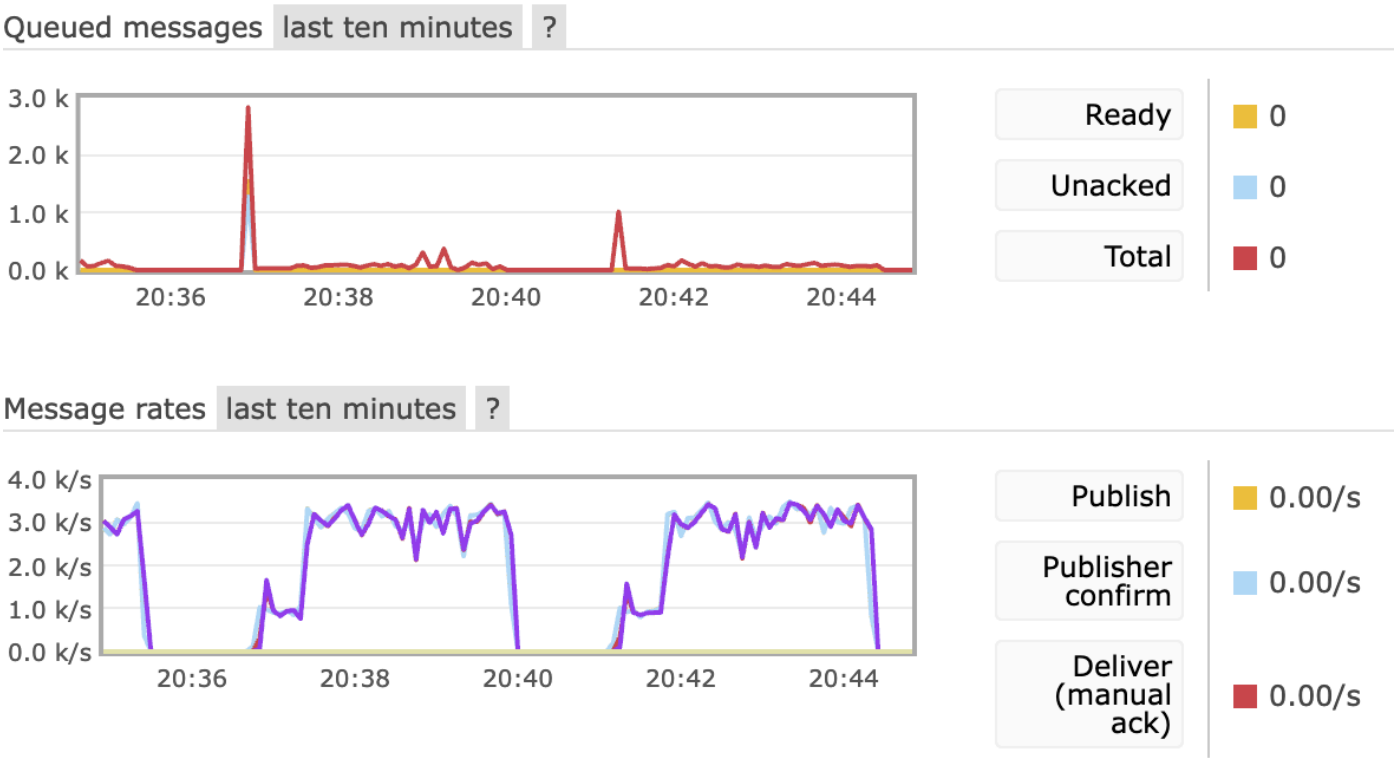
Run 5: 80 workers, 04:36:44 to 04:39:57 UTC



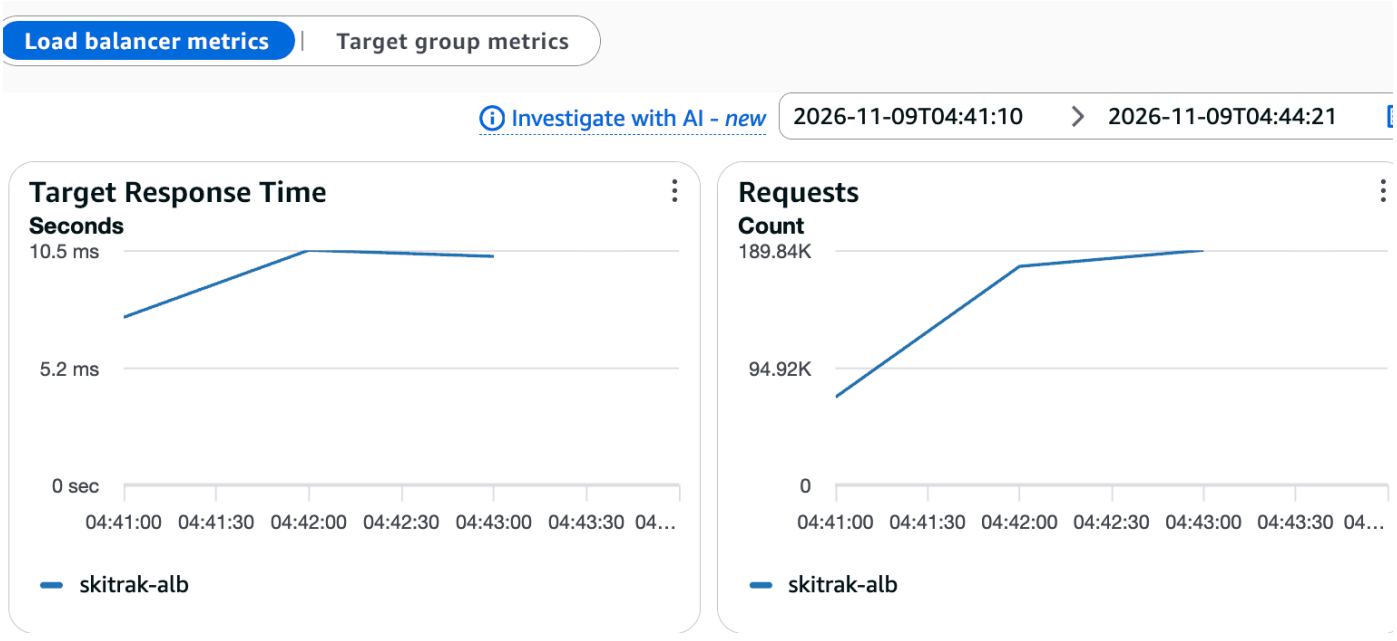
RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). Run 4's spike of about 1.1K is on the left; this run's spike of about 2.8K is at 20:37.

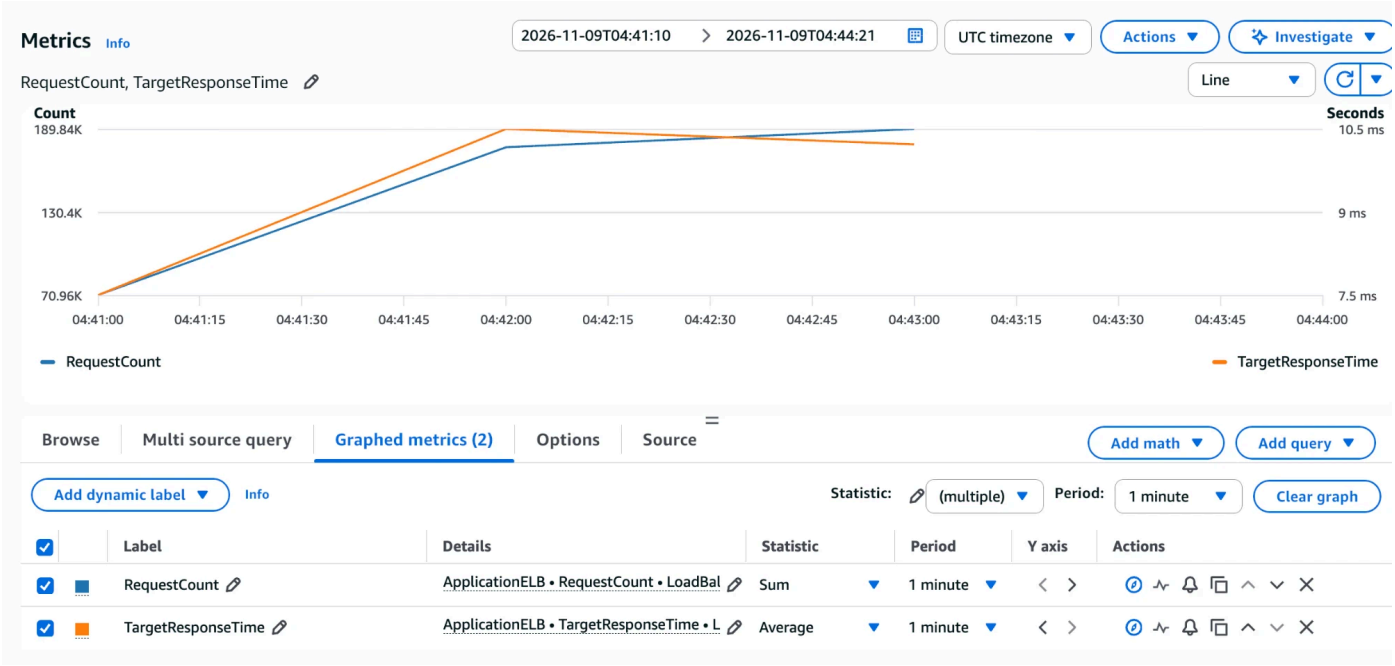

Load balancer metrics
Target group metrics
Investigate with AI - new
2026-11-09T04:36:44
2026-11-09T04:39:57
Target Response Time
Seconds
10.2 ms
5.1 ms
0 sec
04:36:00 04:36:30 04:37:00 04:37:30 04:38:00 04:38:30 04...
skitrak-alb
Requests
Count
180.03K
90.01K
0
04:36:00 04:36:30 04:37:00 04:37:30 04:38:00 04:38:30 04...
skitrak-alb
Metrics
Info
2026-11-09T04:36:44
2026-11-09T04:39:57
UTC timezone
Actions
Investigate
RequestCount, TargetResponseTime
Line
Count
180.03K
96.62K
13.21K
Seconds
10.2 ms
7.5 ms
4.9 ms
04:36:00 04:36:15 04:36:30 04:36:45 04:37:00 04:37:15 04:37:30 04:37:45 04:38:00 04:38:15 04:38:30 04:38:45 04:39:00
RequestCount
TargetResponseTime
Browse
Multi source query
Graphed metrics (2)
Options
Source
Add math
Add query
Add dynamic label
Info
Statistic:
(multiple)
Period:
1 minute
Clear graph
Label
Details
Statistic
Period
Y axis
Actions
RequestCount
ApplicationELB • RequestCount • LoadBal
Sum
1 minute
TargetResponseTime
ApplicationELB • TargetResponseTime • L
Average
1 minute

Run 6: 40 workers, 04:41:10 to 04:44:21 UTC

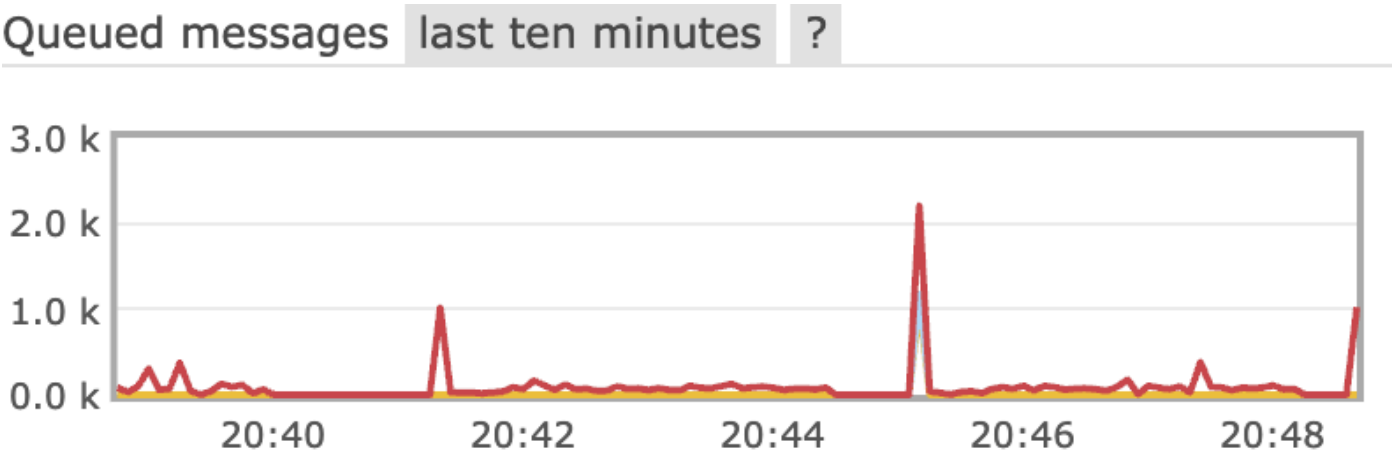


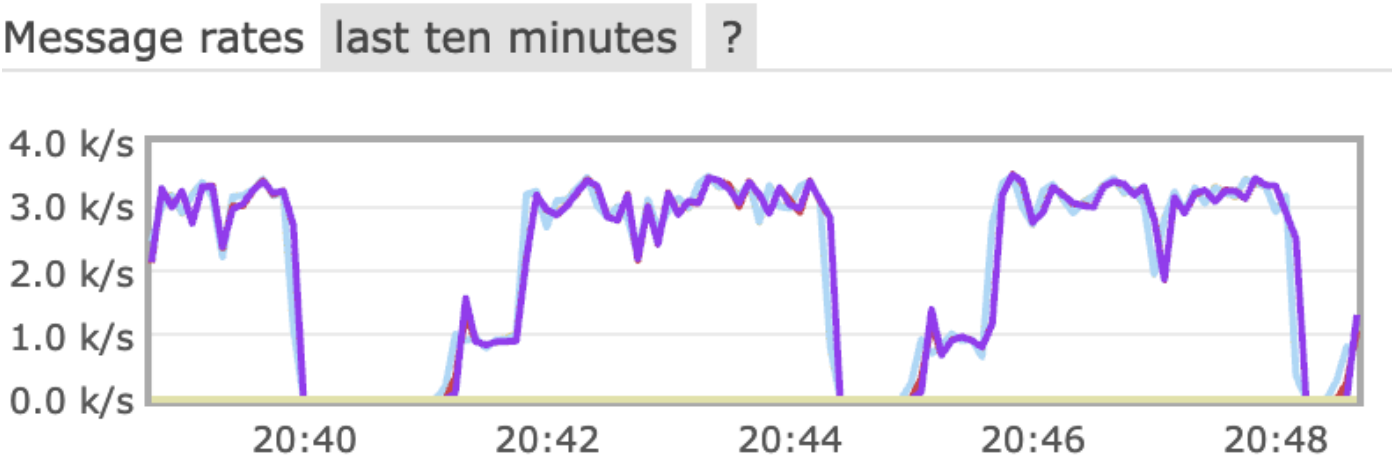
RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). Run 5's 2.8K spike is on the left; this run's spike of about 1.0K is at 20:41.



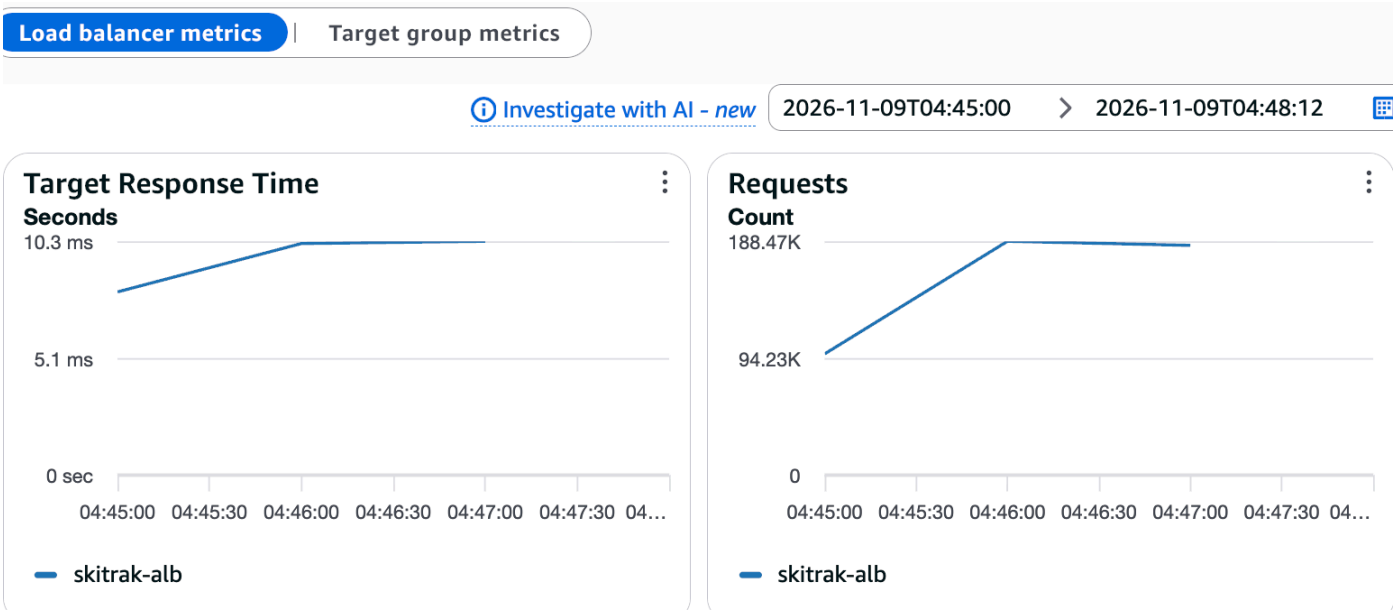


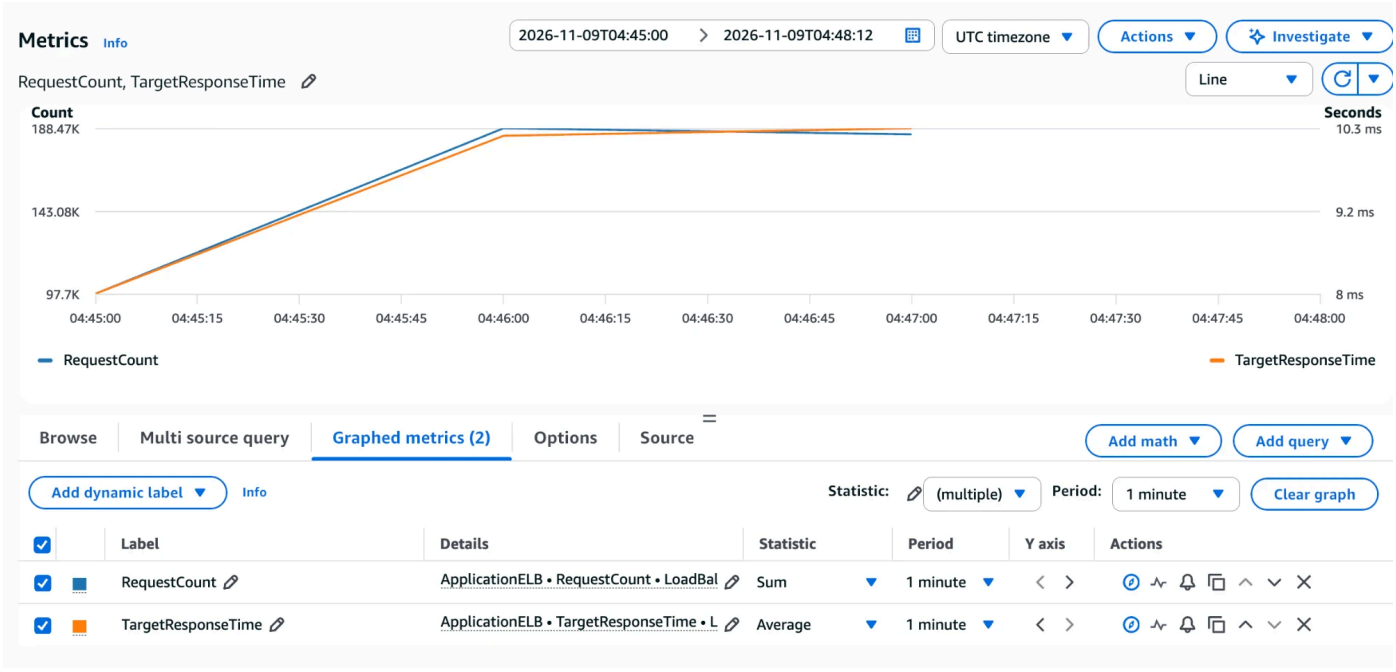
Run 7: 20 workers, 04:45:00 to 04:48:12 UTC



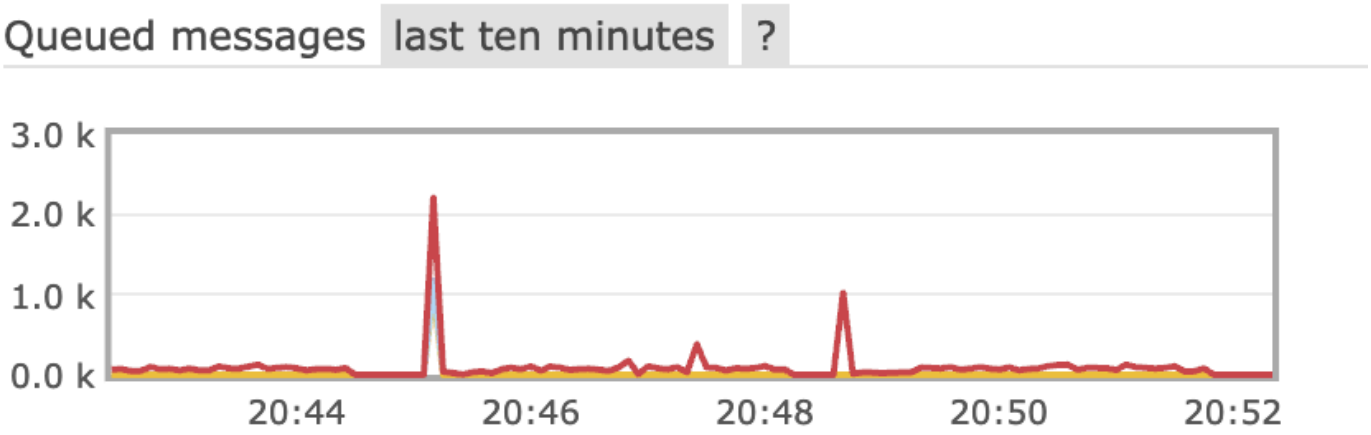


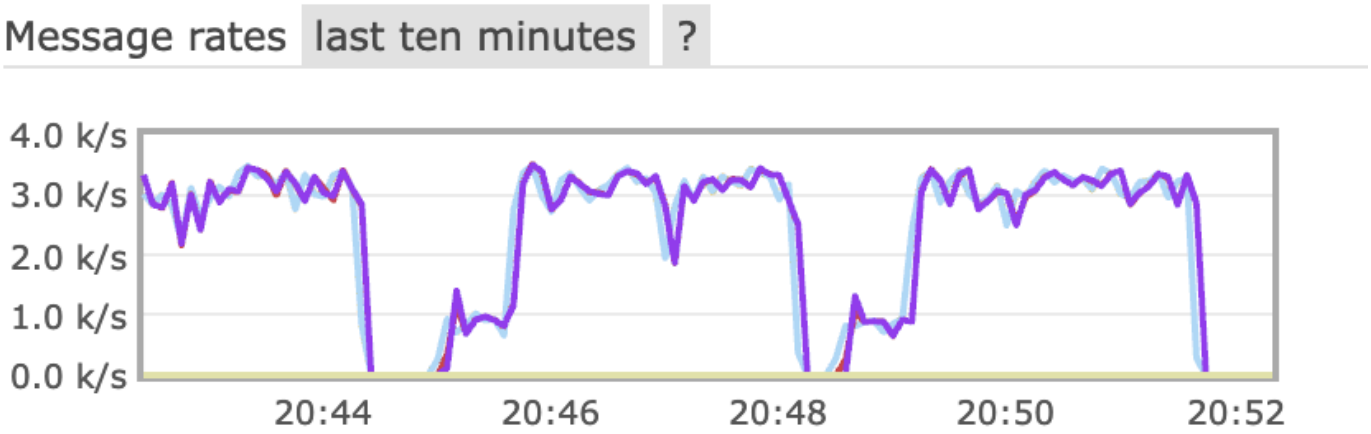
RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). Run 6's spike (about 1.0K) is on the left; this run's spike of about 2.2K is at 20:45, and run 8 is starting at the right edge.



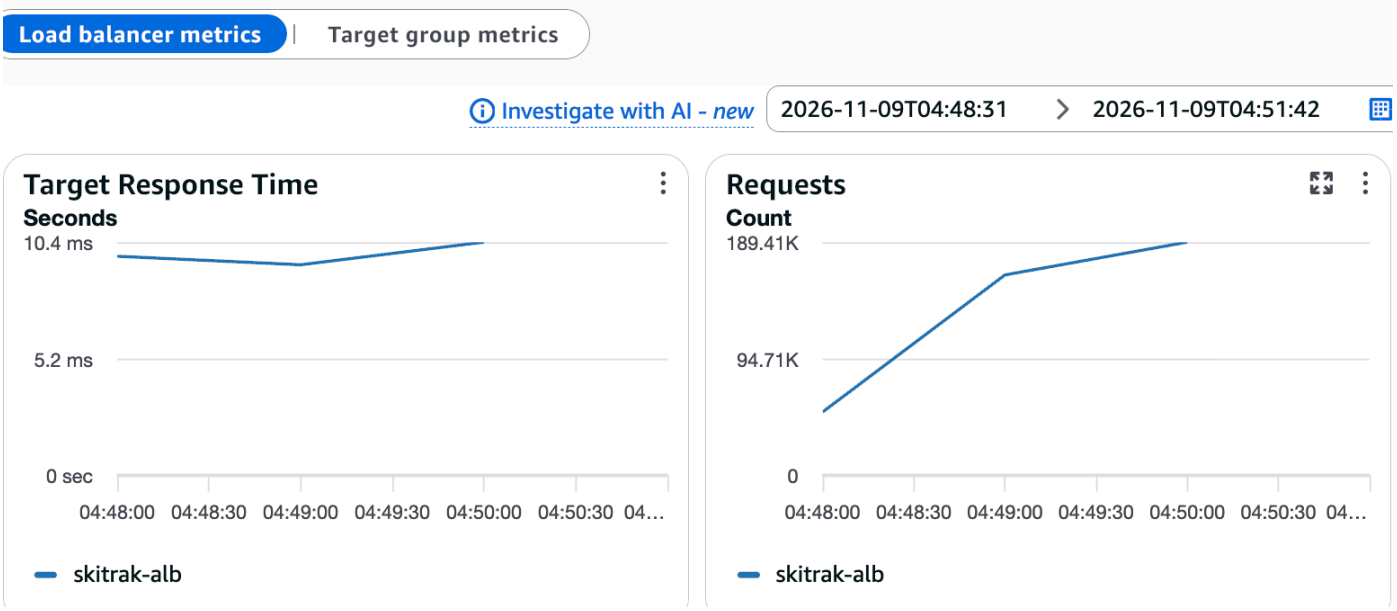


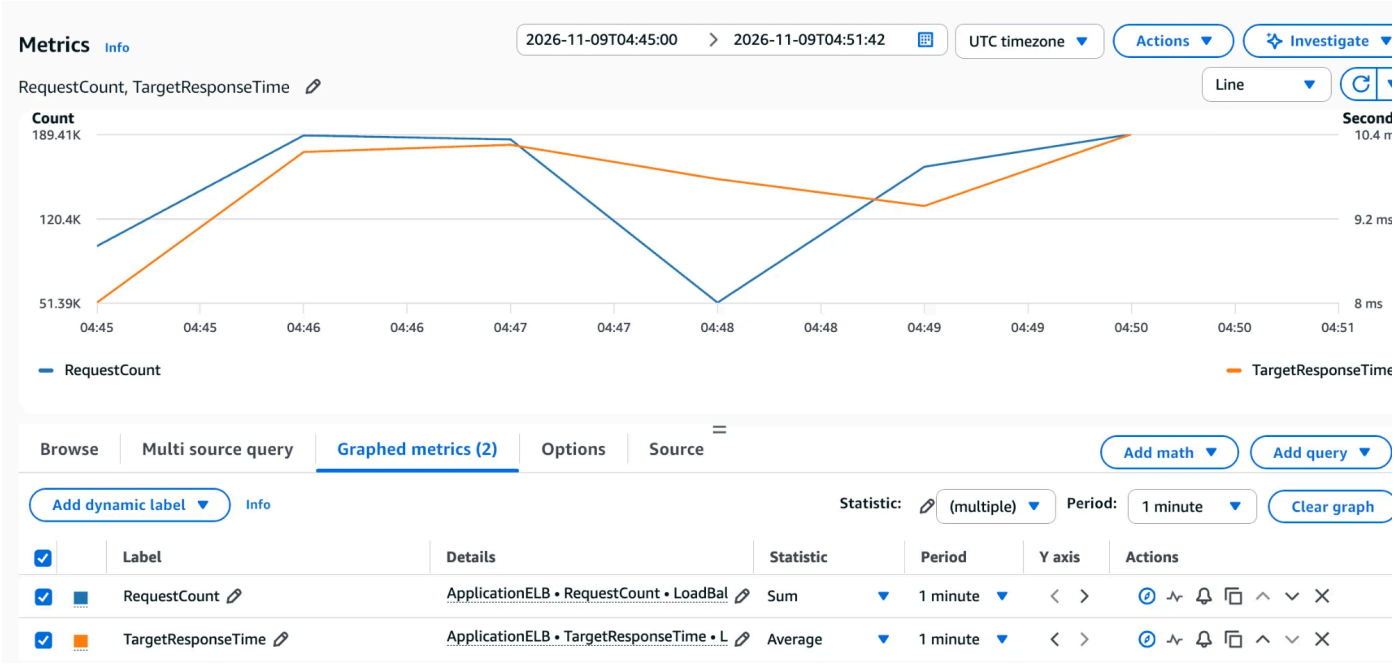
Run 8: 10 workers, 04:48:31 to 04:51:42 UTC





RabbitMQ Overview, last ten minutes (axis in the browser's local time, UTC-8, so 20:06 is 04:06 UTC); totals are Ready plus Unacked and Unacked alone can reach 1,280 (ten queues times prefetch 128). Run 7's 2.2K spike is on the left; this run's spike of about 1.0K is at 20:48.





This RequestCount and TargetResponseTime chart covers 04:45:00 to 04:51:42 UTC, that is runs 7 and 8 together, not run 8 alone; the dip near 04:48 is the interval between the two runs.

Per-run evidence

All queue statistics below cover only raw resort.1 through resort.10 queues. UI Overview charts include other queues as well. Means are arithmetic sample means over the client window including warmup and drain. Broker delivery rate is transport delivery, not necessarily committed aggregation. Consumer lag is the run-specific mean obtained from cumulative sums/counts; it does not establish a bound on every event. Network rates use physical interfaces to avoid Docker bridge double counting.

RabbitMQ Consumers counts AMQP subscriptions, not Java worker threads: ten raw-queue subscriptions plus WebSocket backplane subscriptions can remain present with only five aggregation workers. Worker counts are verified from Consumer configuredThreads/coreThreads/dynamicScaling metrics. Additional drain time includes final WebSocket verification, not only queue processing. The client output Remaining in queue refers to its local send-work queue, not RabbitMQ depth.

instrumented-500000-c64-20261108T135722Z

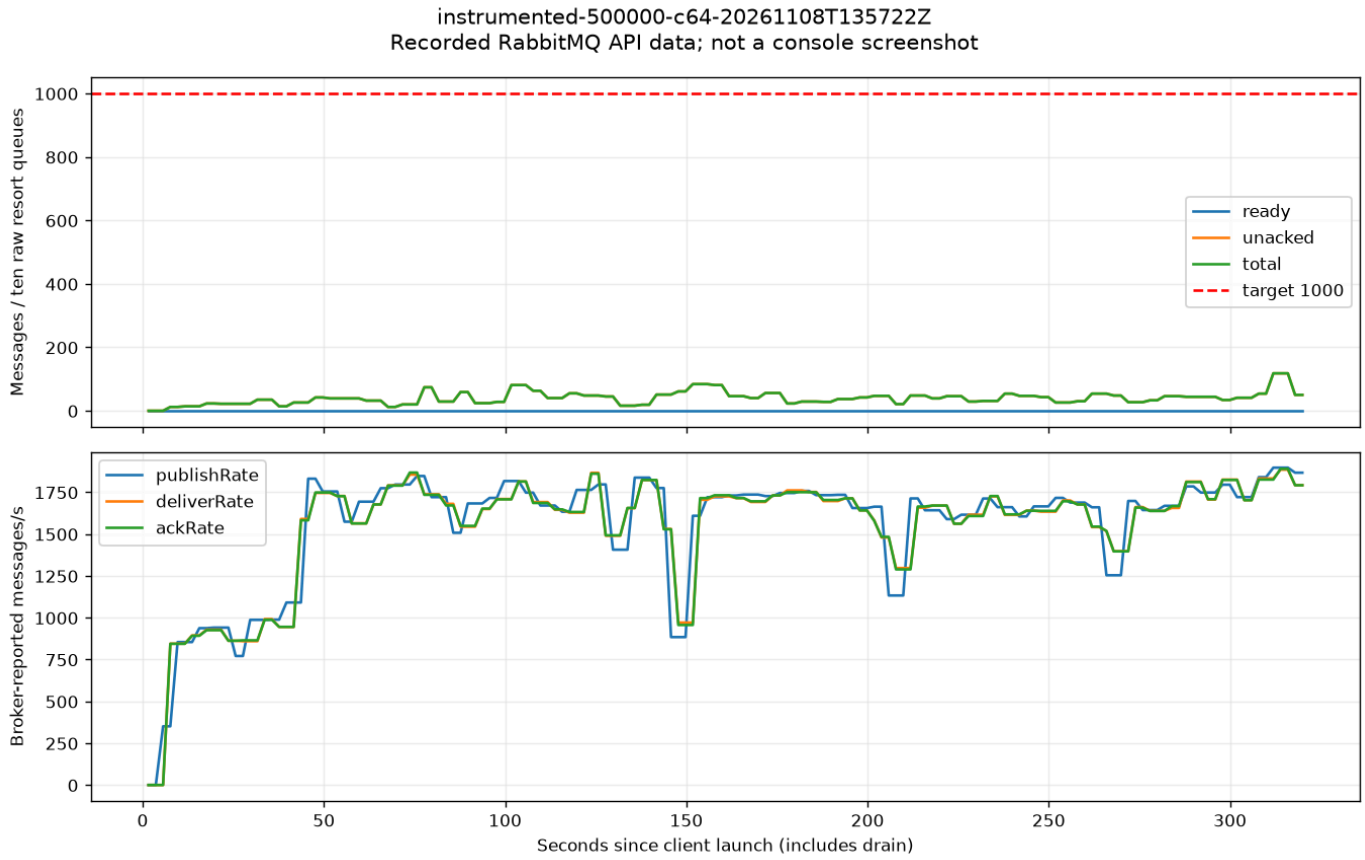
UTC: 2026-11-08T13:57:39.954246+00:00 to 2026-11-08T14:03:01.243264+00:00. Ingests=1; clientThreads=64; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

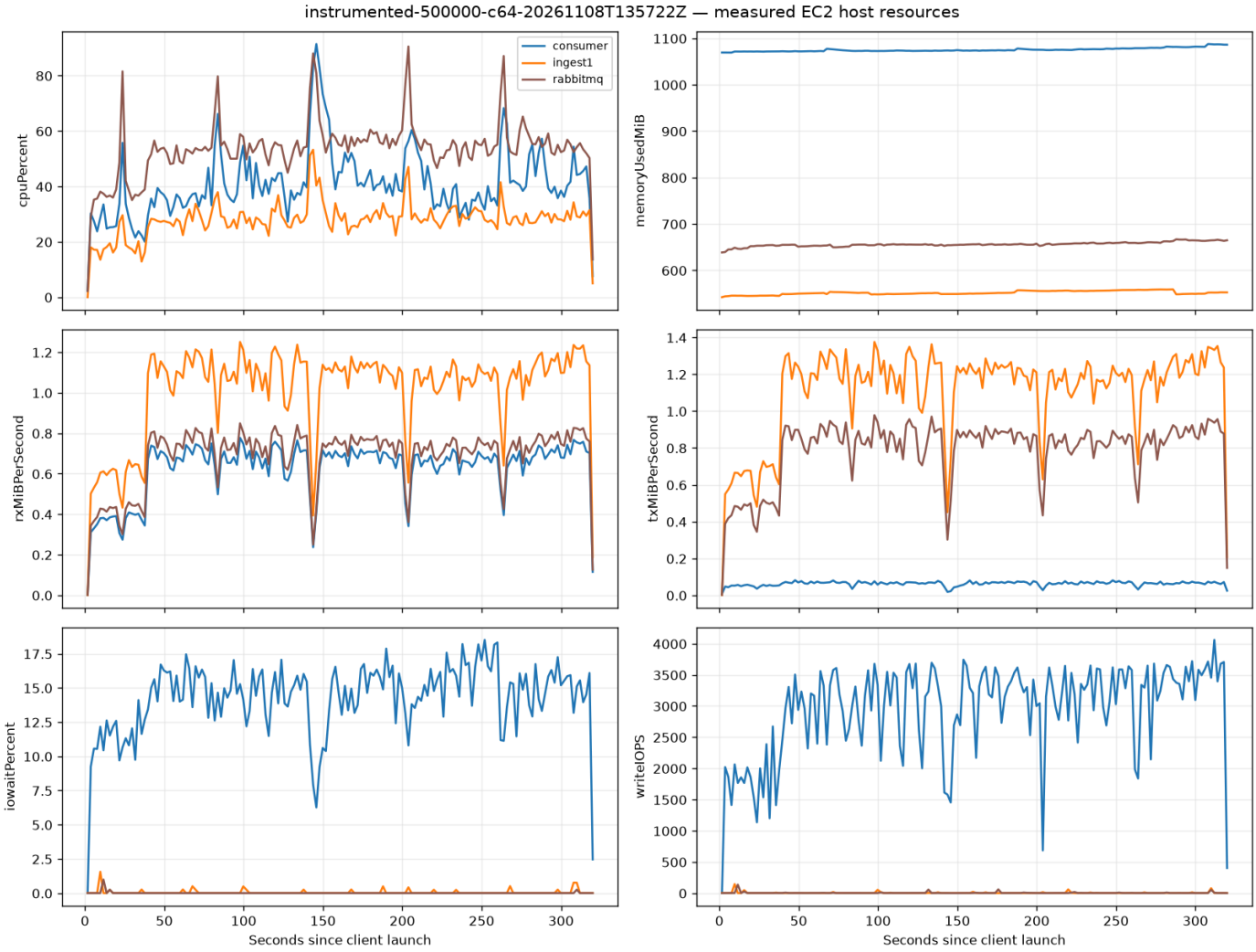
Complete client output I Full metrics, provenance and limits I Evidence page

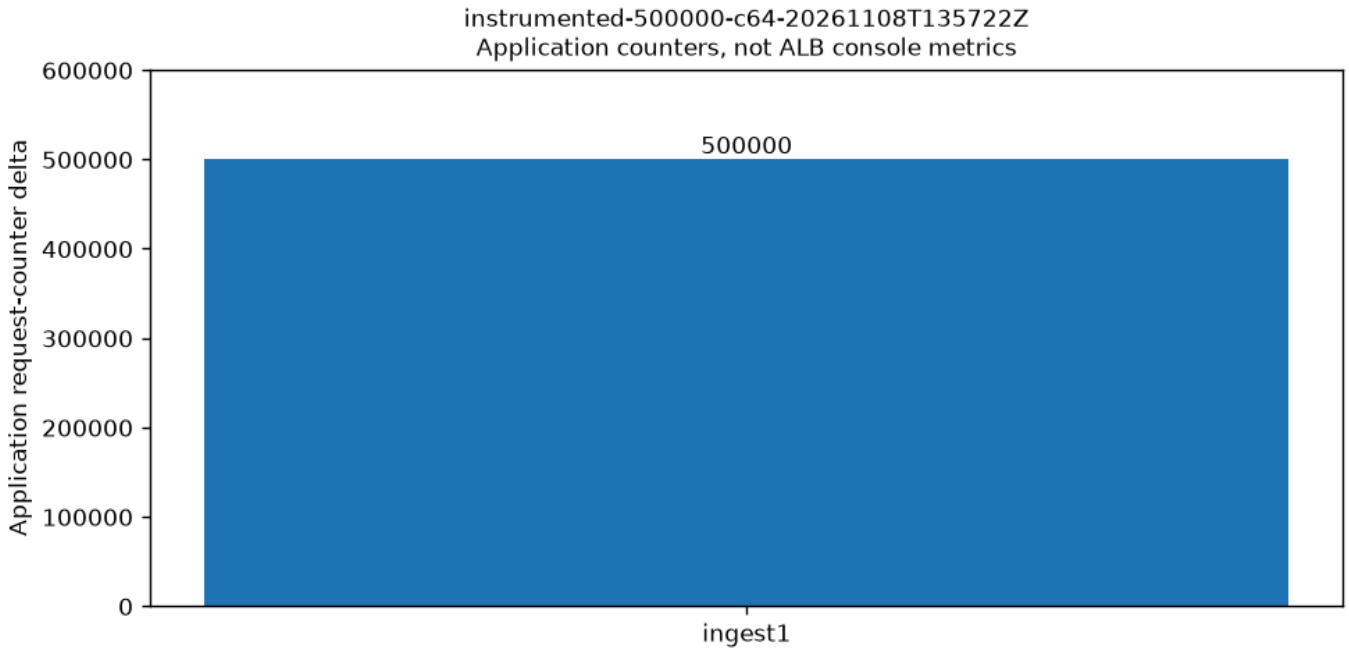
Measurement	Result
Ingestion runtime seconds	316.295
Program runtime seconds	319.491
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	38.18 / 51.73 / 93.47
Ready sampled mean / peak	0.00 / 0
Ready + Unacked peak	118
Mean broker publish / delivery / ACK rates per second	1,552.24 / 1,541.85 / 1,541.61

Measurement	Result
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	911 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

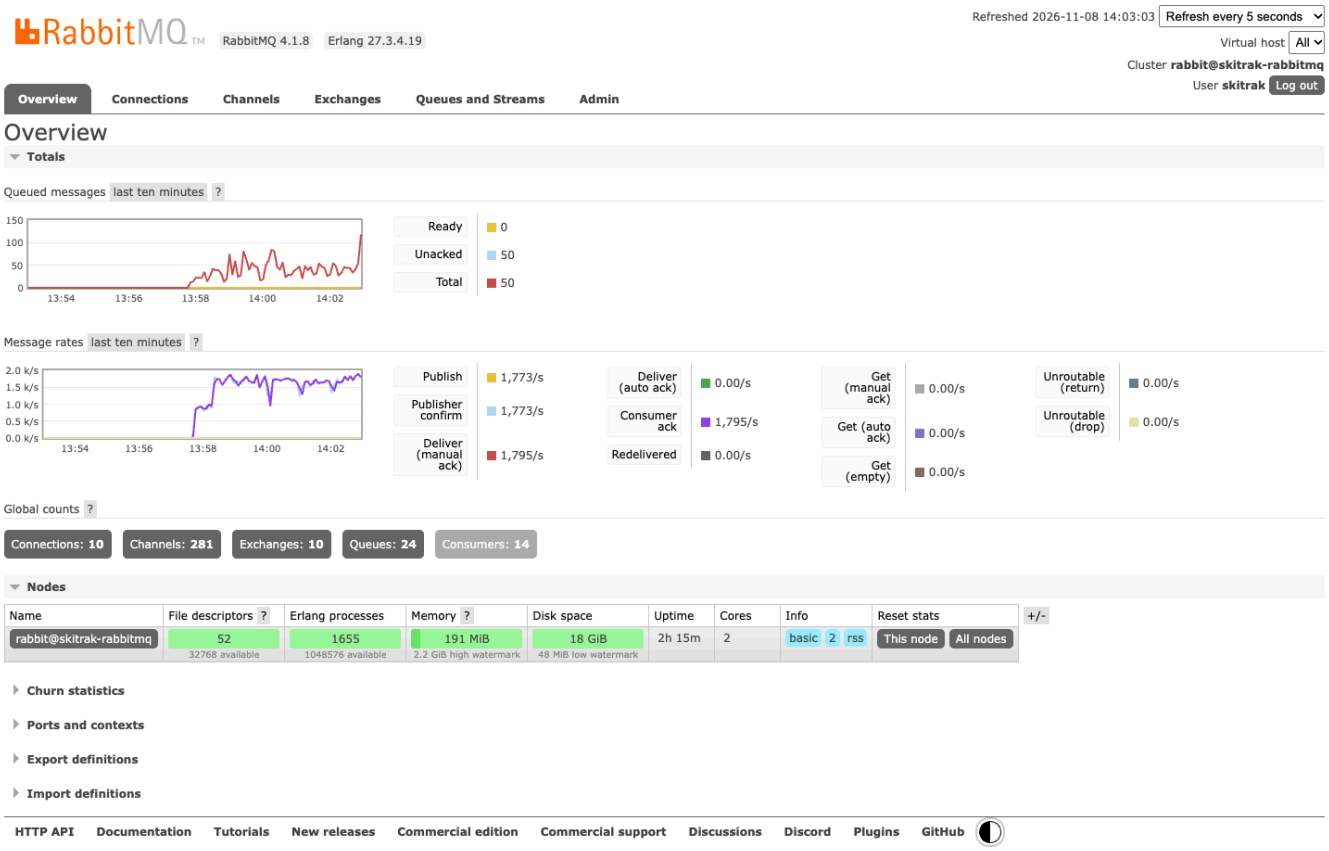




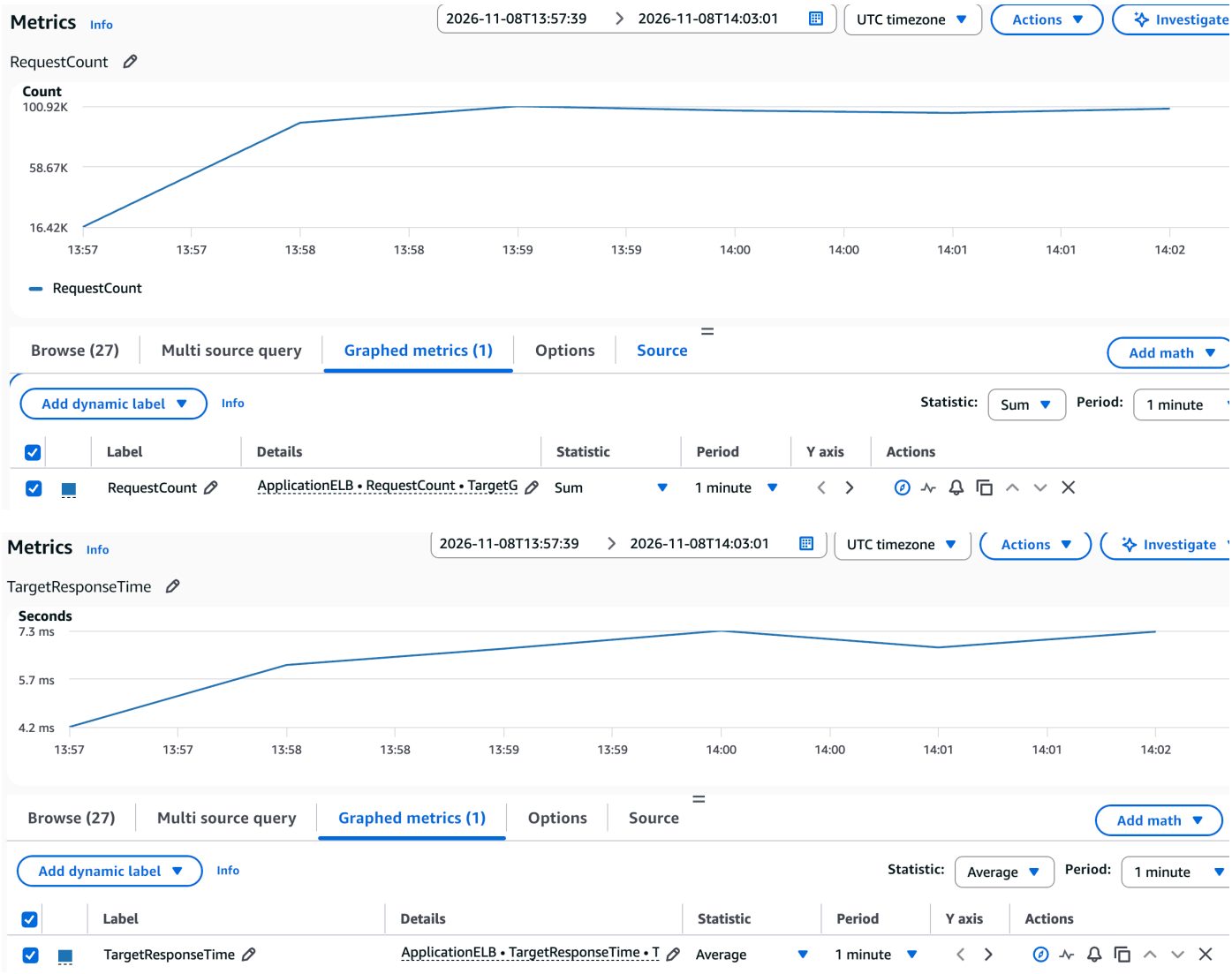


Actual RabbitMQ console captures:

- rabbitmq-20261108T135739900660Z.png
- rabbitmq-20261108T135825406110Z.png
- rabbitmq-20261108T135910635997Z.png
- rabbitmq-20261108T135956010112Z.png
- rabbitmq-20261108T140041357168Z.png
- rabbitmq-20261108T140126802819Z.png
- rabbitmq-20261108T140212310039Z.png
- rabbitmq-20261108T140257736422Z.png
- rabbitmq-20261108T140307393941Z-final.png



Supplied AWS CloudWatch captures:



Collection gaps: None detected in the recorded collectors.

instrumented-500000-c128-20261108T140311Z

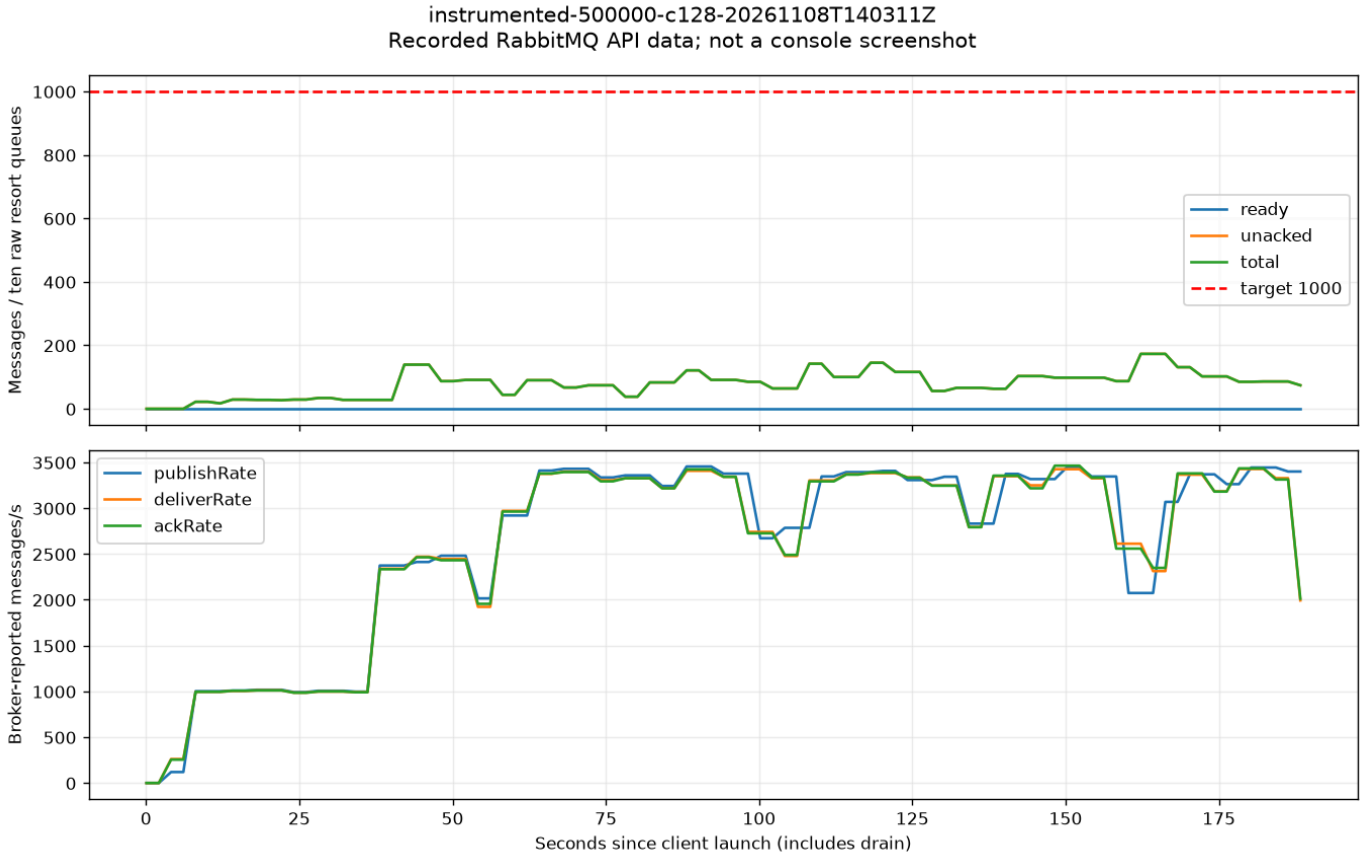
UTC: 2026-11-08T14:03:24.169506+00:00 to 2026-11-08T14:06:32.887825+00:00. Ingests=1; clientThreads=128; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

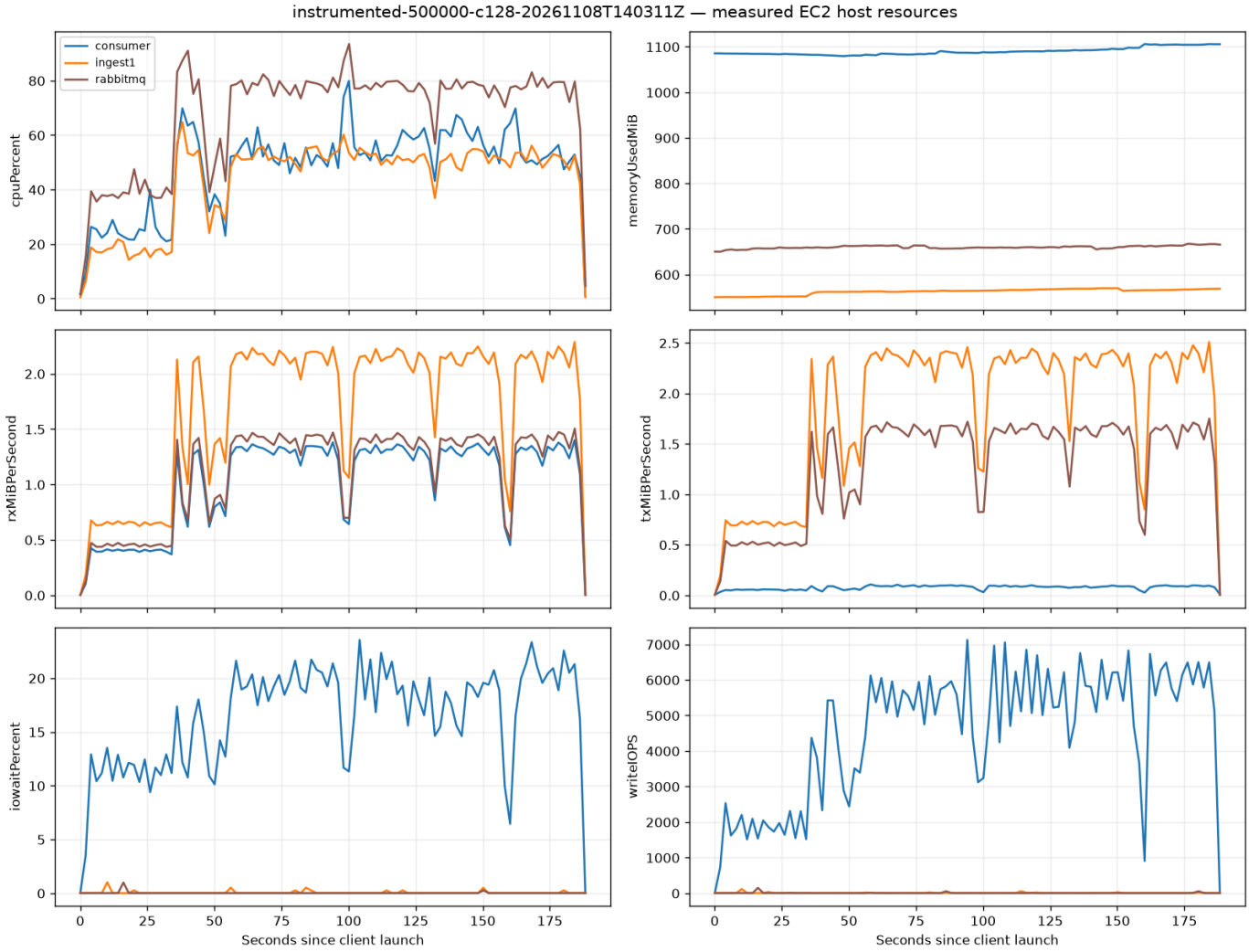
Complete client output | Full metrics, provenance and limits | Evidence page

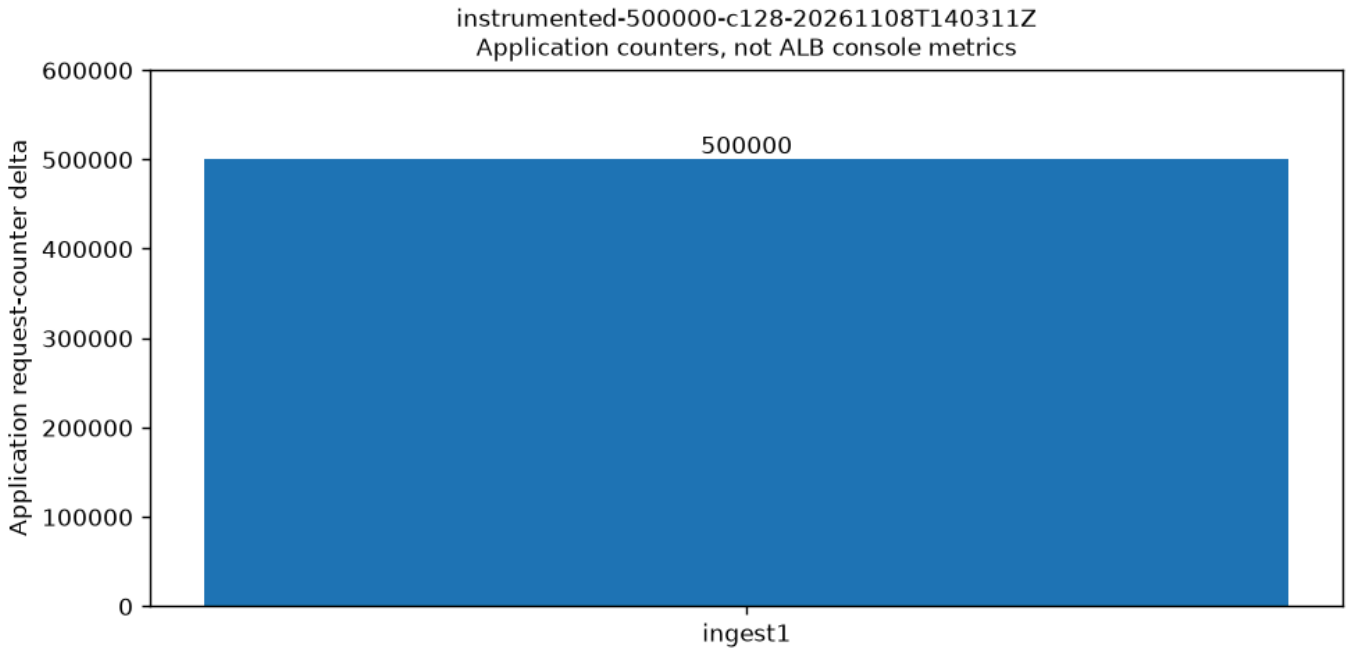
Measurement	Result
Ingestion runtime seconds	184.429
Program runtime seconds	187.473
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	40.94 / 56.09 / 134.59
Ready sampled mean / peak	0.00 / 0
Ready + Unacked peak	173
Mean broker publish / delivery / ACK rates per second	2,637.75 / 2,607.96 / 2,607.50
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0

Measurement	Result
WebSocket updates / reconnections	911 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

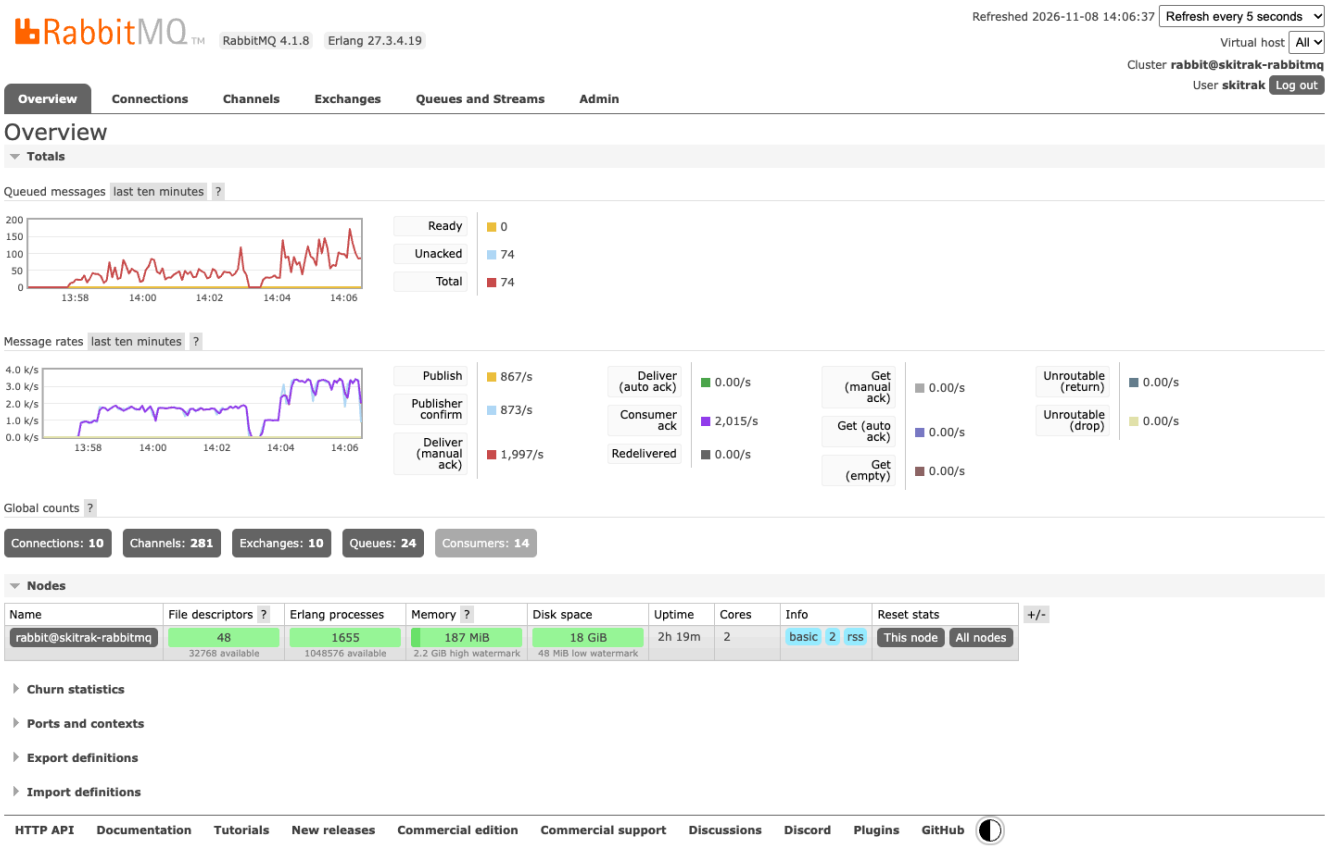




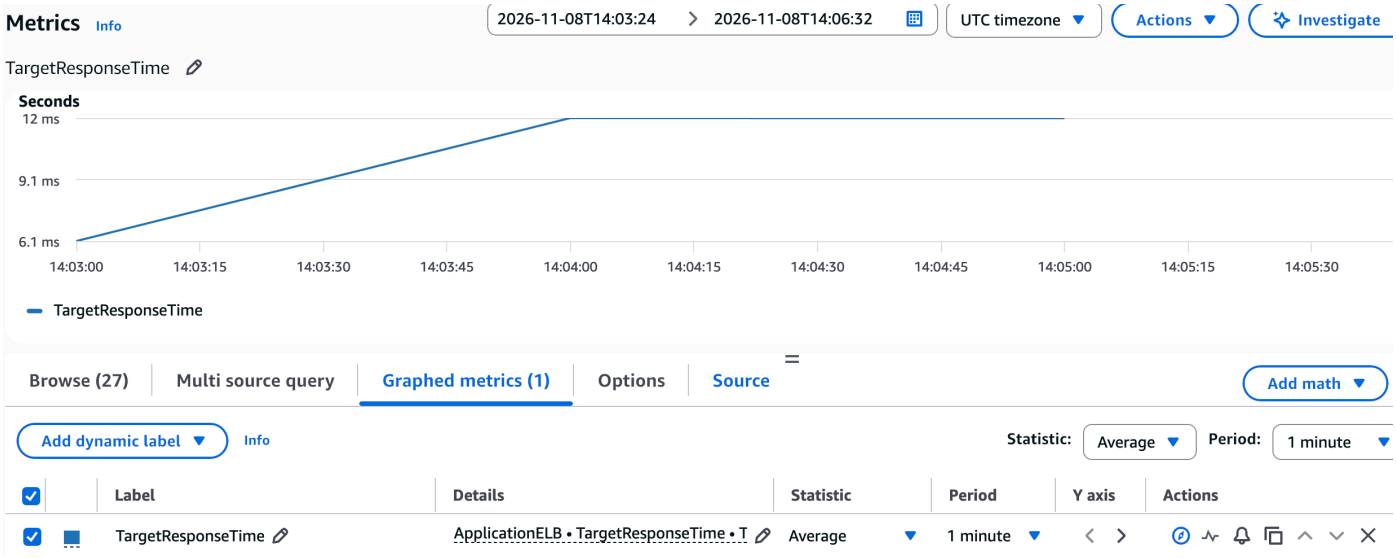


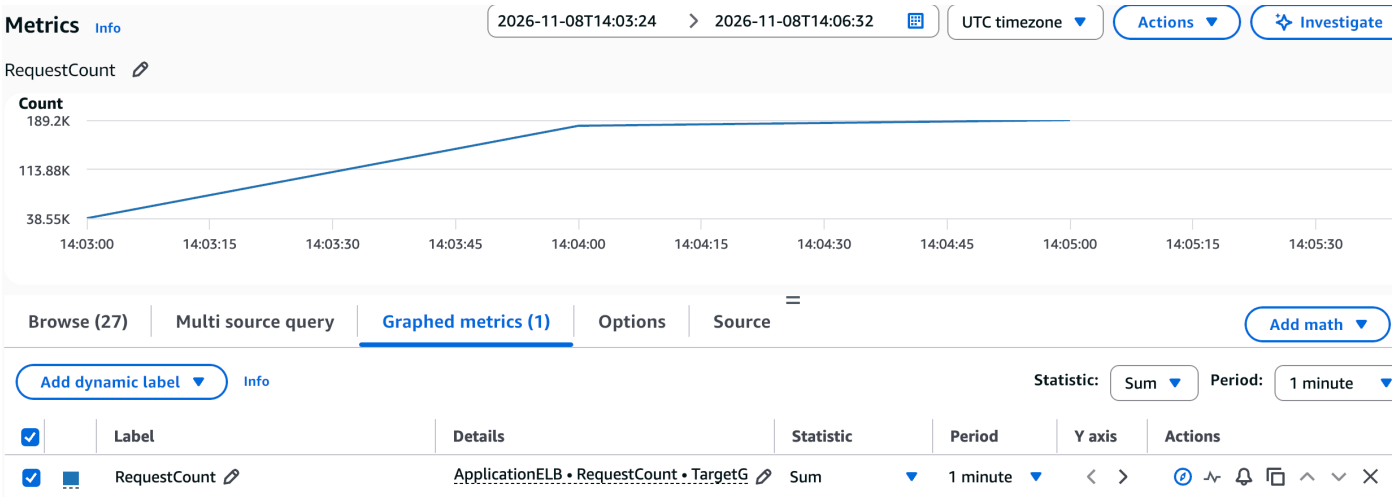
Actual RabbitMQ console captures:

- rabbitmq-20261108T140324010140Z.png
- rabbitmq-20261108T140409444633Z.png
- rabbitmq-20261108T140454846661Z.png
- rabbitmq-20261108T140540312026Z.png
- rabbitmq-20261108T140625886271Z.png
- rabbitmq-20261108T140639050709Z-final.png



Supplied AWS CloudWatch captures:





Collection gaps: None detected in the recorded collectors.

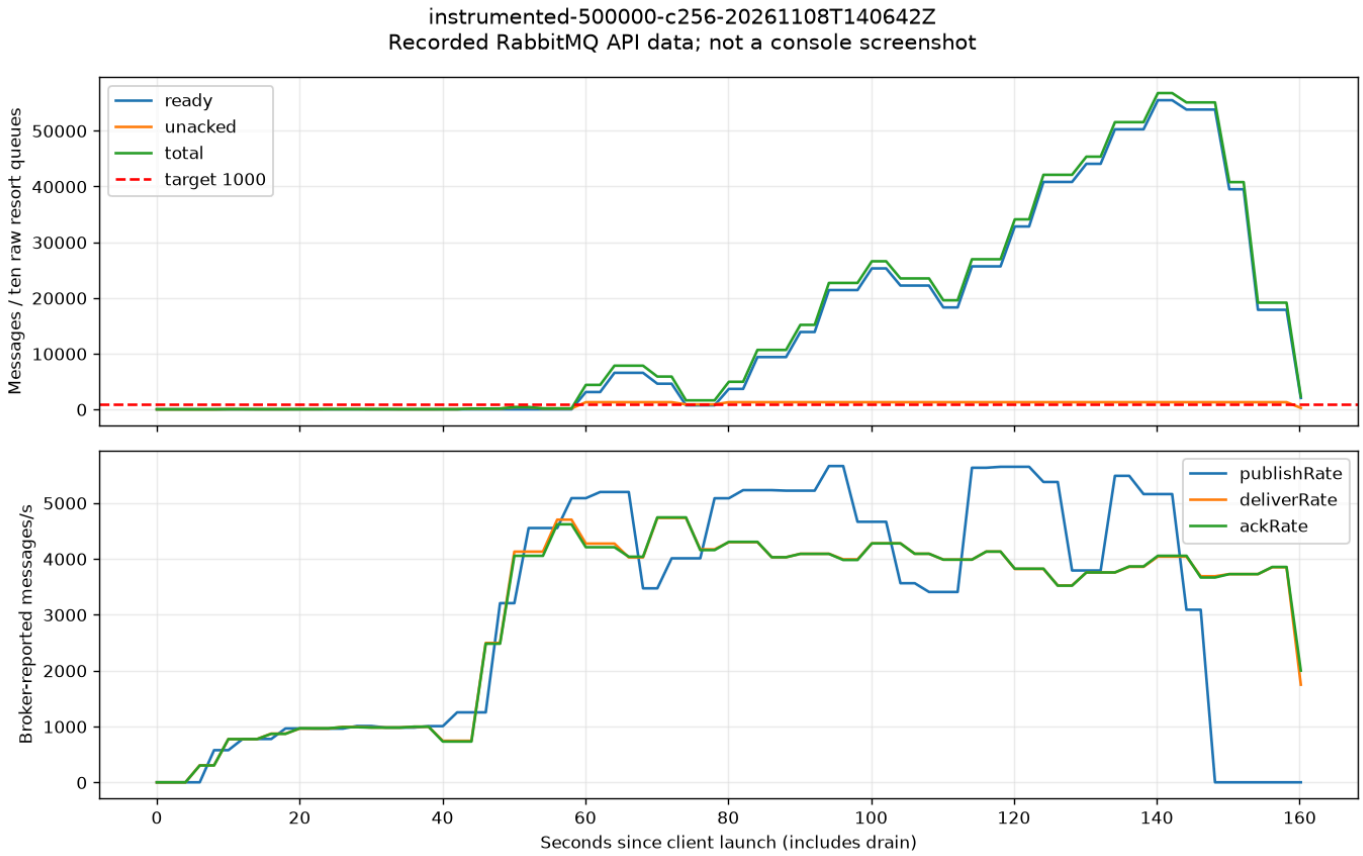
instrumented-500000-c256-20261108T140642Z

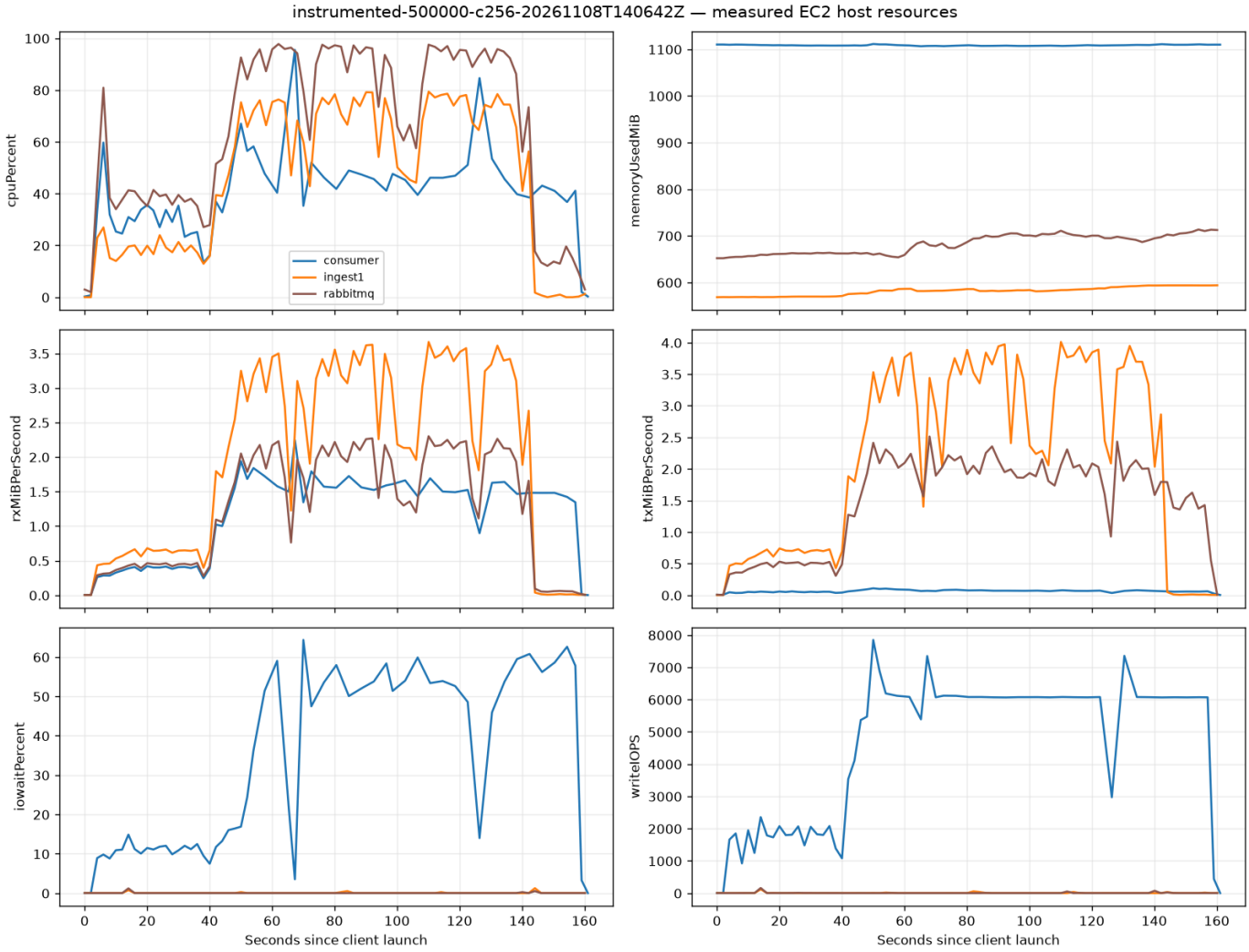
UTC: 2026-11-08T14:06:57.686350+00:00 to 2026-11-08T14:09:38.884009+00:00. Ingests=1; clientThreads=256; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

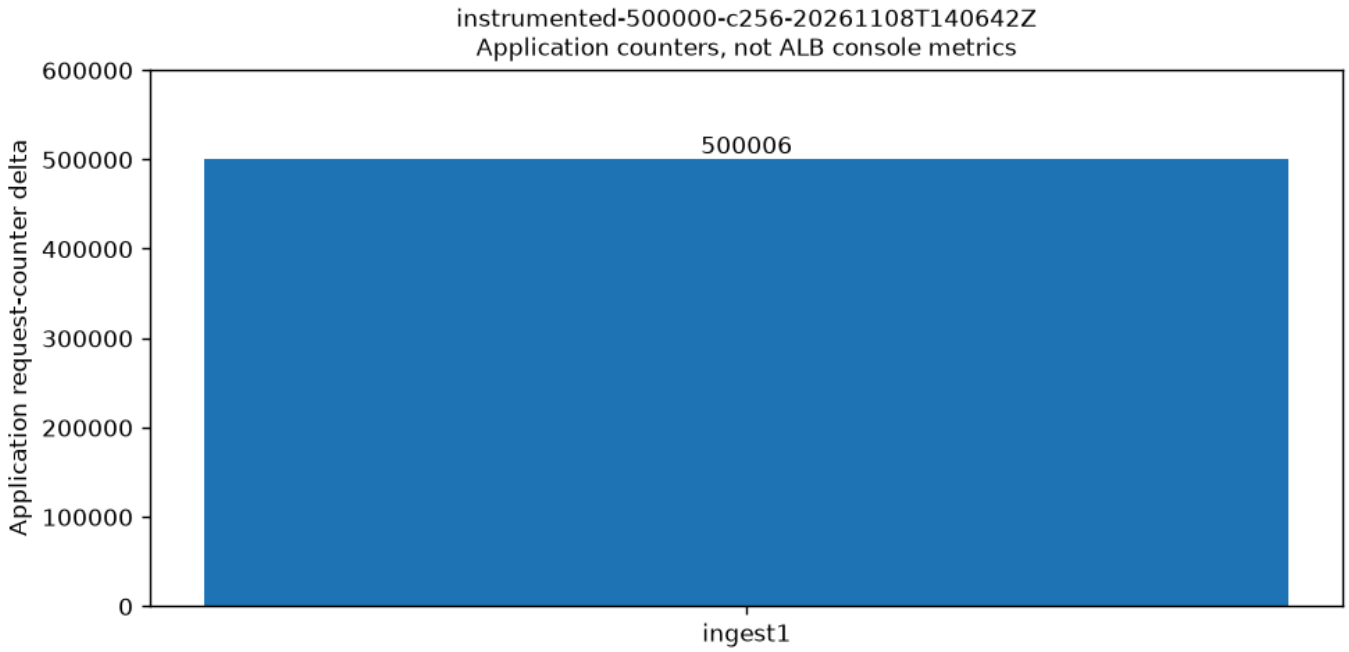
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	139.854
Program runtime seconds	160.128
Successful / final failed / retries	500,000 / 0 / 6
HTTP mean / p95 / p99 ms	55.23 / 100.20 / 275.29
Ready sampled mean / peak	15,173.65 / 55,463
Ready + Unacked peak	56,743
Mean broker publish / delivery / ACK rates per second	3,096.52 / 3,047.12 / 3,042.60
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	914 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

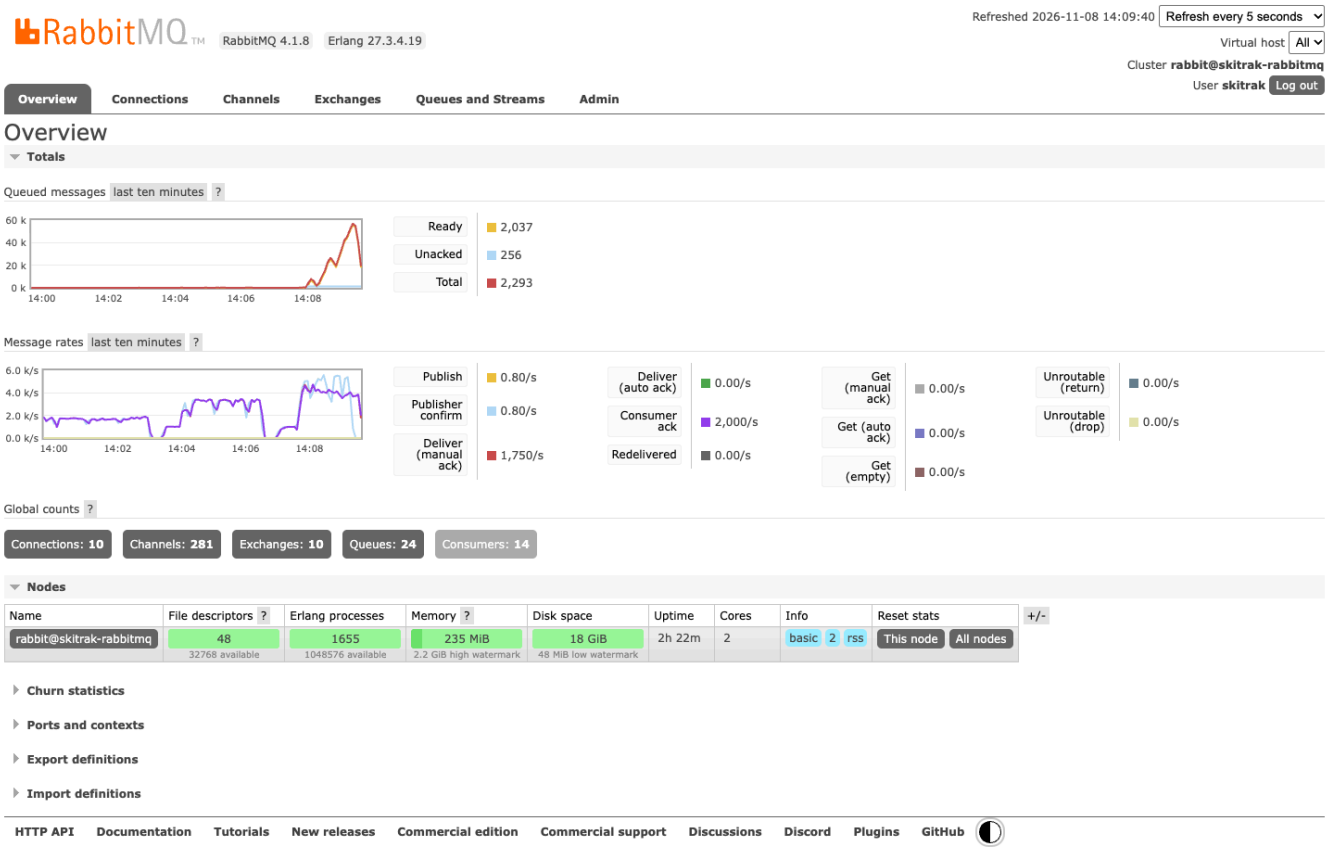

instrumented-500000-c256-20261108T140642Z
Recorded RabbitMQ API data; not a console screenshot
ready
unacked
total
target 1000
Messages / ten raw resort queues
0
10000
20000
30000
40000
50000
publishRate
deliverRate
ackRate
Broker-reported messages/s
0
1000
2000
3000
4000
5000
0
20
40
60
80
100
120
140
160
Seconds since client launch (includes drain)



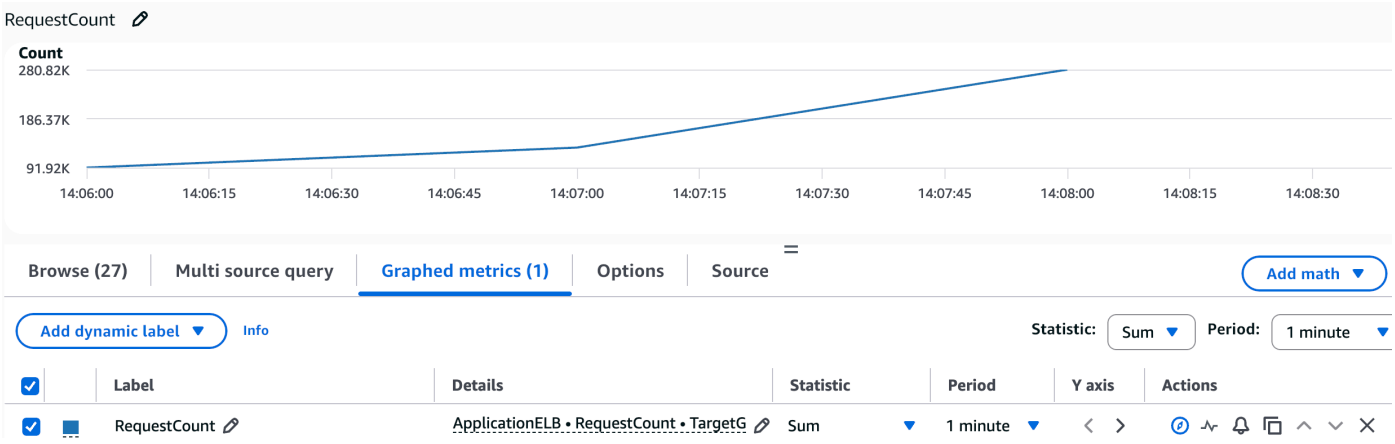


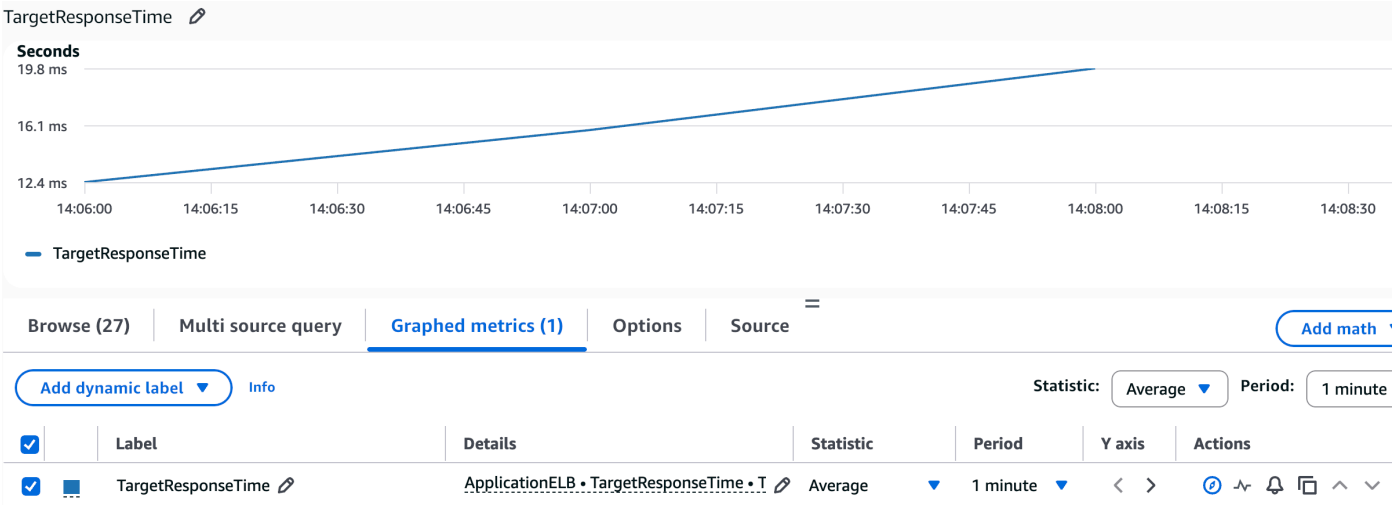
Actual RabbitMQ console captures:

- rabbitmq-20261108T140657440522Z.png
- rabbitmq-20261108T140742967673Z.png
- rabbitmq-20261108T140828558224Z.png
- rabbitmq-20261108T140913964838Z.png
- rabbitmq-20261108T140945061337Z-final.png



Supplied AWS CloudWatch captures:





Collection gaps: consumer collection errors: {"timeout": 20}

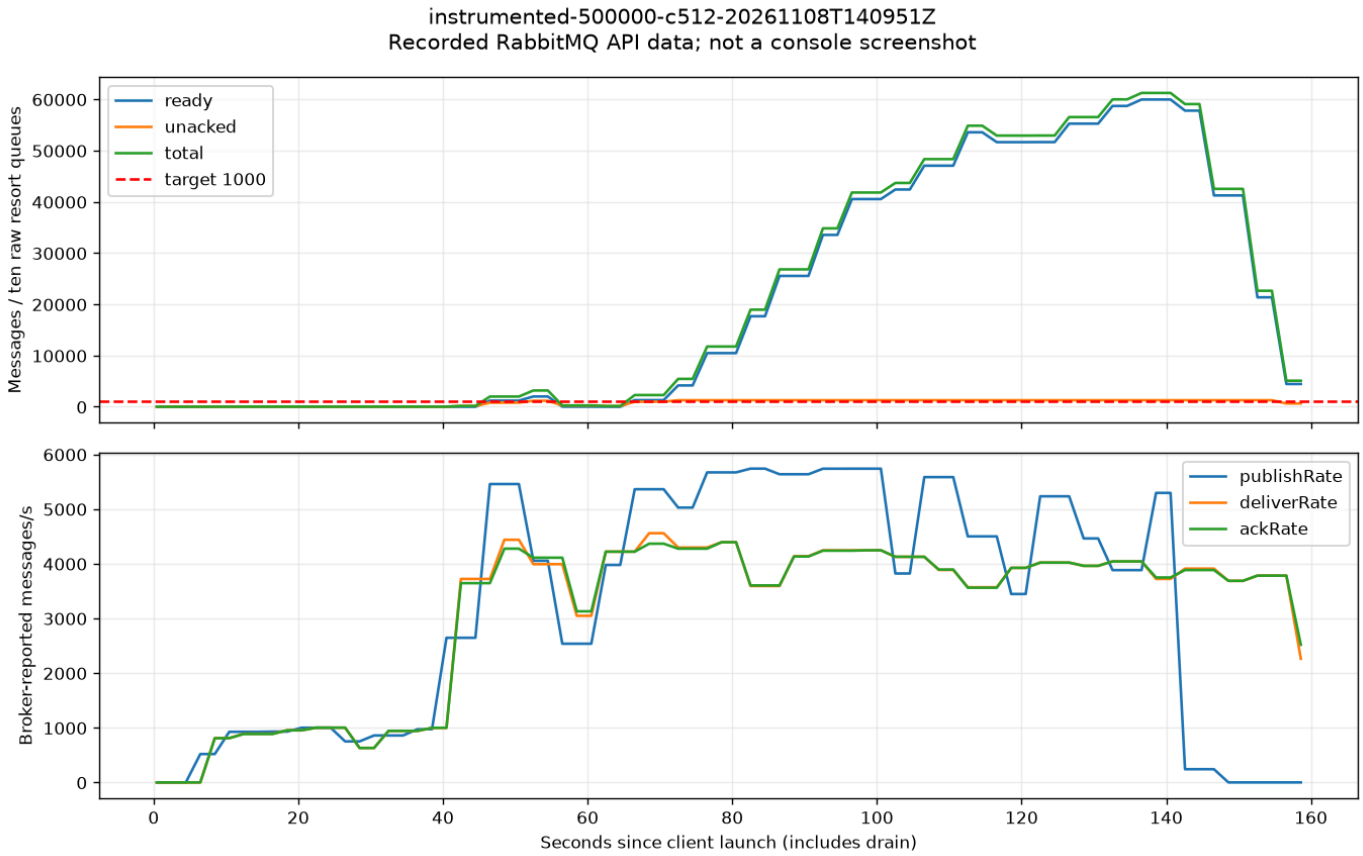
instrumented-500000-c512-20261108T140951Z

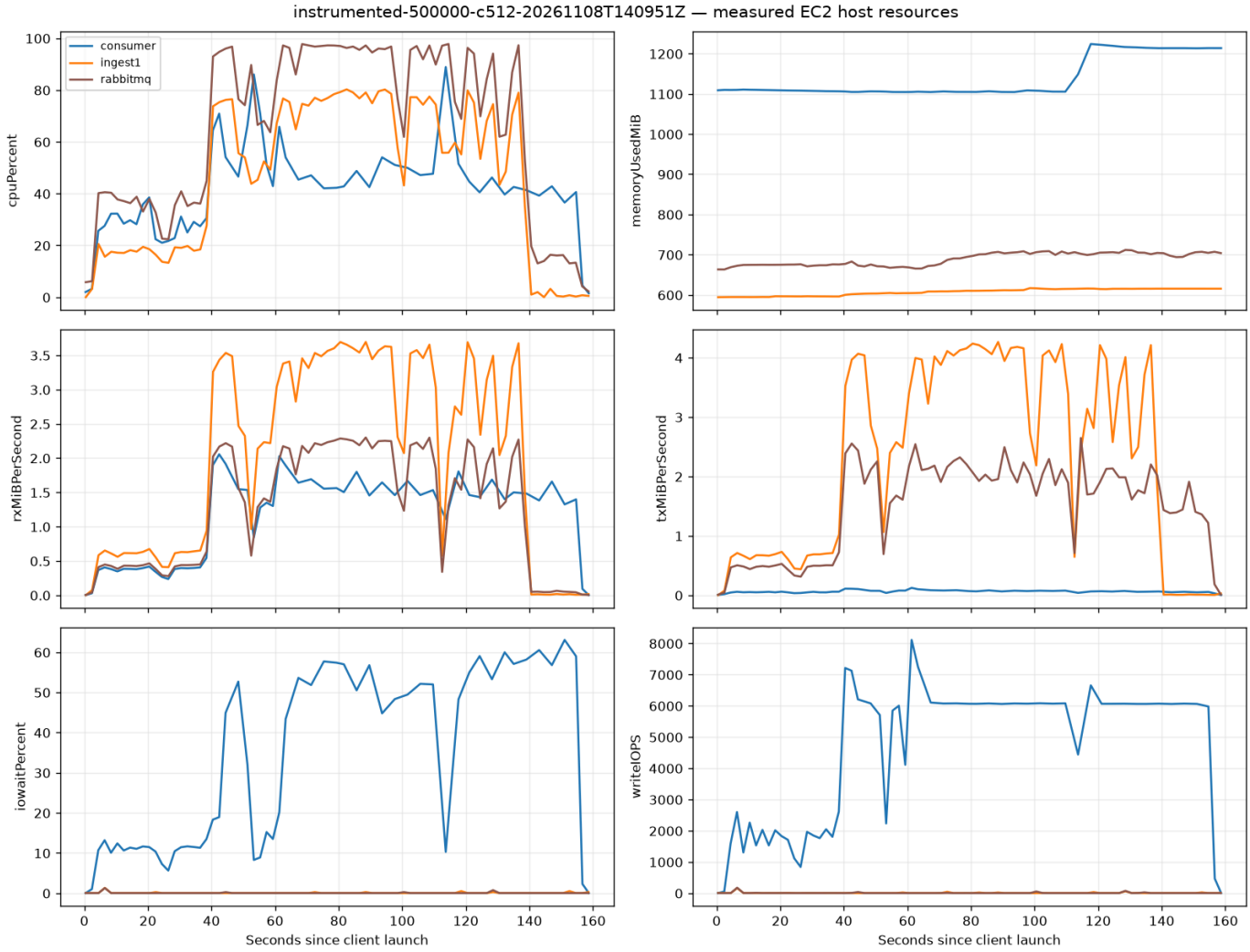
UTC: 2026-11-08T14:10:10.758007+00:00 to 2026-11-08T14:12:49.682730+00:00. Ingests=1; clientThreads=512; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

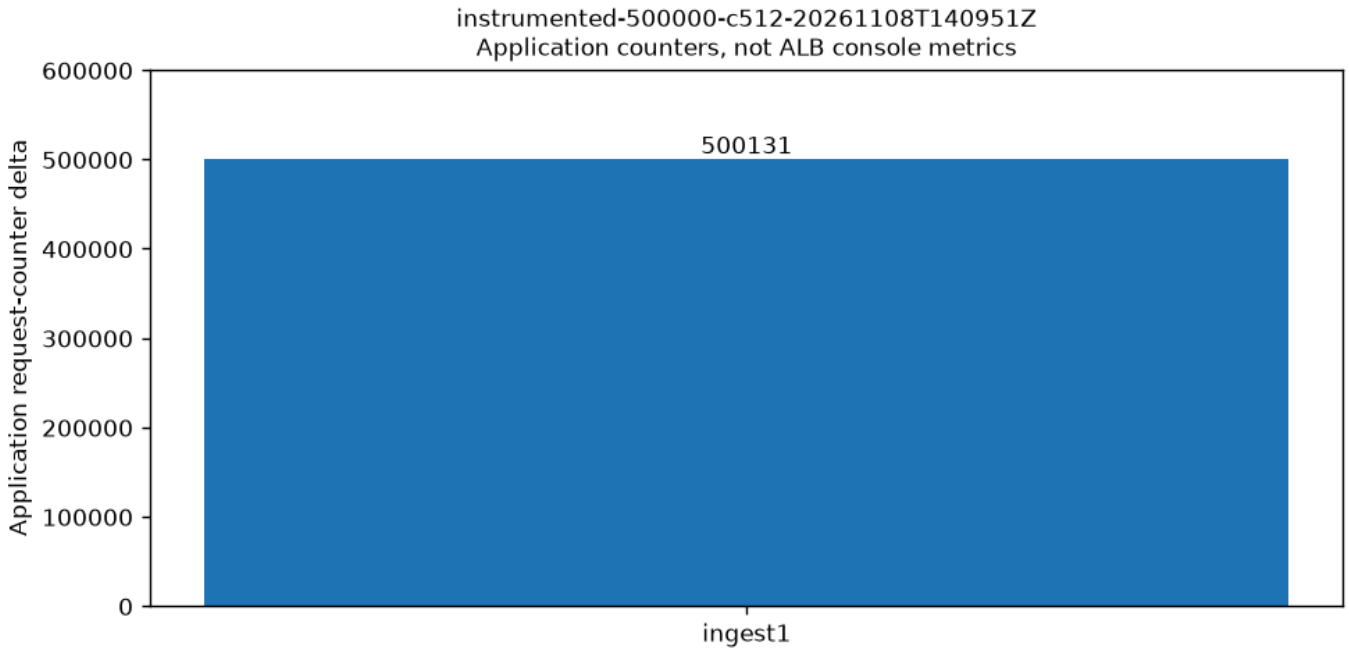
Complete client output I Full metrics, provenance and limits I Evidence page

Measurement	Result
Ingestion runtime seconds	135.398
Program runtime seconds	157.901
Successful / final failed / retries	500,000 / 0 / 131
HTTP mean / p95 / p99 ms	104.23 / 285.71 / 469.87
Ready sampled mean / peak	21,216.08 / 59,967
Ready + Unacked peak	61,247
Mean broker publish / delivery / ACK rates per second	3,190.88 / 3,101.10 / 3,097.74
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	913 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

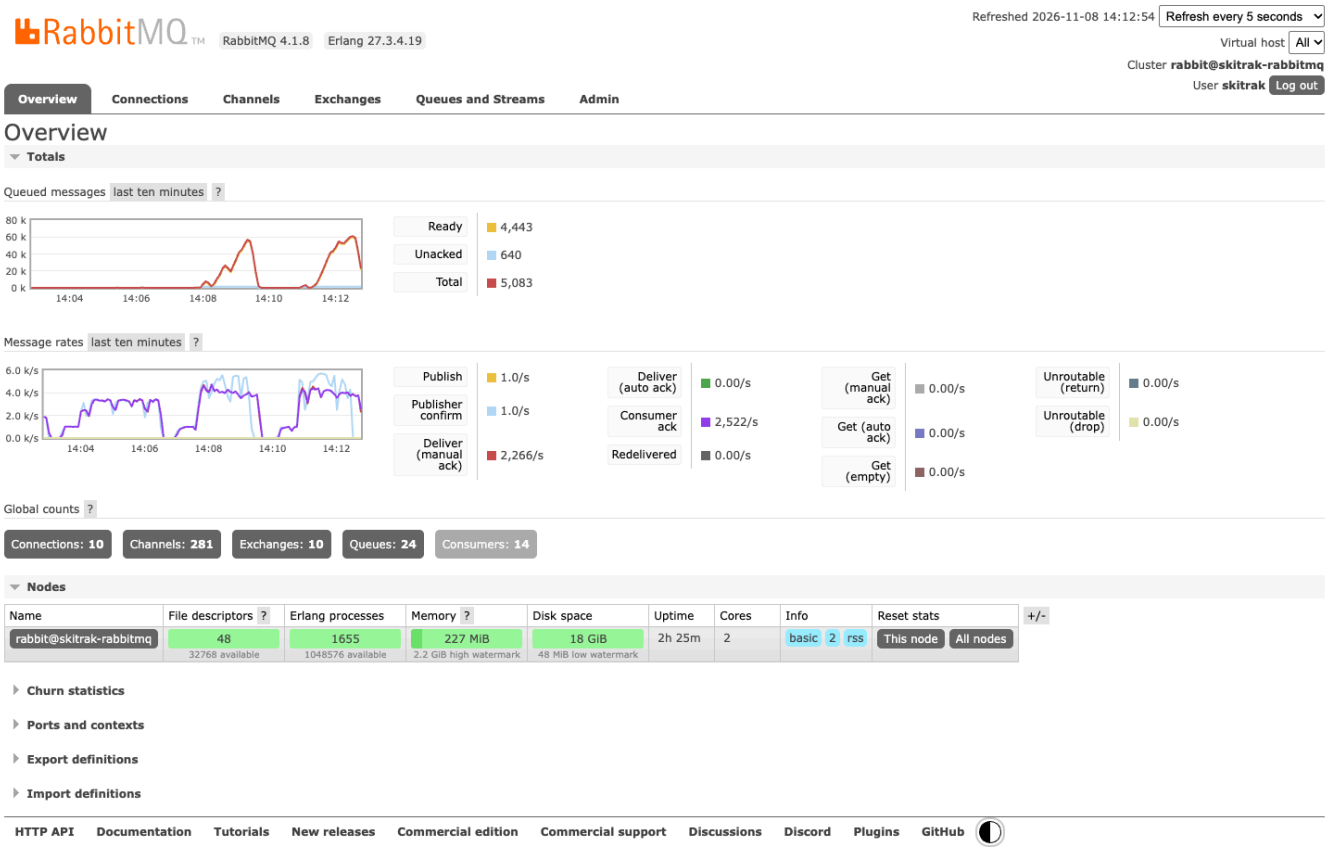

instrumented-500000-c512-20261108T140951Z
Recorded RabbitMQ API data; not a console screenshot
Messages / ten raw resort queues
ready
unacked
total
target 1000
Broker-reported messages/s
publishRate
deliverRate
ackRate
Seconds since client launch (includes drain)



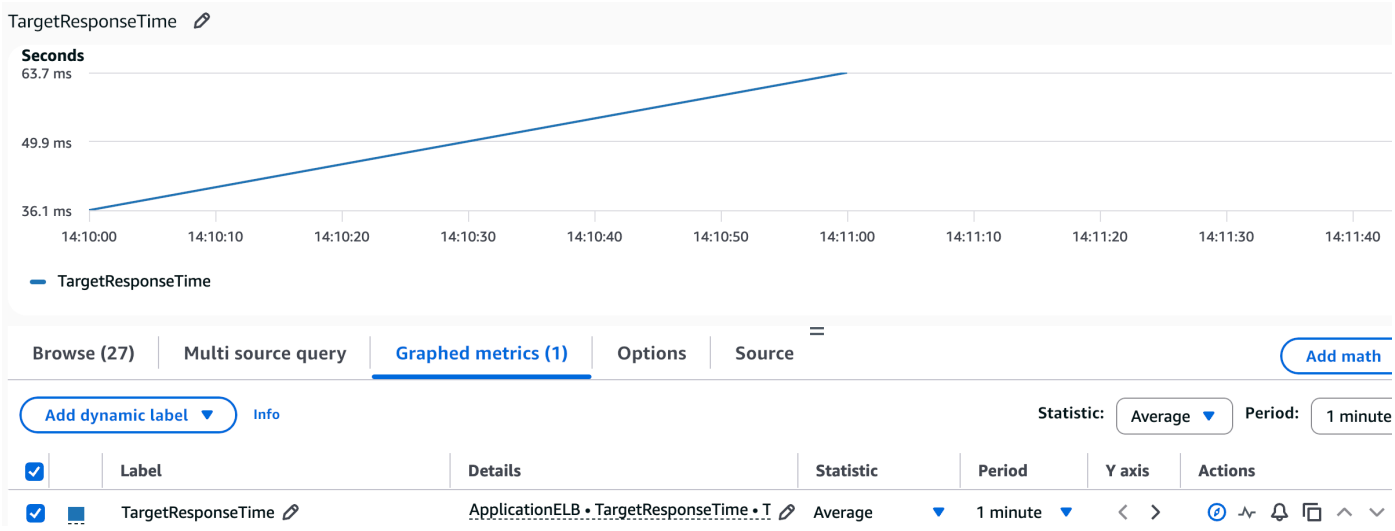


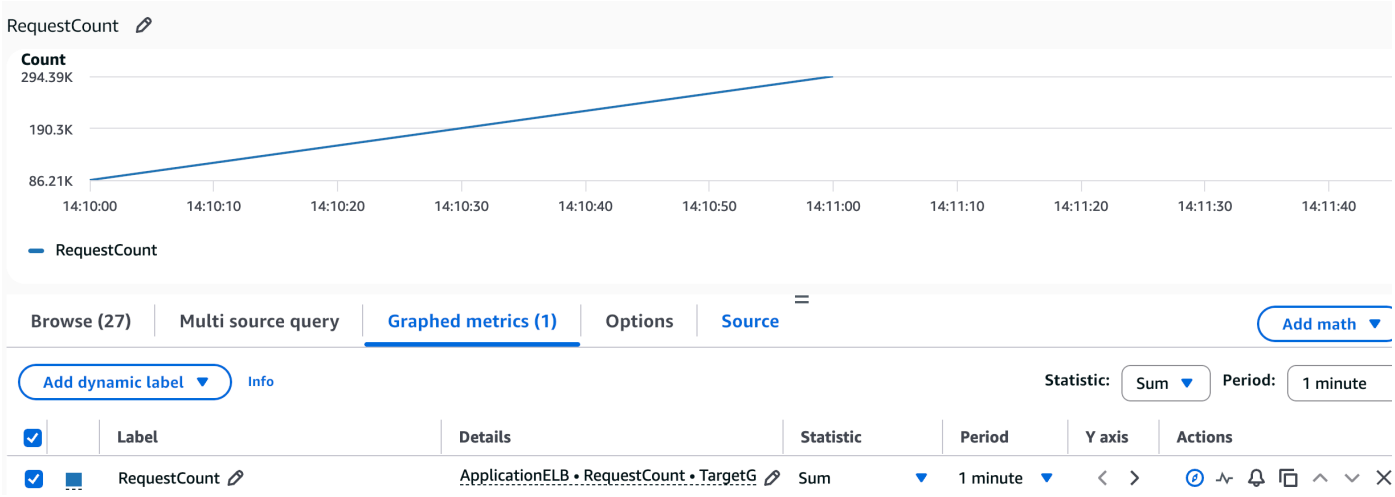
Actual RabbitMQ console captures:

- rabbitmq-20261108T141010418728Z.png
- rabbitmq-20261108T141055869692Z.png
- rabbitmq-20261108T141141324212Z.png
- rabbitmq-20261108T141226959878Z.png
- rabbitmq-20261108T141255950142Z-final.png



Supplied AWS CloudWatch captures:





Collection gaps: consumer collection errors: {"timeout": 19}

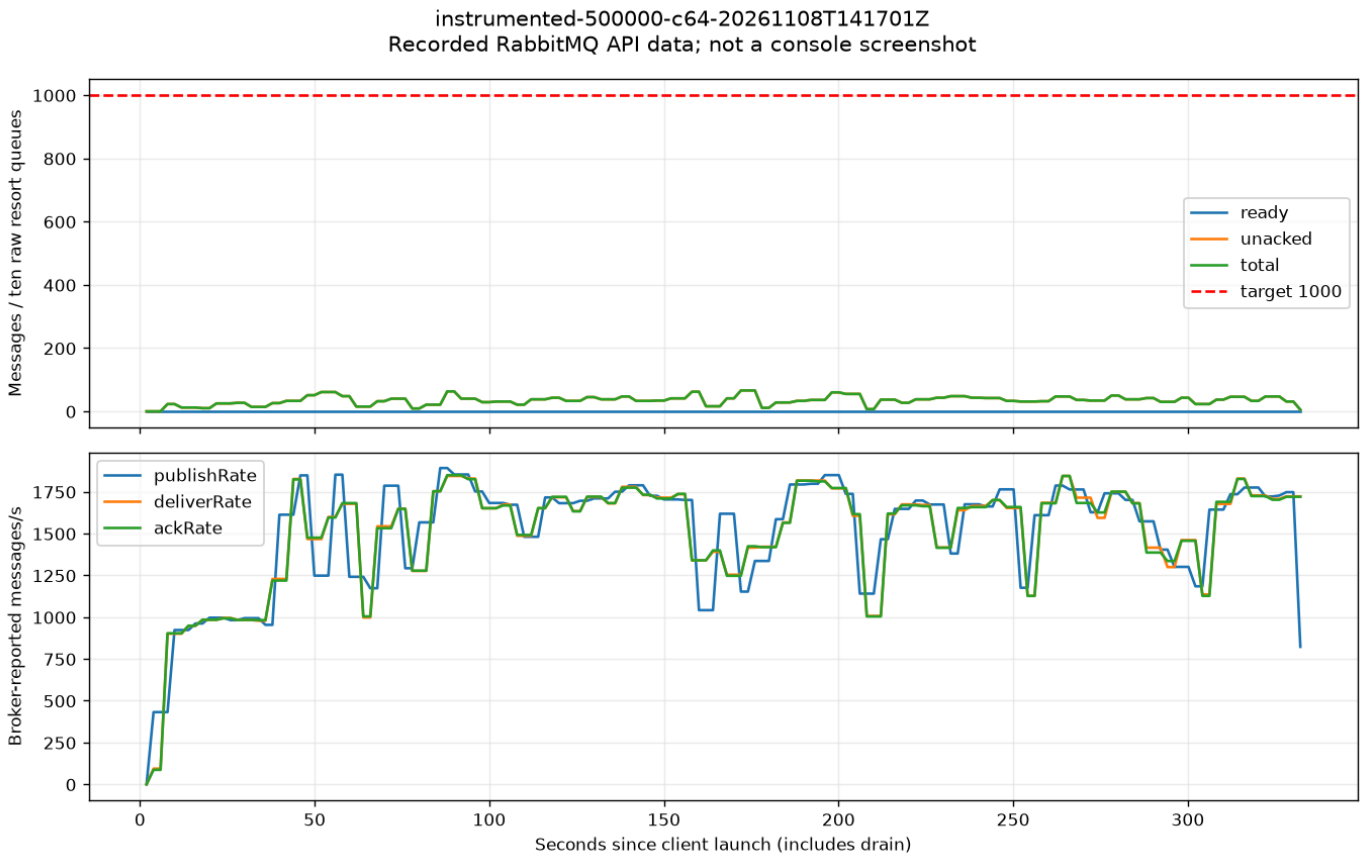
instrumented-500000-c64-20261108T141701Z

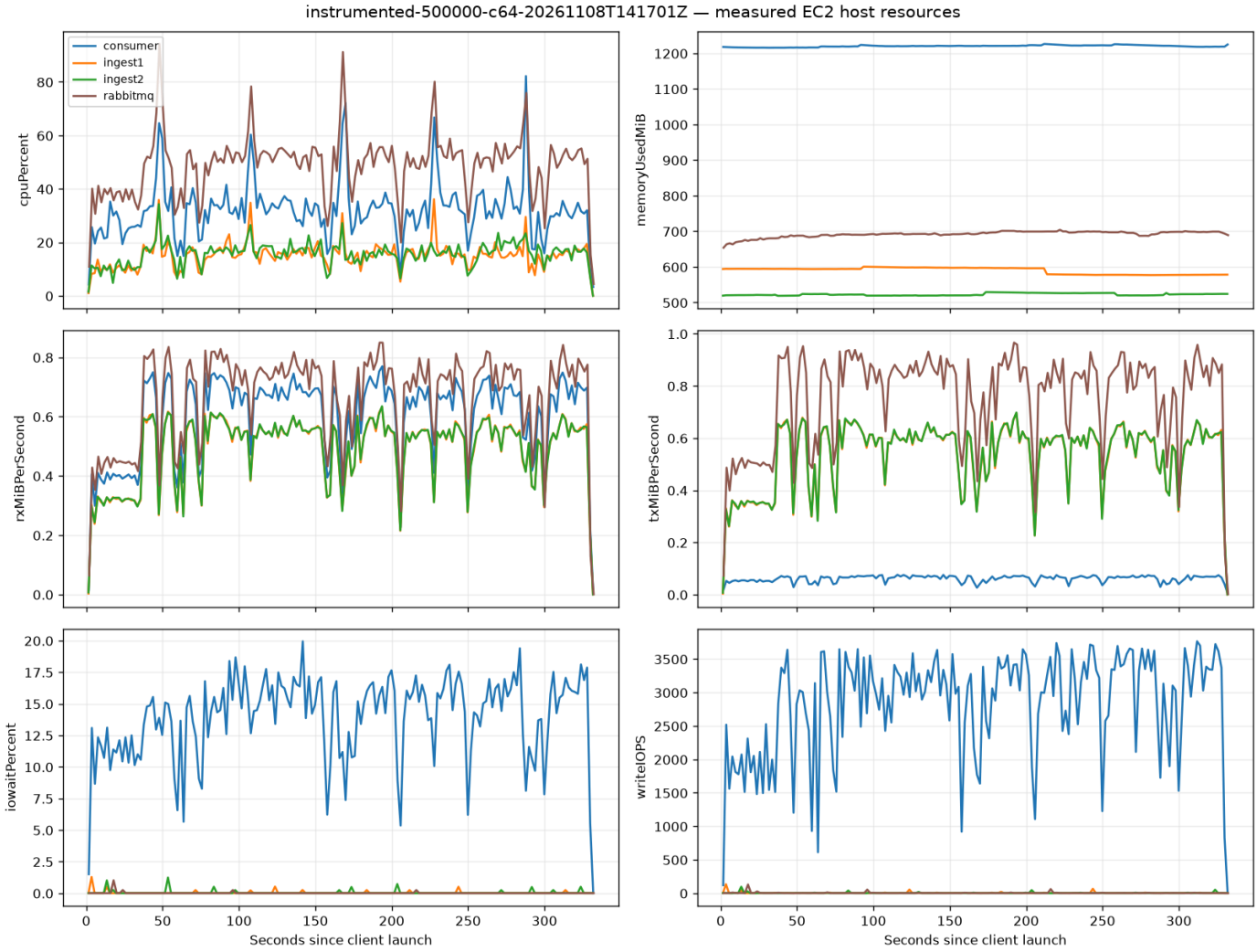
UTC: 2026-11-08T14:17:15.532676+00:00 to 2026-11-08T14:22:48.038979+00:00. Ingests=2; clientThreads=64; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

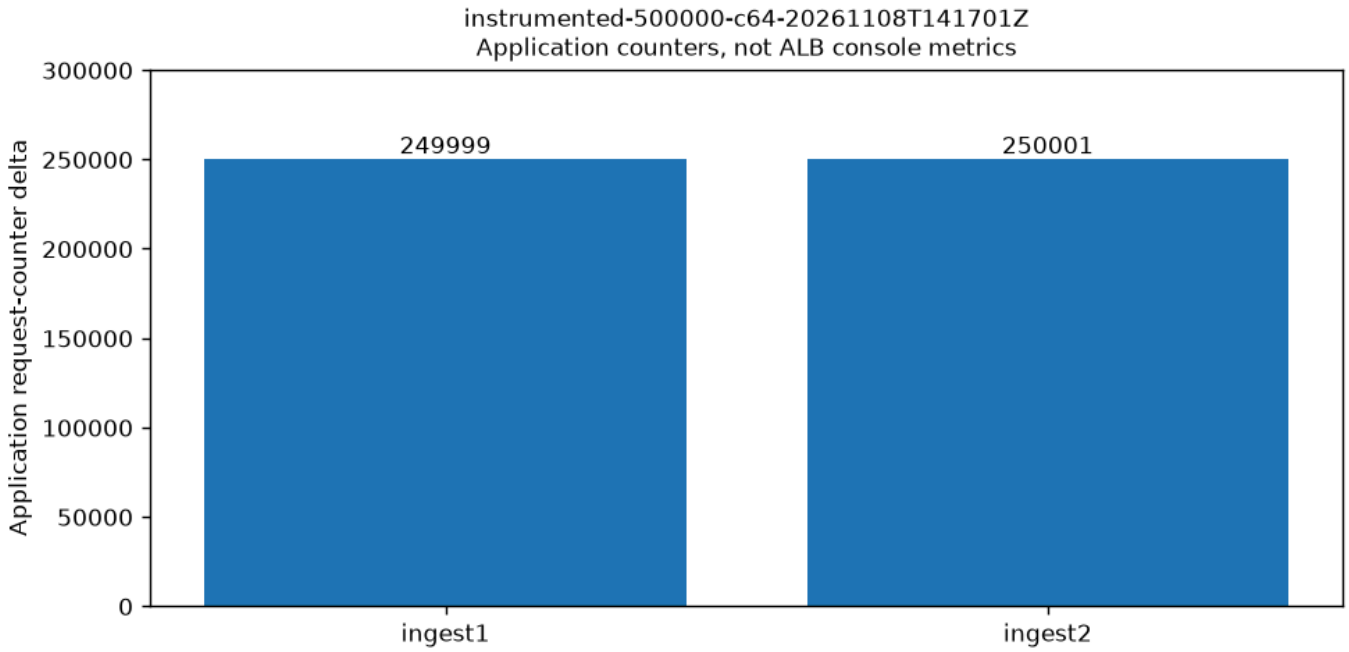
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	327.098
Program runtime seconds	331.398
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	39.67 / 54.30 / 153.19
Ready sampled mean / peak	0.00 / 0
Ready + Unacked peak	66
Mean broker publish / delivery / ACK rates per second	1,509.35 / 1,501.61 / 1,500.93
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	911 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

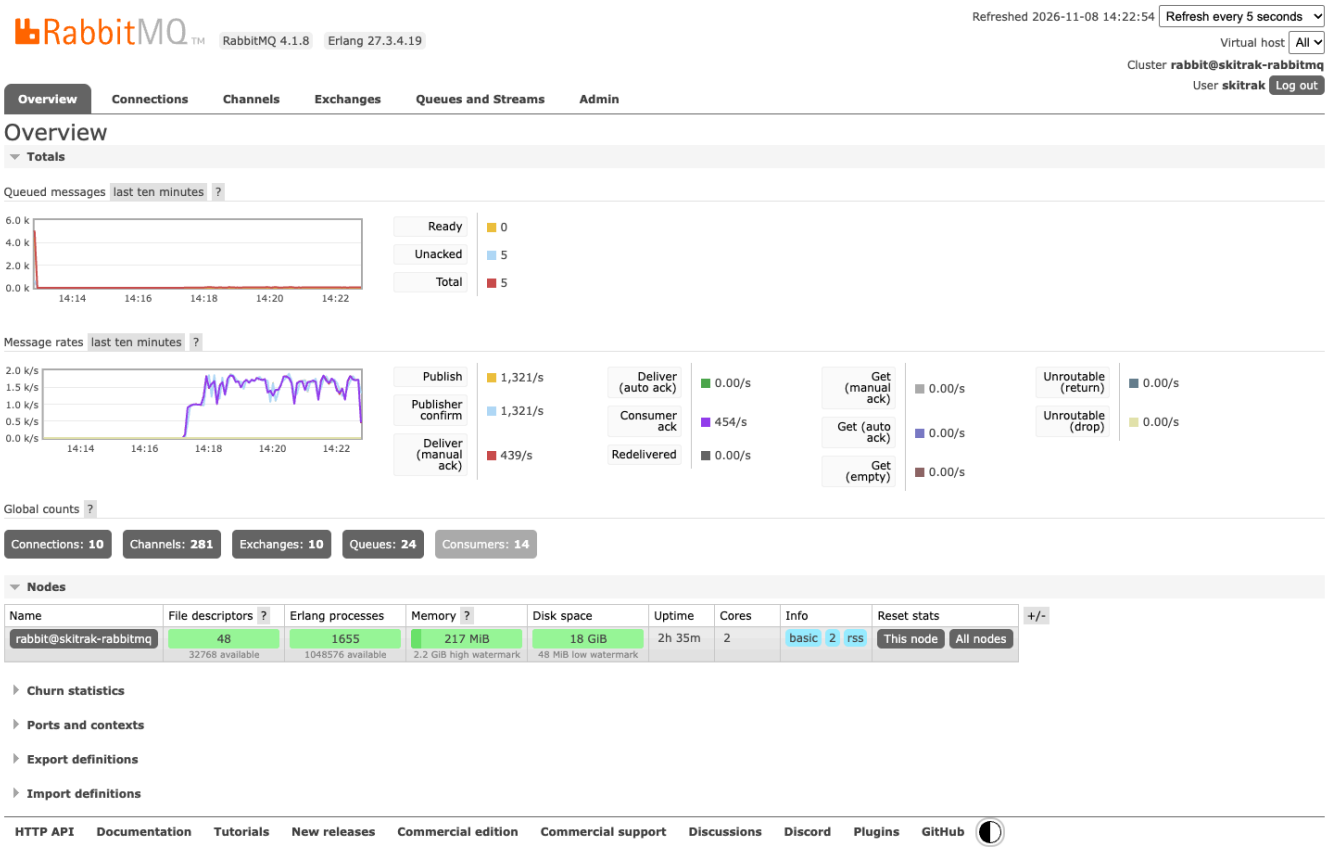

instrumented-500000-c64-20261108T141701Z
Recorded RabbitMQ API data; not a console screenshot
Messages / ten raw resort queues
1000
800
600
400
200
0
ready
unacked
total
target 1000
Broker-reported messages/s
1750
1500
1250
1000
750
500
250
0
publishRate
deliverRate
ackRate
0
50
100
150
200
250
300
Seconds since client launch (includes drain)



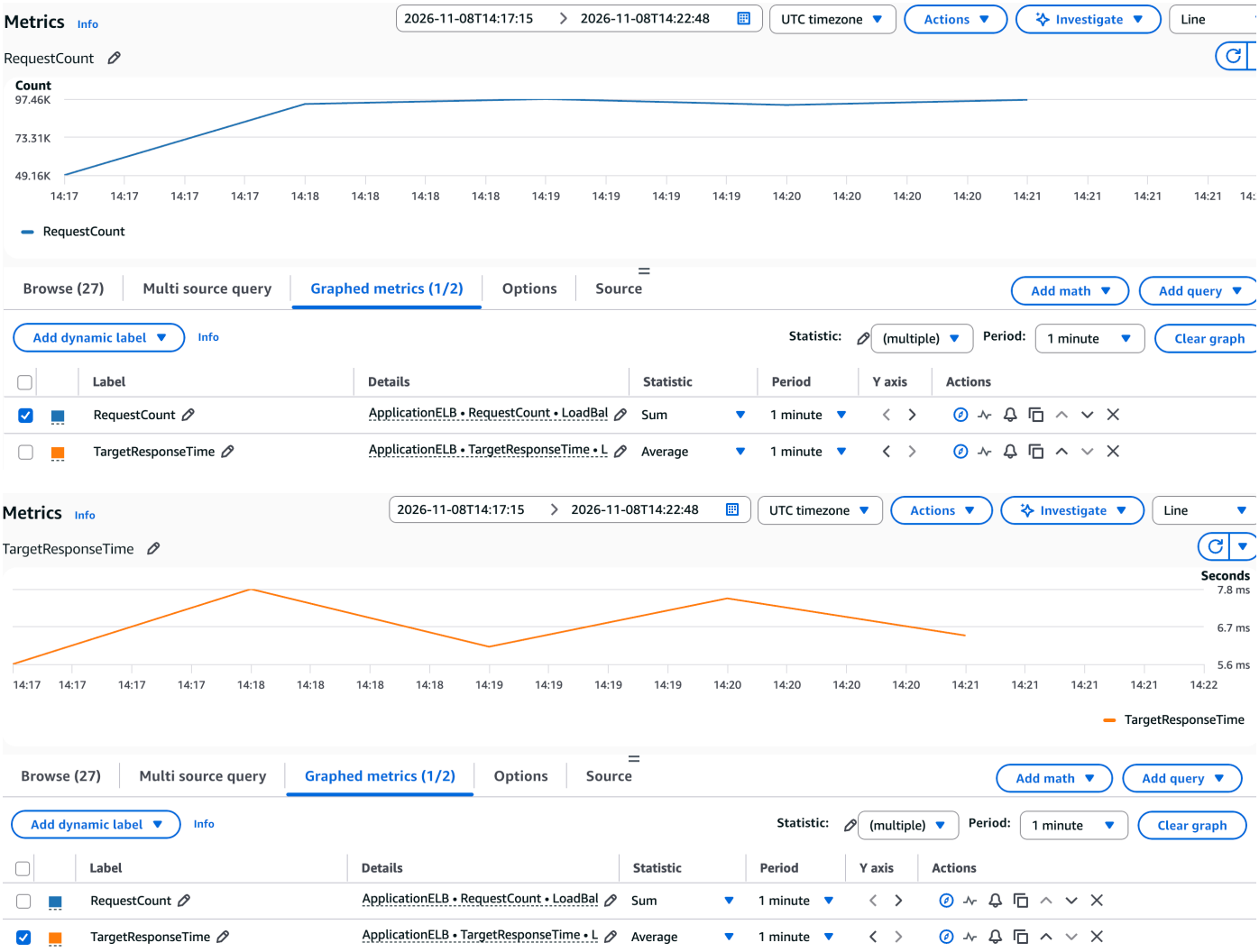


Actual RabbitMQ console captures:

- rabbitmq-20261108T141715459083Z.png
- rabbitmq-20261108T141800934393Z.png
- rabbitmq-20261108T141846408126Z.png
- rabbitmq-20261108T141931906860Z.png
- rabbitmq-20261108T142017454100Z.png
- rabbitmq-20261108T142103021592Z.png
- rabbitmq-20261108T142148401347Z.png
- rabbitmq-20261108T142233893222Z.png
- rabbitmq-20261108T142254621796Z-final.png



Supplied AWS CloudWatch captures:



Collection gaps: None detected in the recorded collectors.

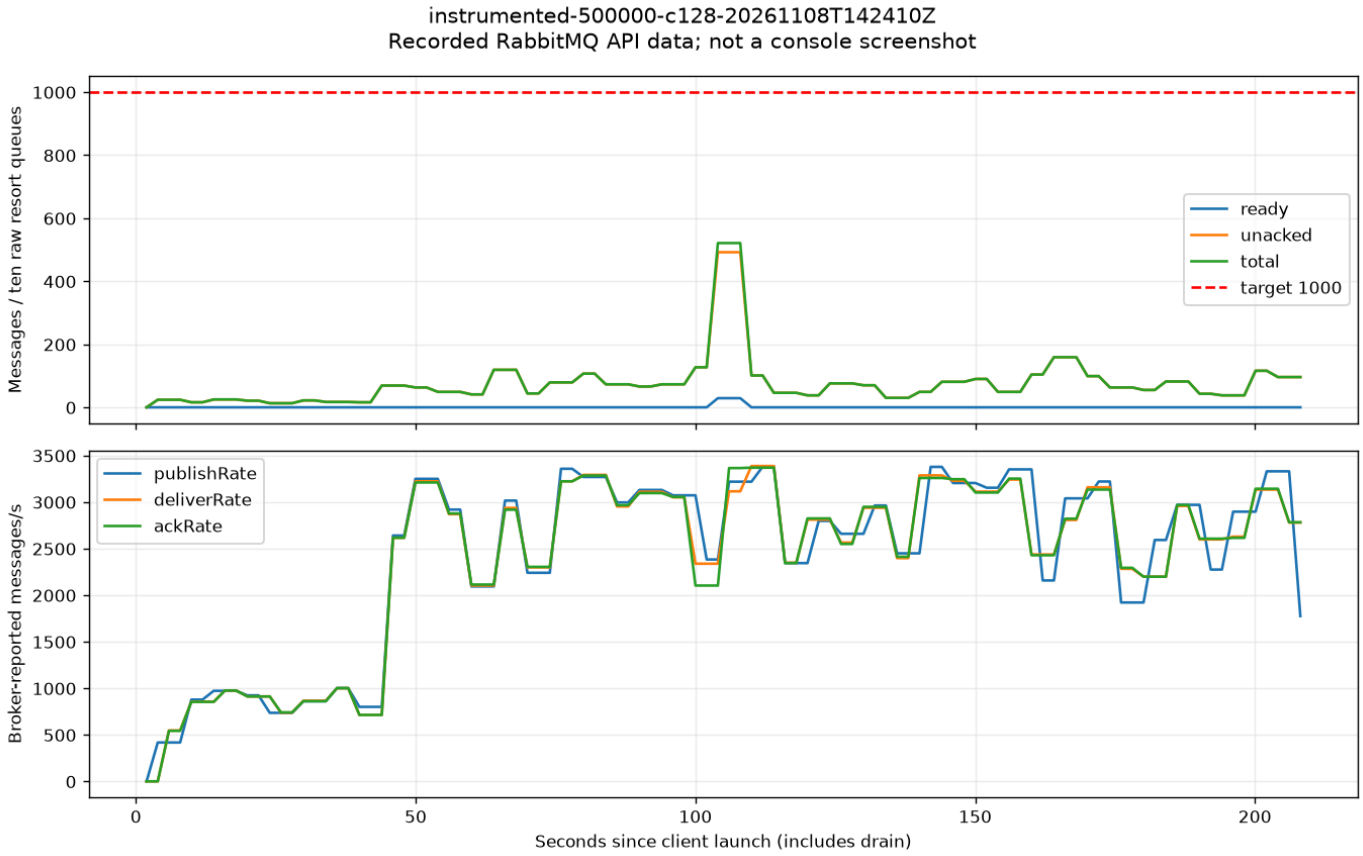
instrumented-500000-c128-20261108T142410Z

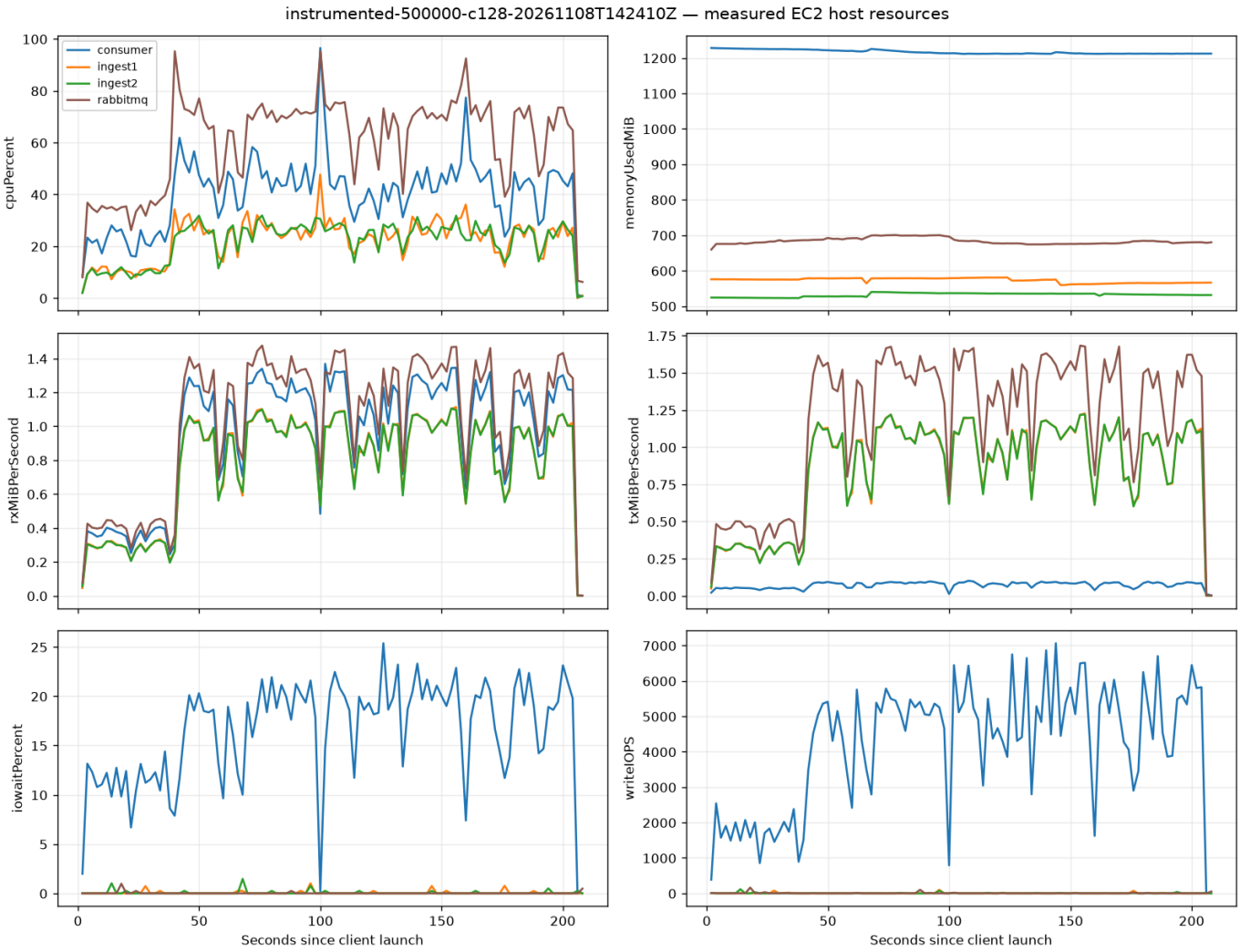
UTC: 2026-11-08T14:24:23.473898+00:00 to 2026-11-08T14:27:52.242822+00:00. Ingests=2; clientThreads=128; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

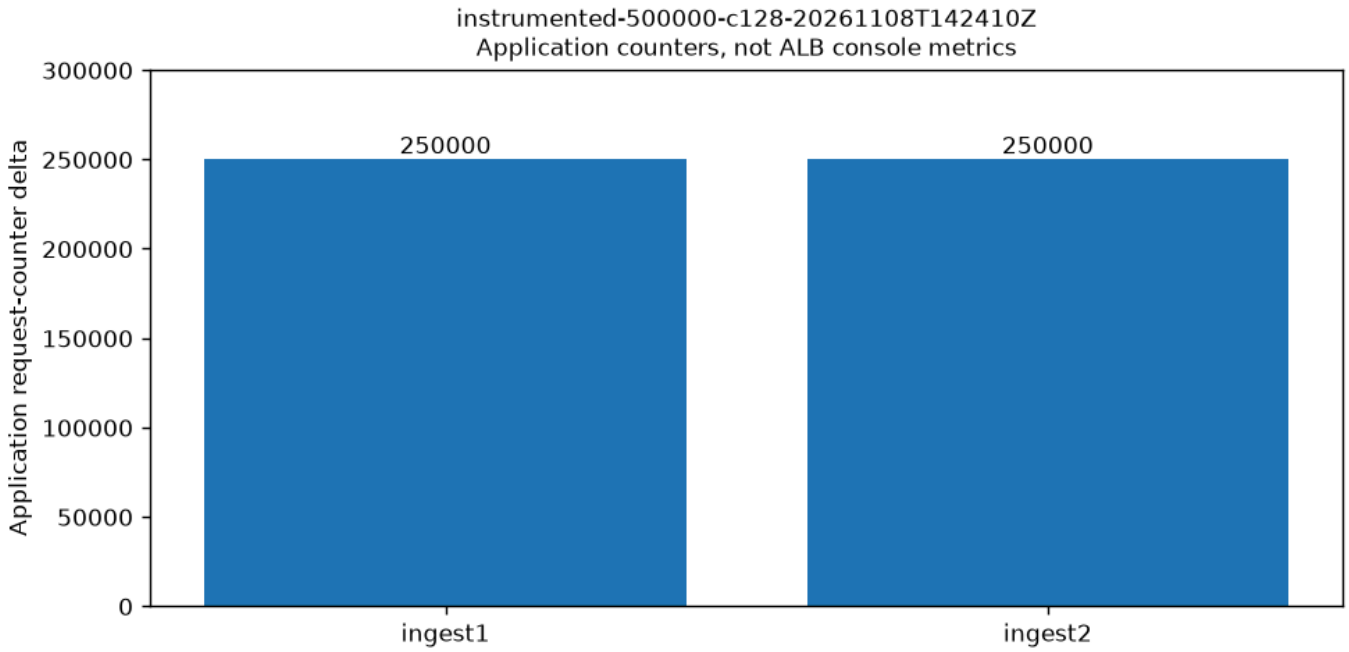
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	202.451
Program runtime seconds	207.123
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	44.75 / 70.57 / 191.59
Ready sampled mean / peak	0.84 / 29
Ready + Unacked peak	521
Mean broker publish / delivery / ACK rates per second	2,408.02 / 2,391.88 / 2,388.94
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	910 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

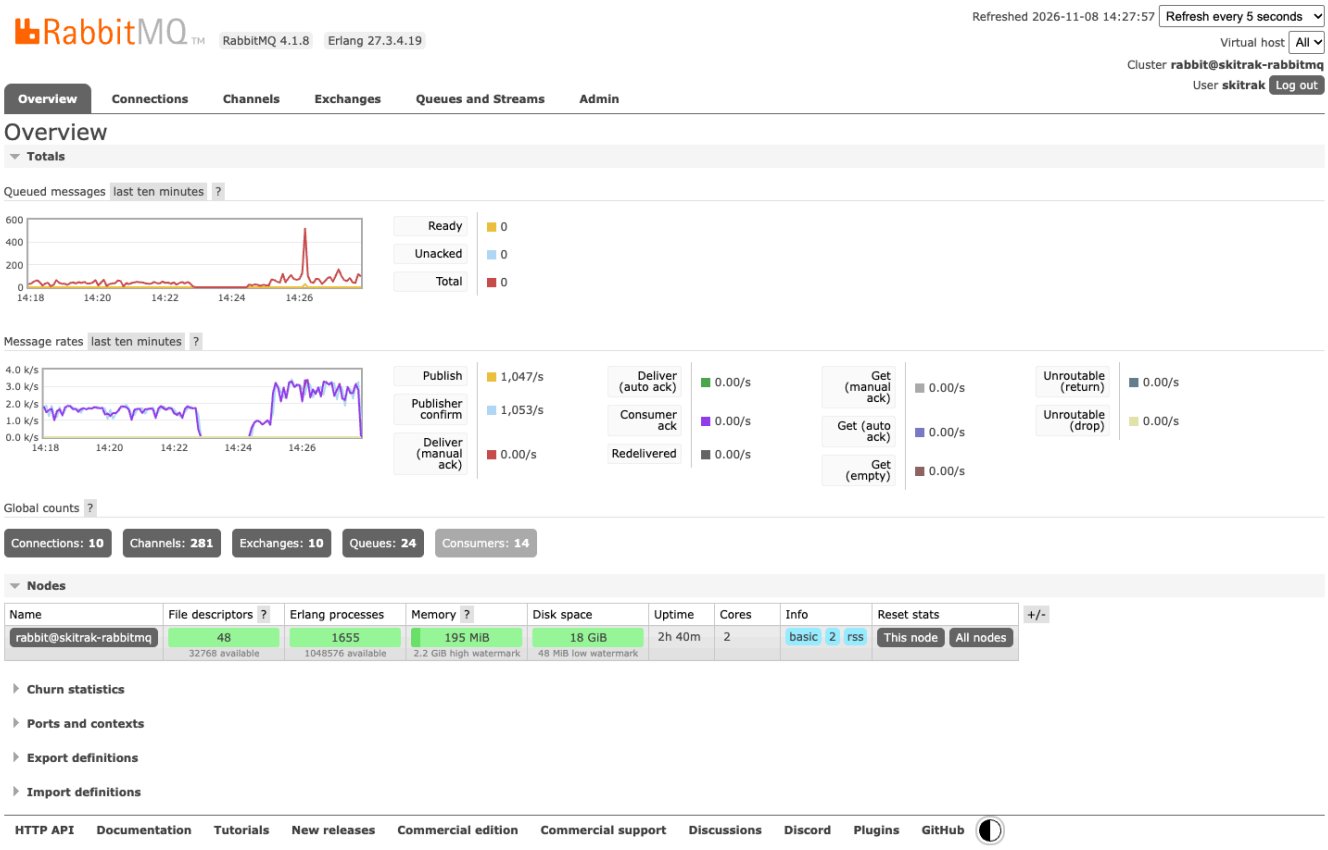




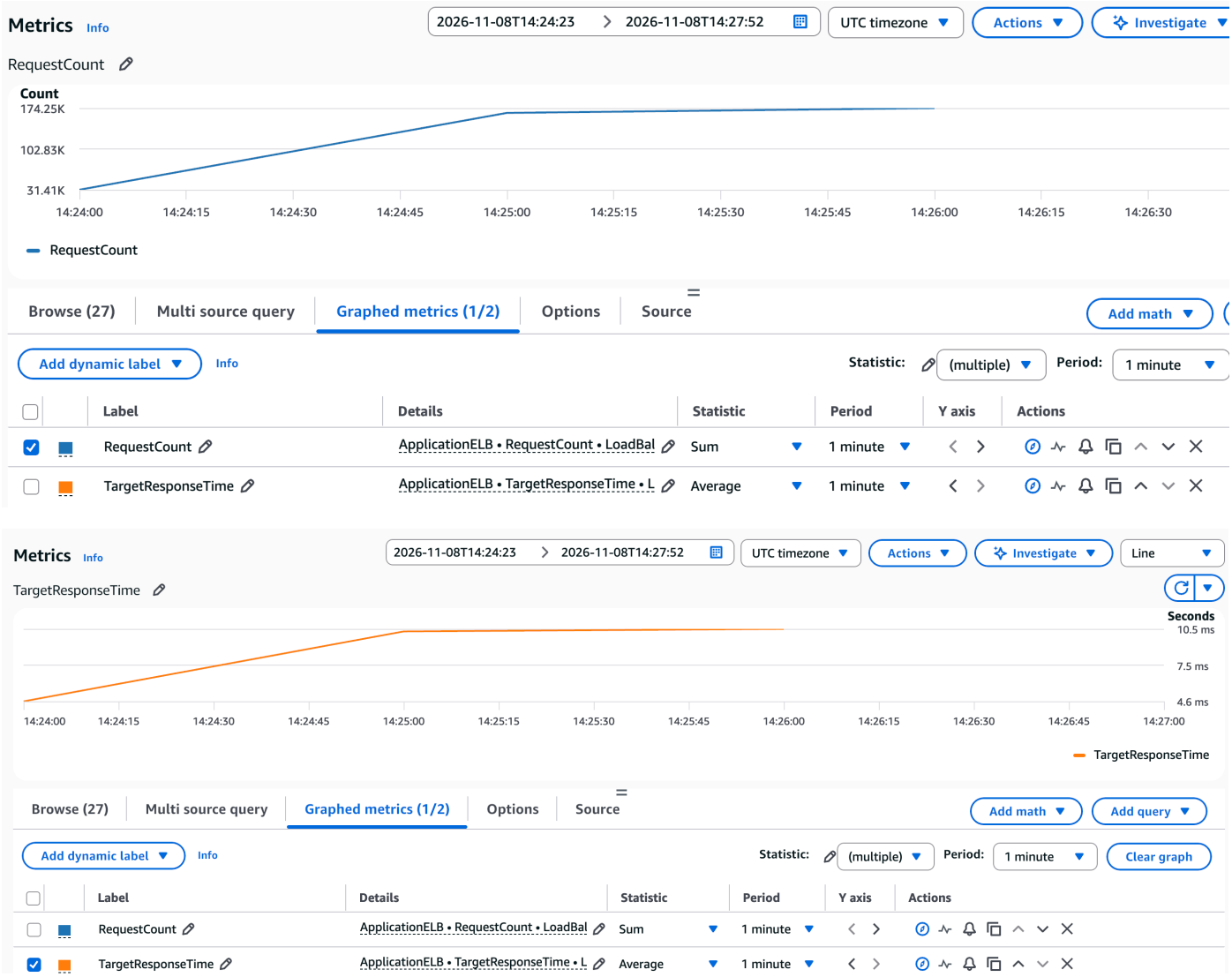


Actual RabbitMQ console captures:

- rabbitmq-20261108T142423480227Z.png
- rabbitmq-20261108T142508642707Z.png
- rabbitmq-20261108T142553803961Z.png
- rabbitmq-20261108T142639326312Z.png
- rabbitmq-20261108T142724845969Z.png
- rabbitmq-20261108T142758760836Z-final.png



Supplied AWS CloudWatch captures:



Collection gaps: None detected in the recorded collectors.

instrumented-500000-c256-20261108T142912Z

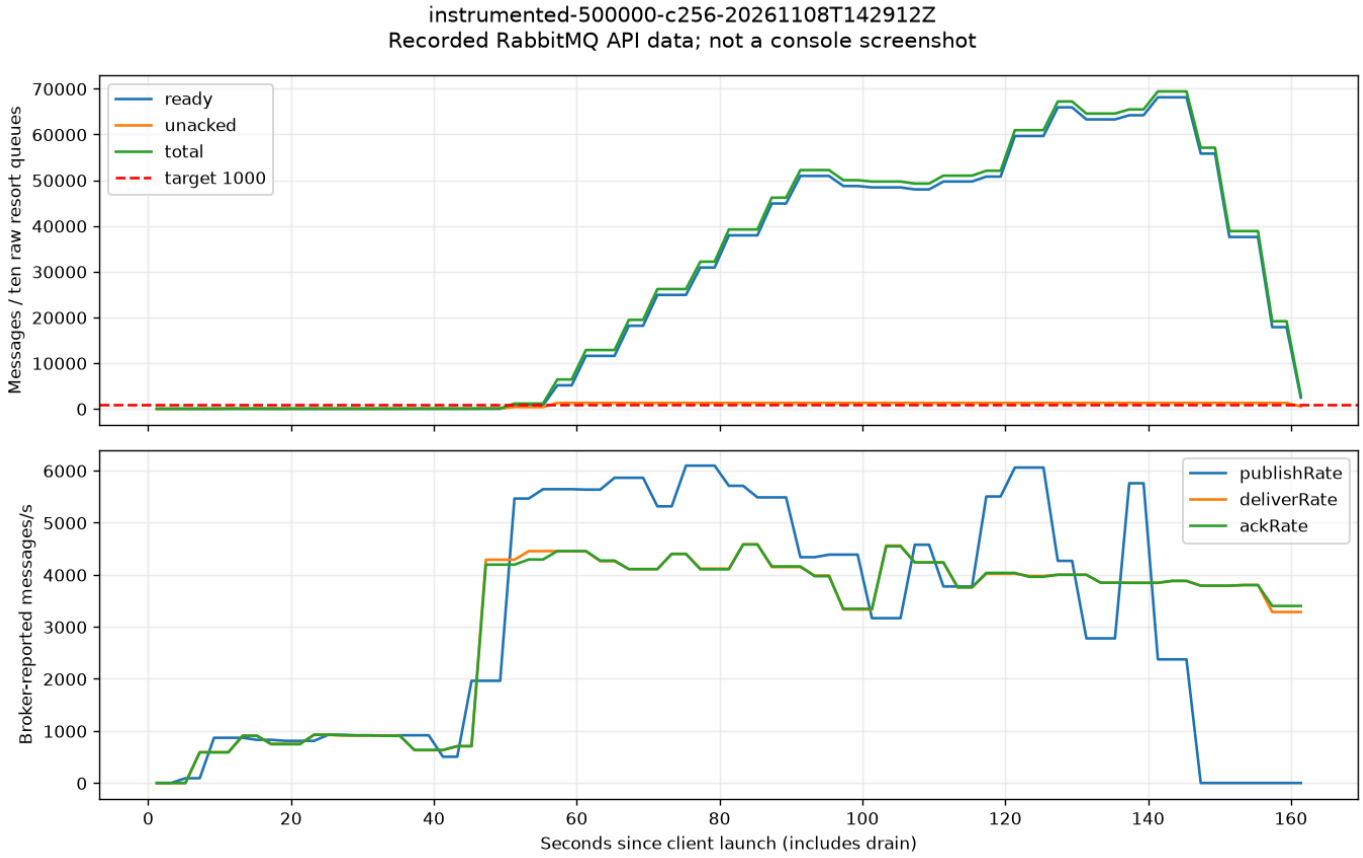
UTC: 2026-11-08T14:29:26.256846+00:00 to 2026-11-08T14:32:07.986202+00:00. Ingests=2; clientThreads=256; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

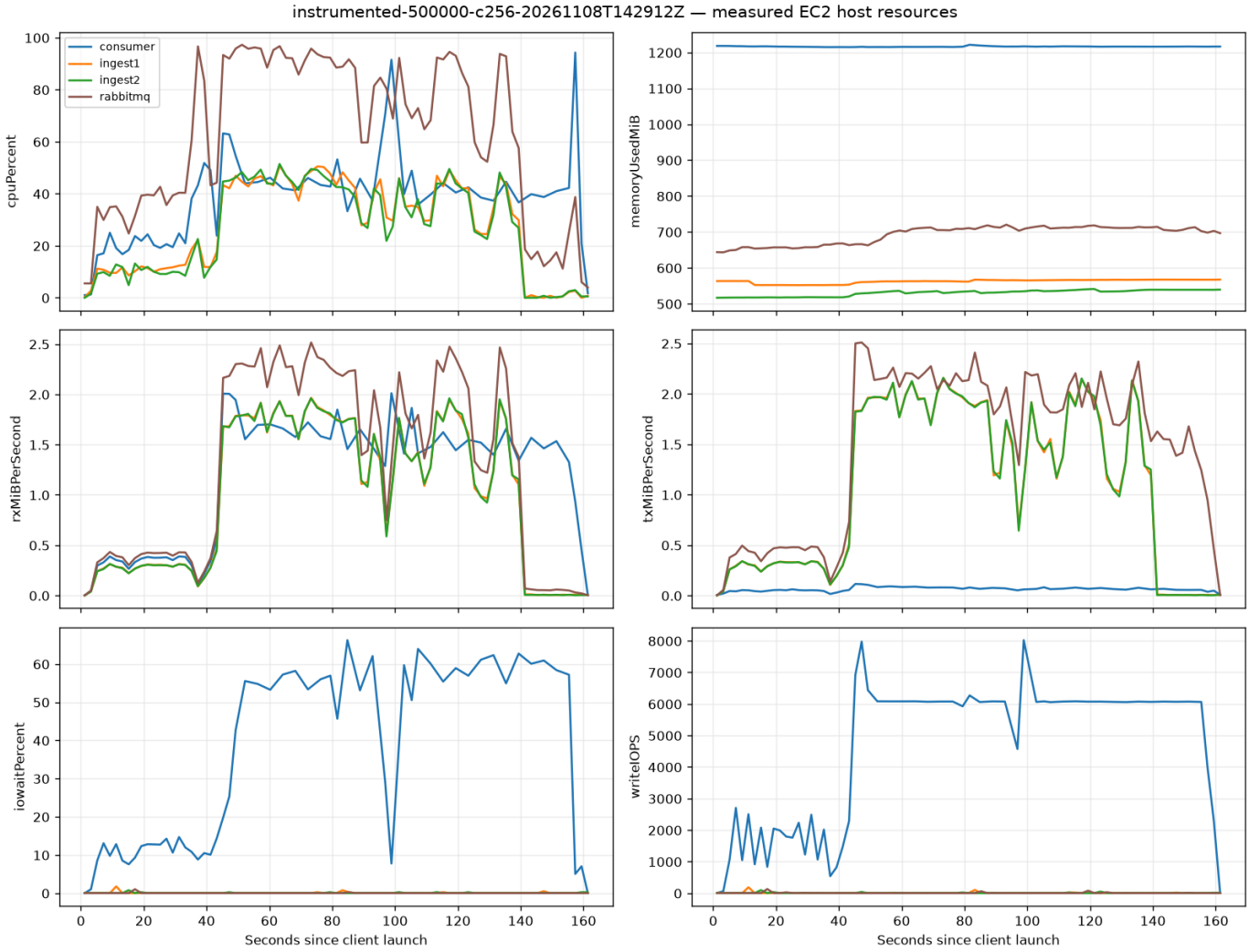
Complete client output | Full metrics, provenance and limits | Evidence page

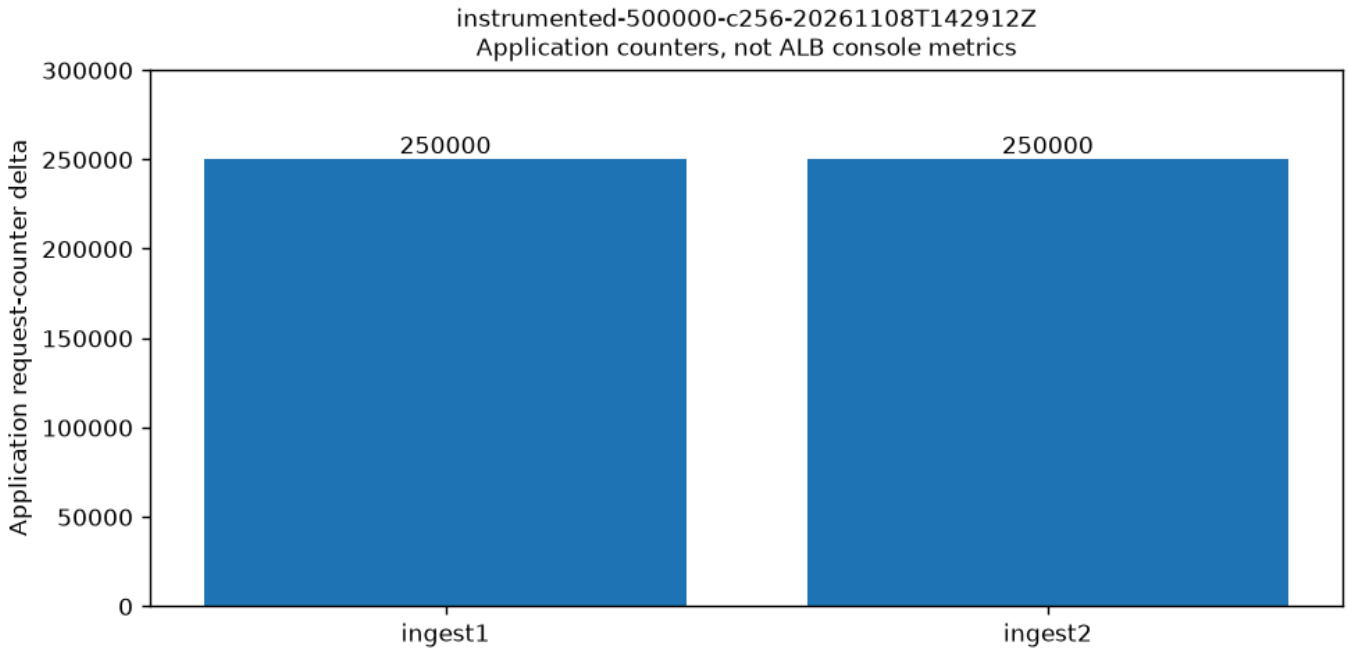
Measurement	Result
Ingestion runtime seconds	136.650
Program runtime seconds	160.362
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	52.01 / 96.41 / 272.08
Ready sampled mean / peak	27,919.26 / 68,120
Ready + Unacked peak	69,400
Mean broker publish / delivery / ACK rates per second	3,099.88 / 3,079.26 / 3,076.42
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0

Measurement	Result
WebSocket updates / reconnections	904 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

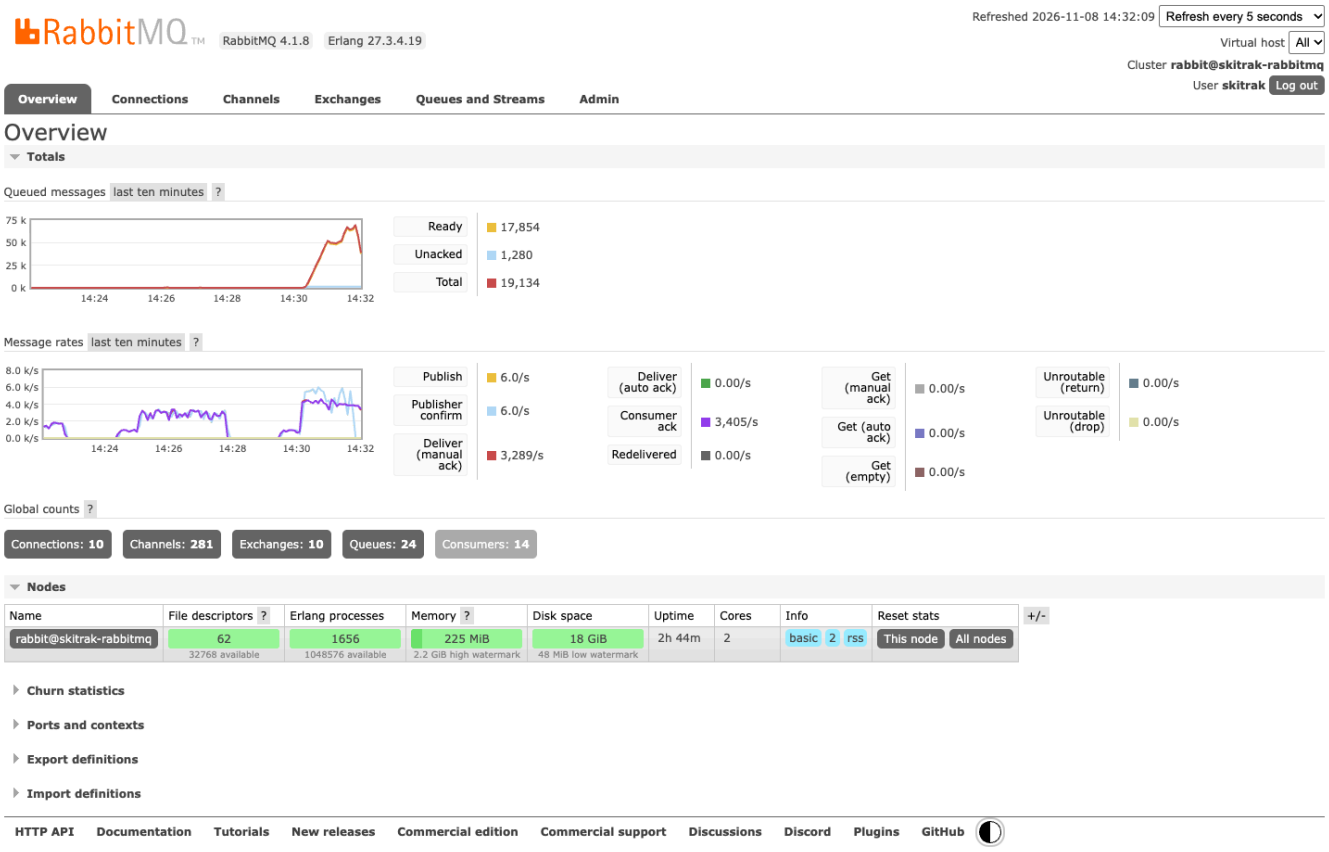




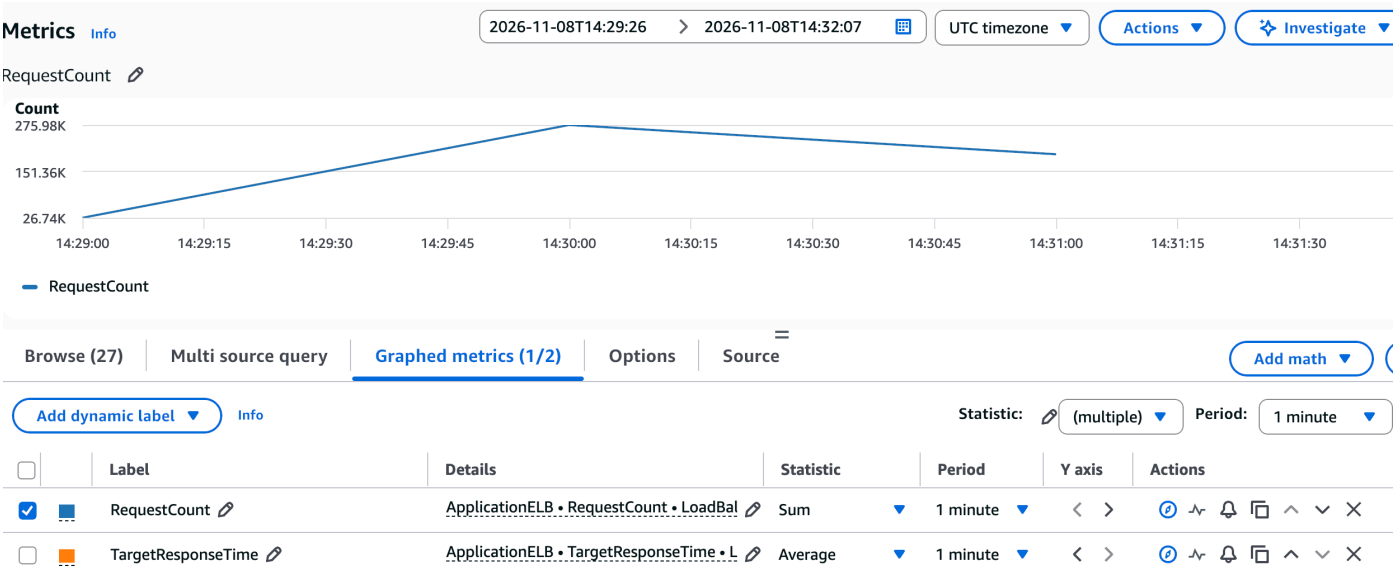


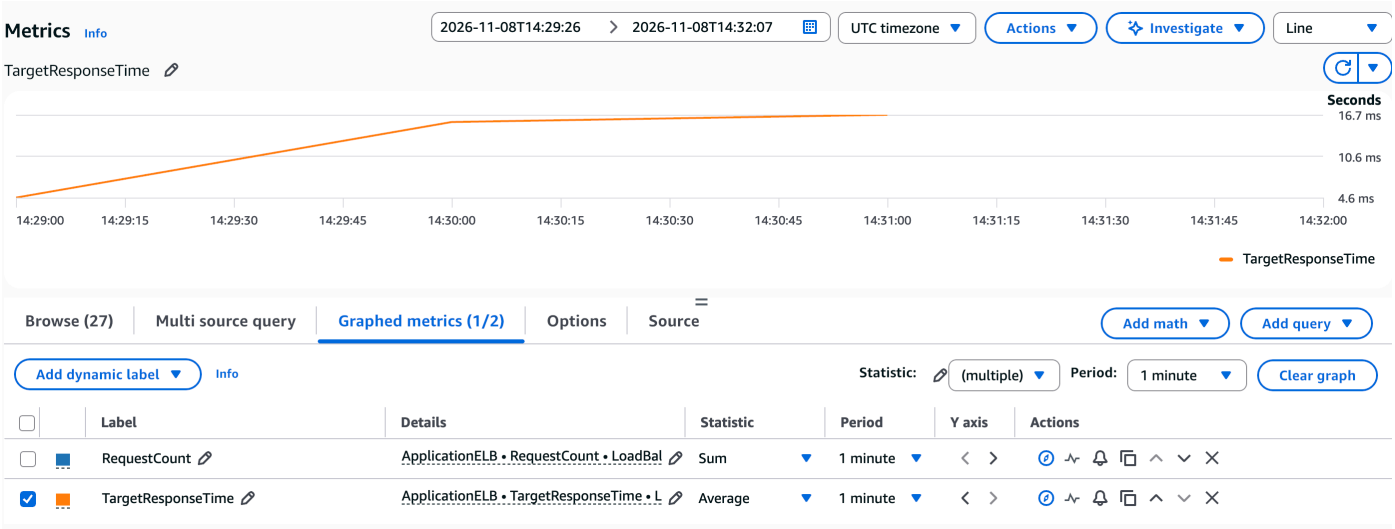
Actual RabbitMQ console captures:

- rabbitmq-20261108T142926139298Z.png
- rabbitmq-20261108T143011729404Z.png
- rabbitmq-20261108T143057065814Z.png
- rabbitmq-20261108T143142490944Z.png
- rabbitmq-20261108T143214353302Z-final.png



Supplied AWS CloudWatch captures:





Collection gaps: consumer collection errors: {"timeout": 22}

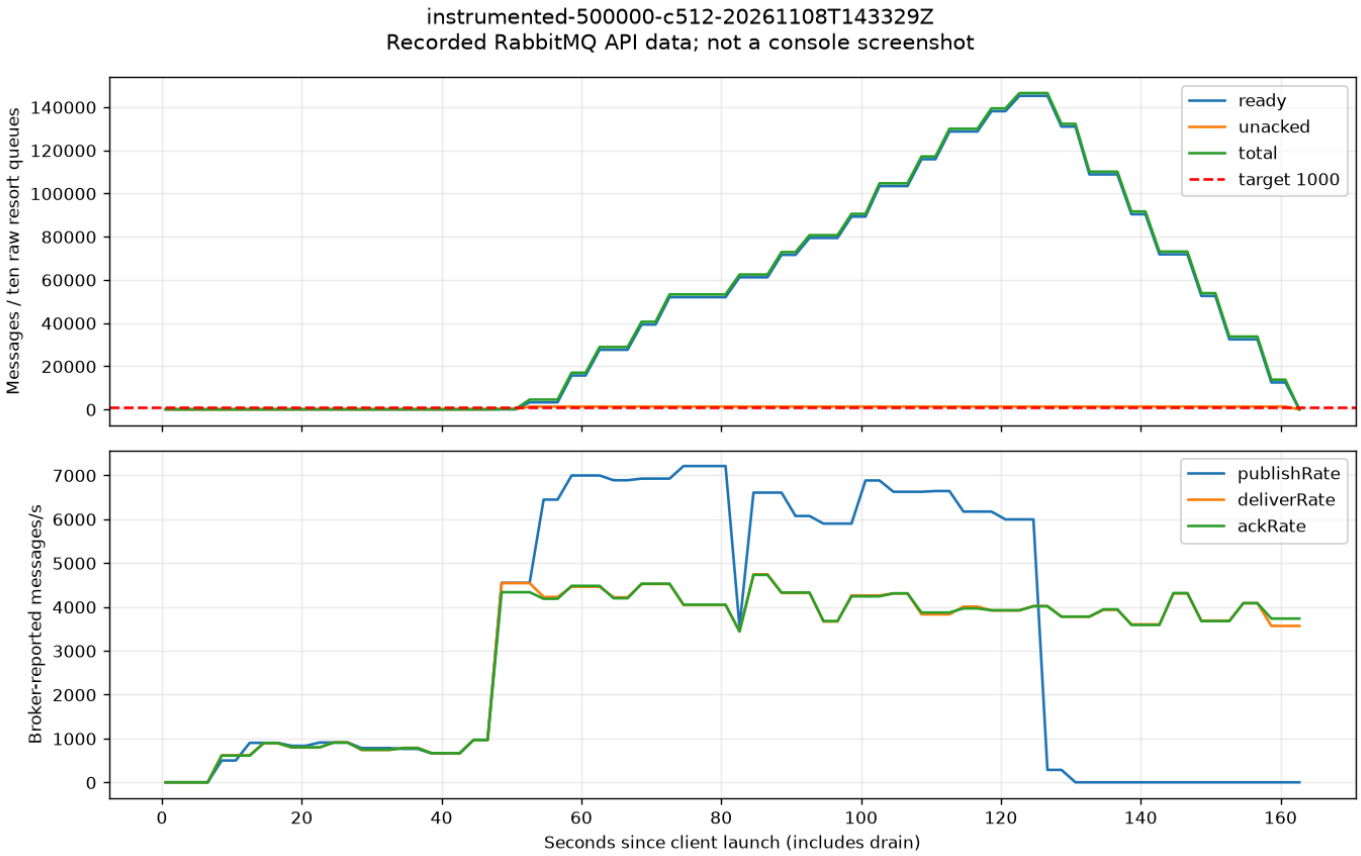
instrumented-500000-c512-20261108T143329Z

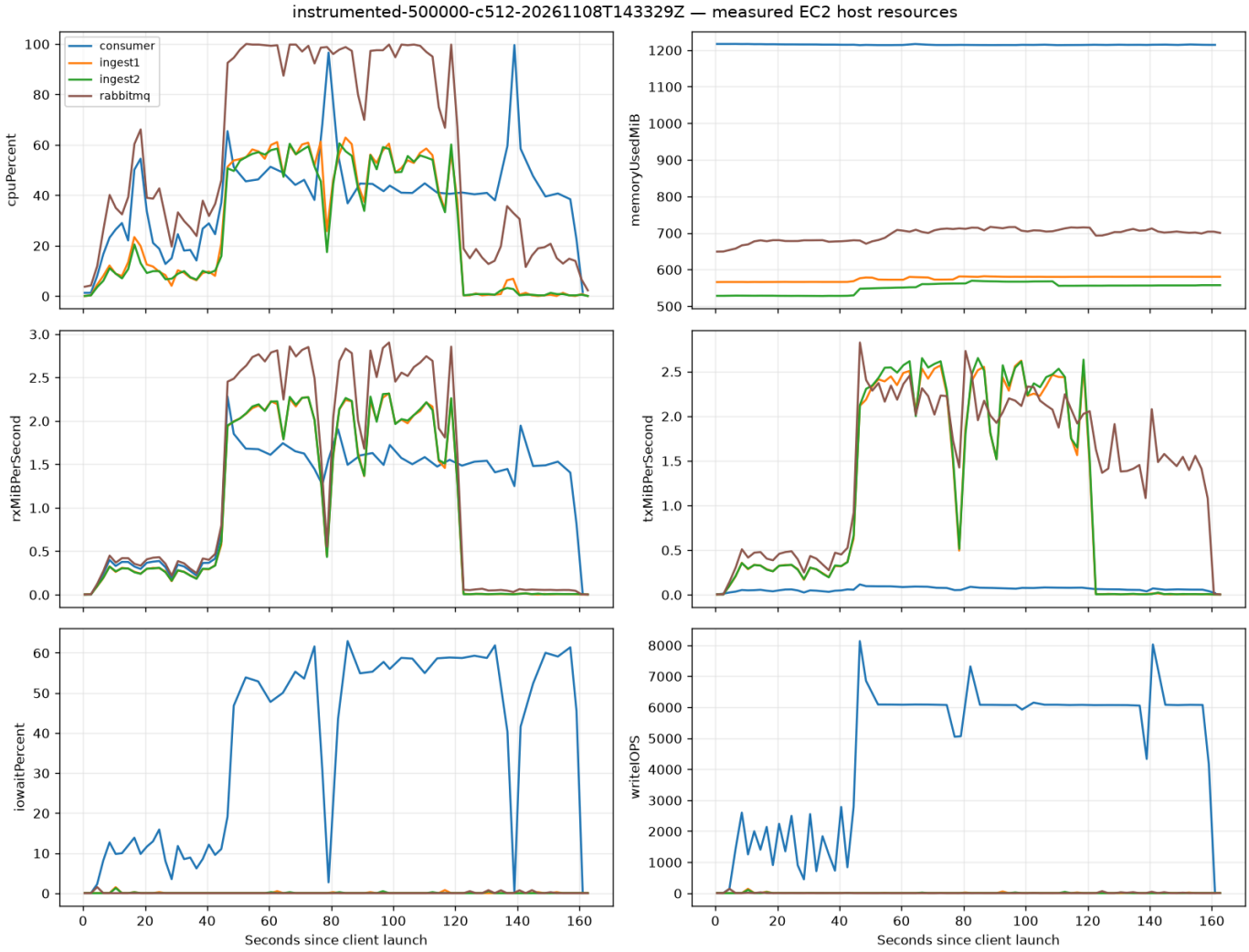
UTC: 2026-11-08T14:33:45.429523+00:00 to 2026-11-08T14:36:28.147351+00:00. Ingests=2; clientThreads=512; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

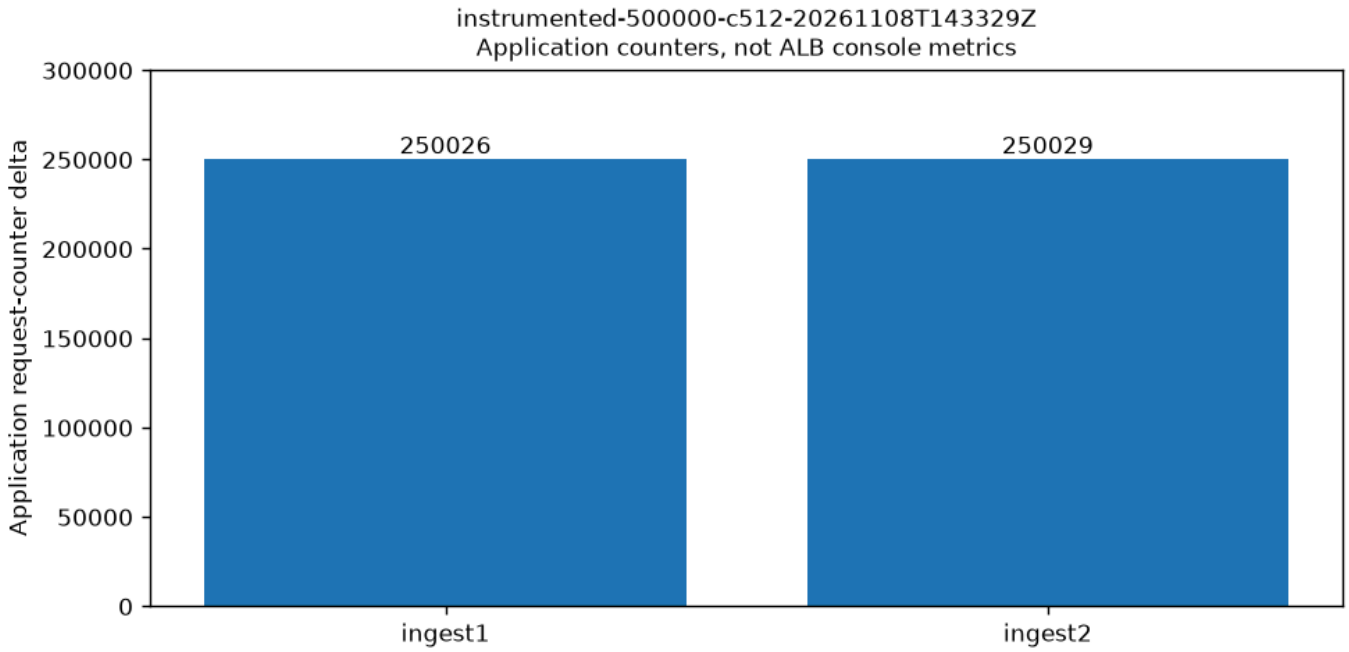
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	117.156
Program runtime seconds	161.400
Successful / final failed / retries	500,000 / 0 / 55
HTTP mean / p95 / p99 ms	80.52 / 172.69 / 314.39
Ready sampled mean / peak	49,459.70 / 145,164
Ready + Unacked peak	146,444
Mean broker publish / delivery / ACK rates per second	3,217.80 / 3,067.16 / 3,065.21
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	903 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

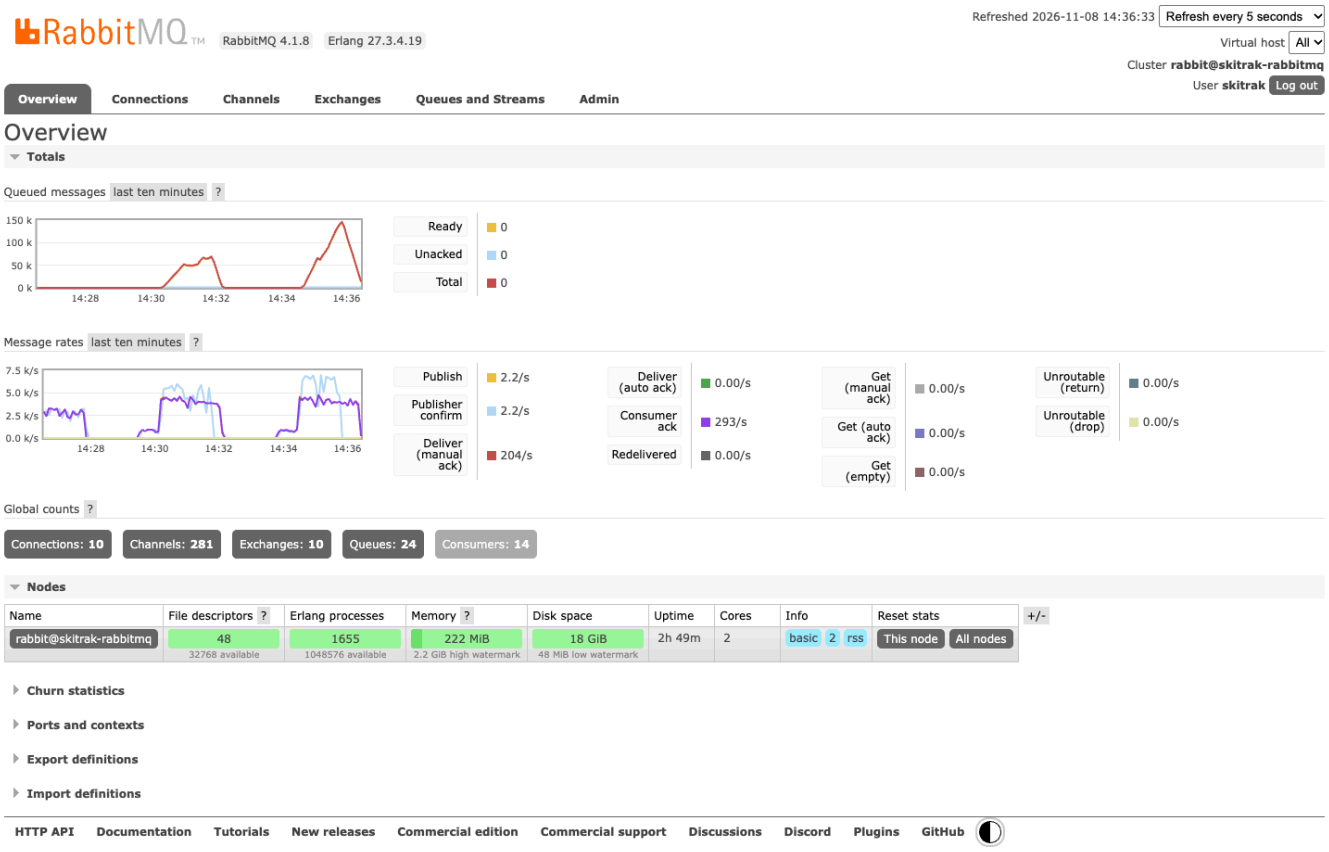

instrumented-500000-c512-20261108T143329Z
Recorded RabbitMQ API data; not a console screenshot
Messages / ten raw resort queues
ready
unacked
total
target 1000
Broker-reported messages/s
publishRate
deliverRate
ackRate
Seconds since client launch (includes drain)



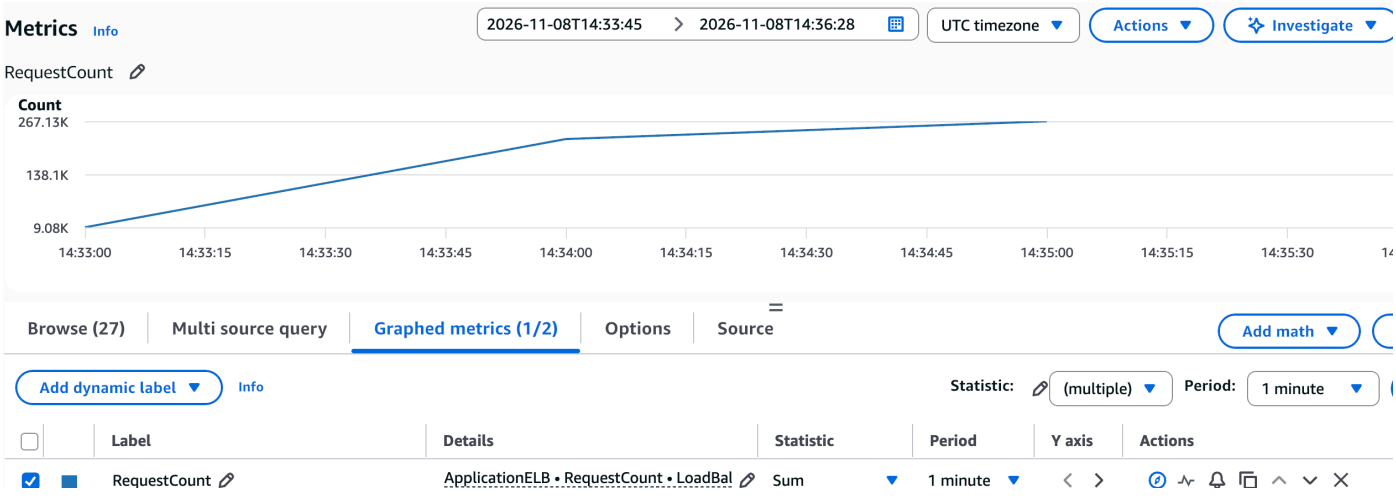


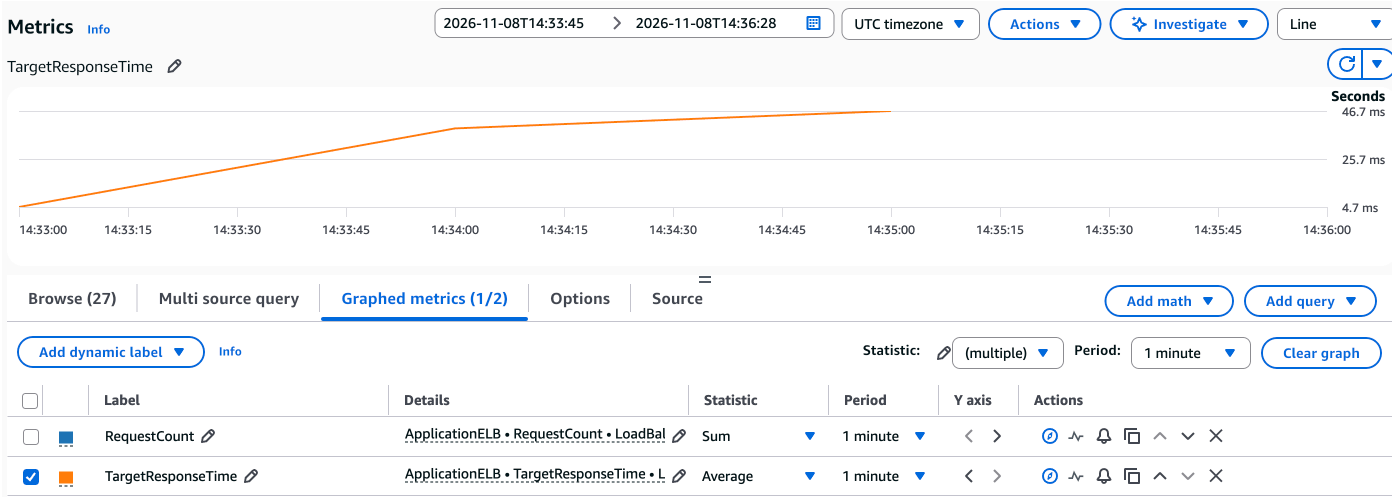
Actual RabbitMQ console captures:

- rabbitmq-20261108T143345177119Z.png
- rabbitmq-20261108T143430551073Z.png
- rabbitmq-20261108T143515830322Z.png
- rabbitmq-20261108T143601173864Z.png
- rabbitmq-20261108T143634584570Z-final.png



Supplied AWS CloudWatch captures:





Collection gaps: consumer collection errors: {"timeout": 16}

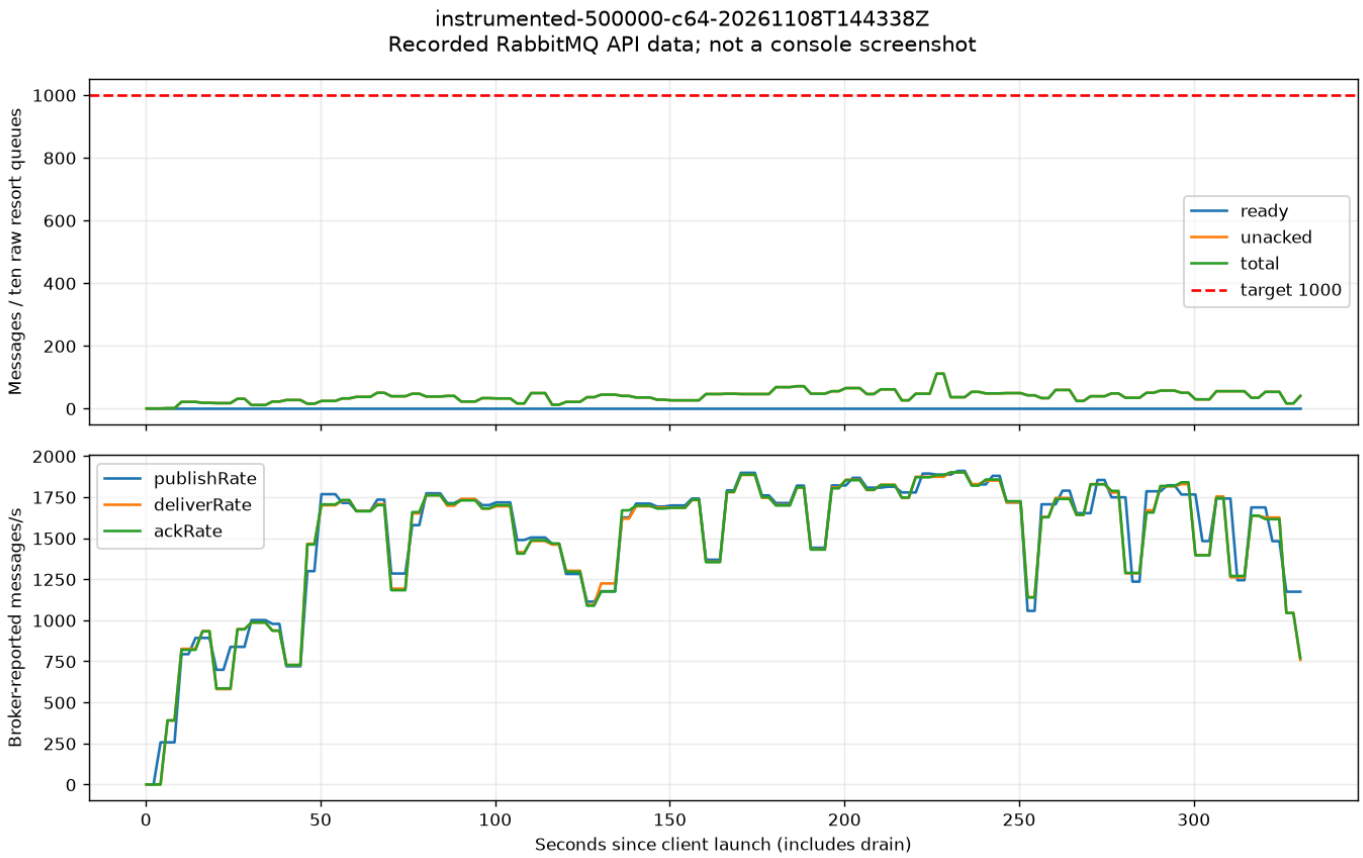
instrumented-500000-c64-20261108T144338Z

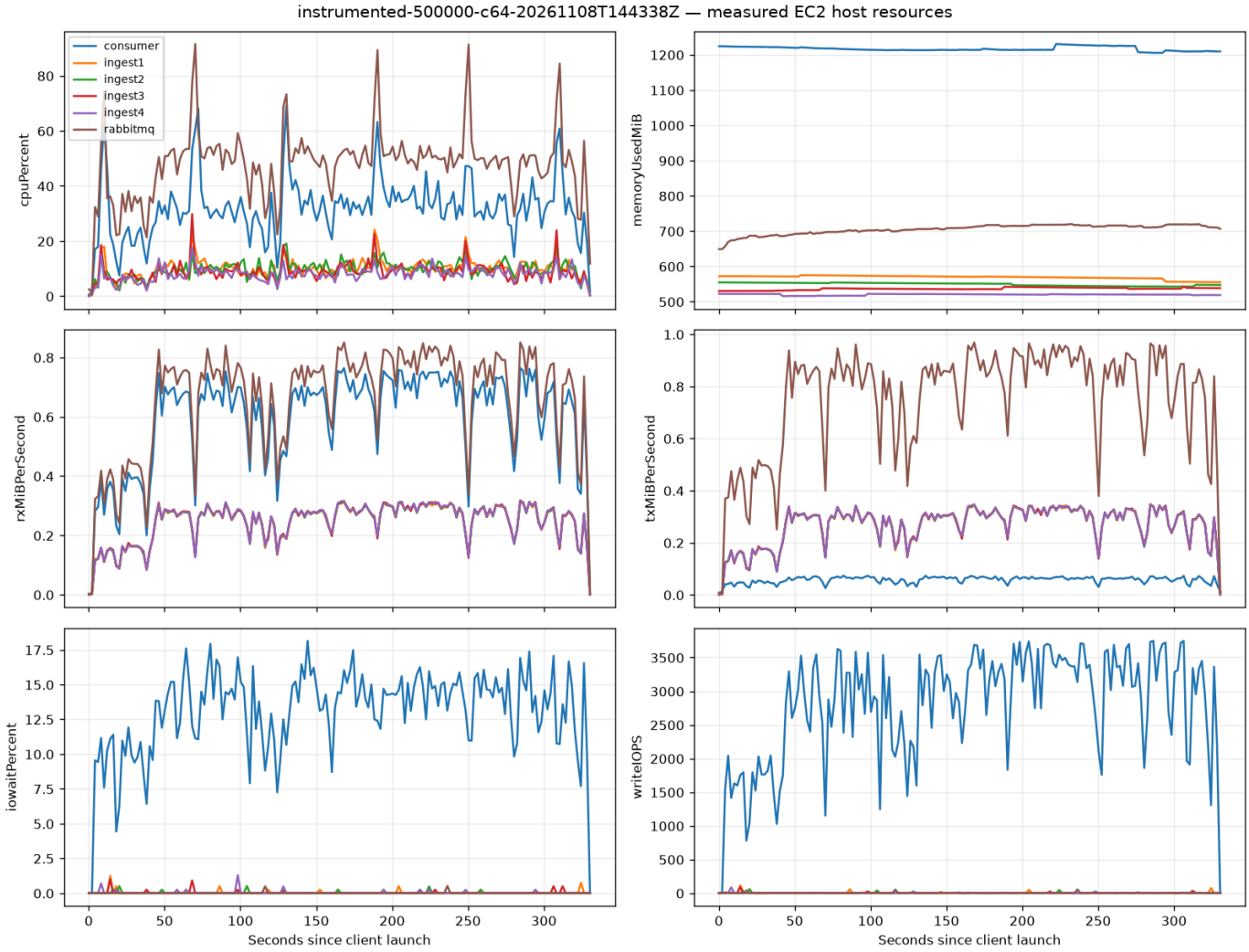
UTC: 2026-11-08T14:43:53.547961+00:00 to 2026-11-08T14:49:25.707862+00:00. Ingests=4; clientThreads=64; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

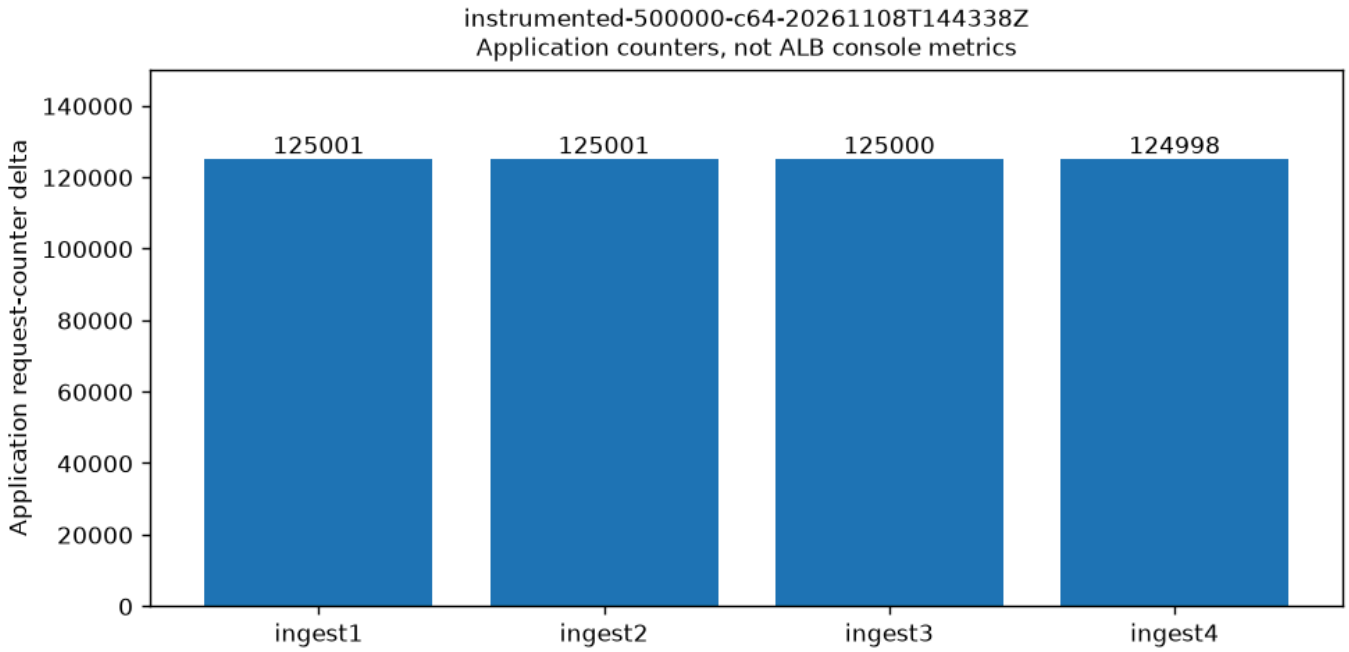
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	325.391
Program runtime seconds	330.596
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	39.13 / 56.60 / 155.48
Ready sampled mean / peak	0.00 / 0
Ready + Unacked peak	111
Mean broker publish / delivery / ACK rates per second	1,509.88 / 1,493.06 / 1,492.61
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	905 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

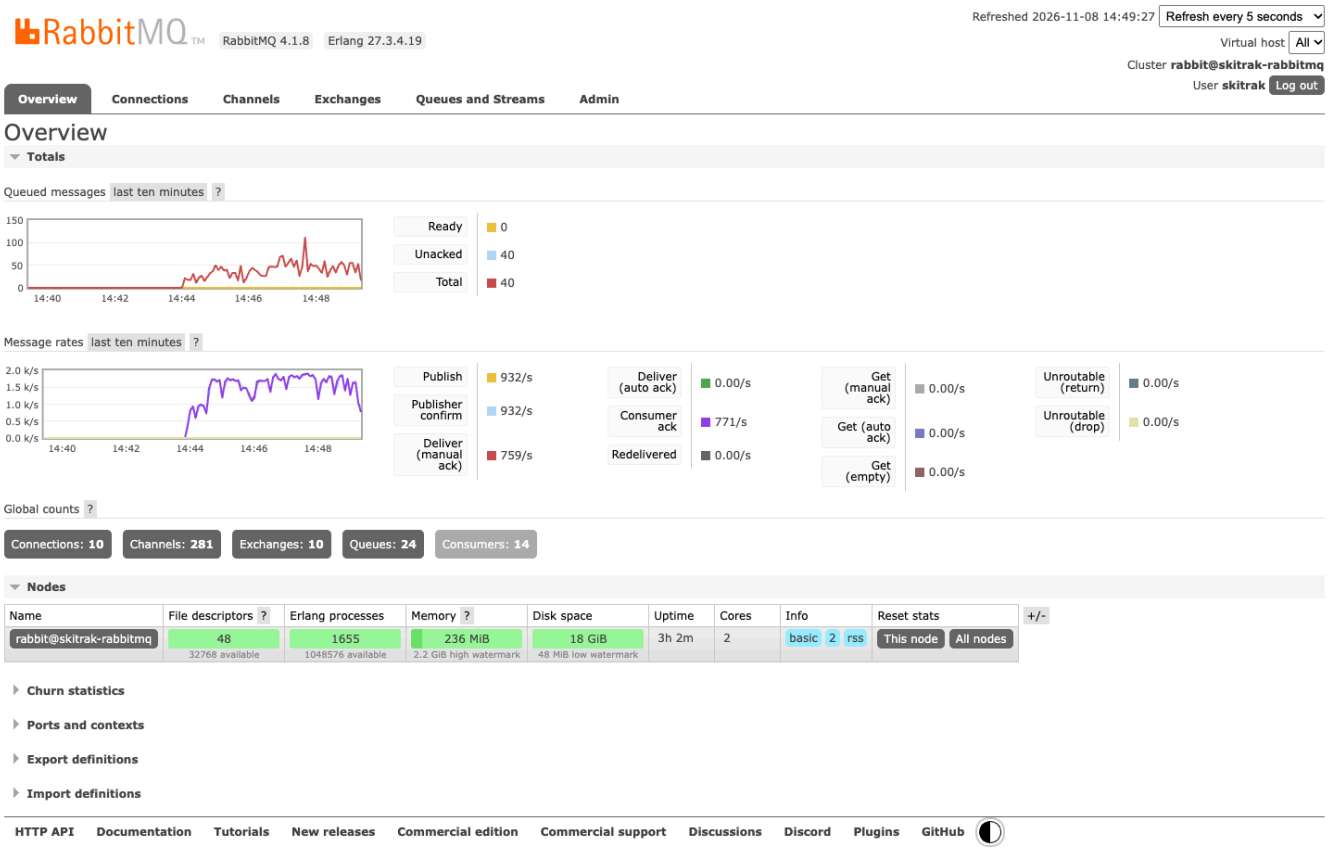

instrumented-500000-c64-20261108T144338Z
Recorded RabbitMQ API data; not a console screenshot
Messages / ten raw resort queues
ready
unacked
total
target 1000
Broker-reported messages/s
publishRate
deliverRate
ackRate
Seconds since client launch (includes drain)





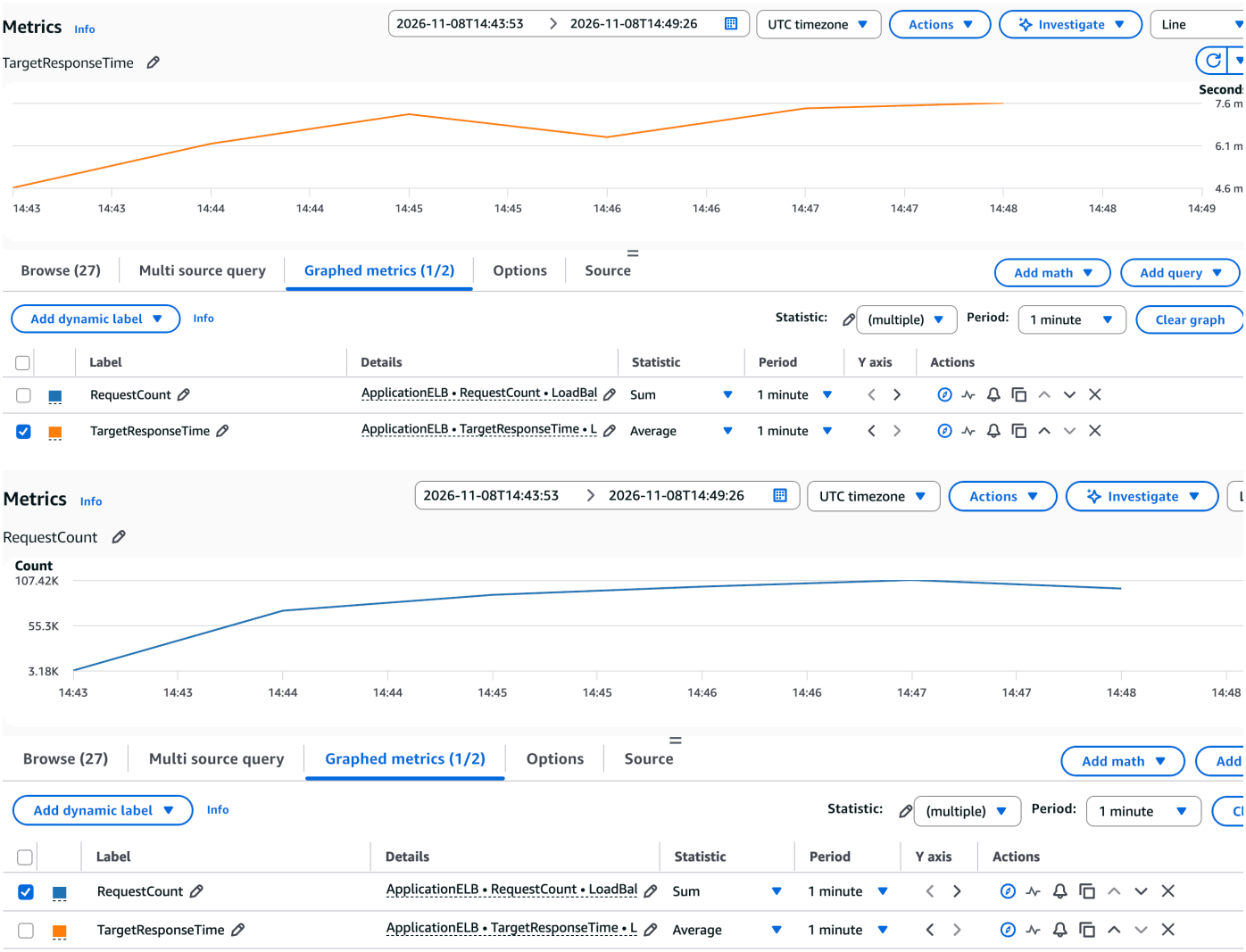
Actual RabbitMQ console captures:

- rabbitmq-20261108T144353330749Z.png
- rabbitmq-20261108T144438860476Z.png
- rabbitmq-20261108T144524157640Z.png
- rabbitmq-20261108T144609311605Z.png
- rabbitmq-20261108T144654750139Z.png
- rabbitmq-20261108T144740234488Z.png
- rabbitmq-20261108T144825361876Z.png
- rabbitmq-20261108T144910816459Z.png
- rabbitmq-20261108T144931869234Z-final.png



Supplied AWS CloudWatch captures:

Verified LoadBalancer aggregate; 2026-11-08 14:43:53–14:49:26 UTC. RequestCount Sum, TargetResponseTime Average, period 60 seconds.



Collection gaps: None detected in the recorded collectors.

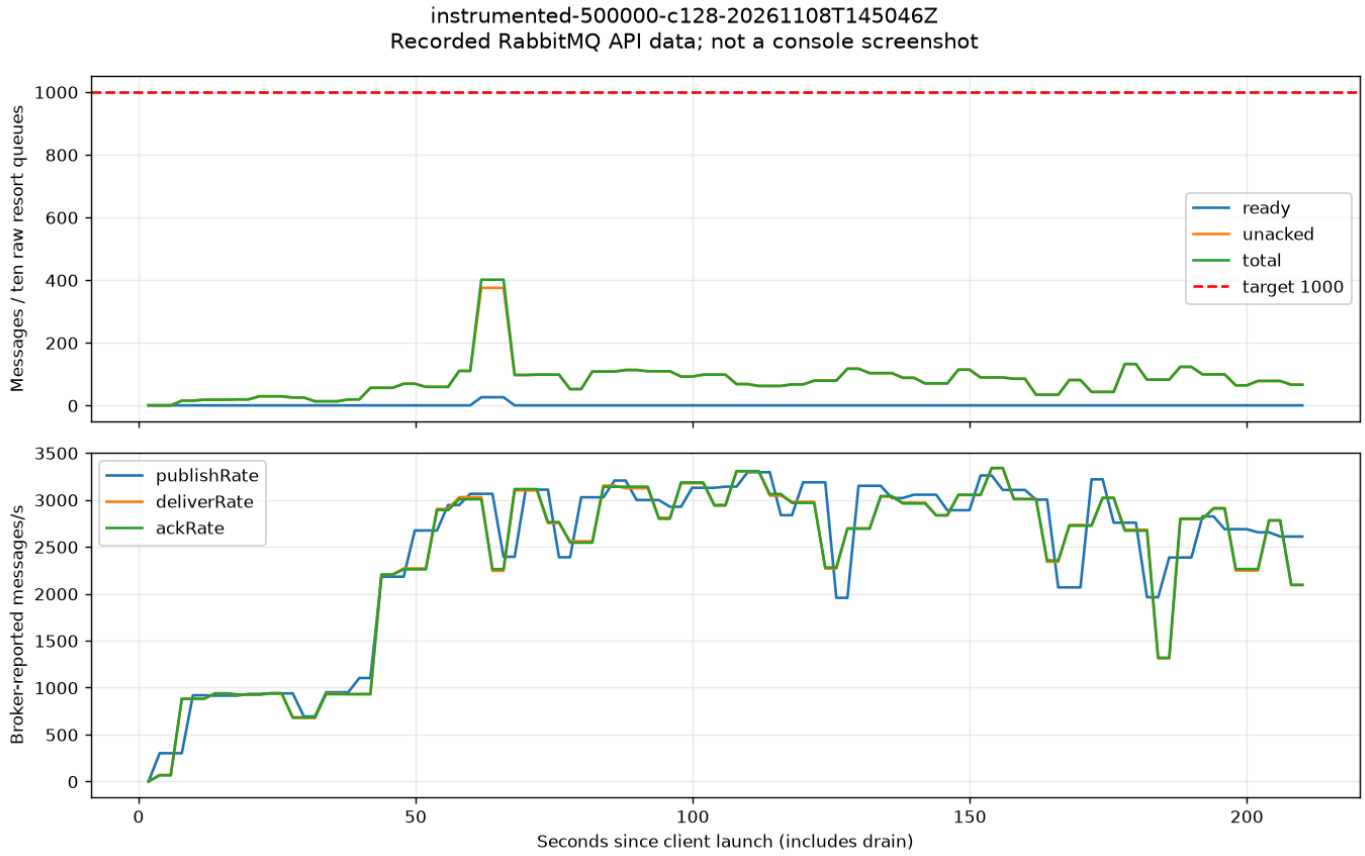
instrumented-500000-c128-20261108T145046Z

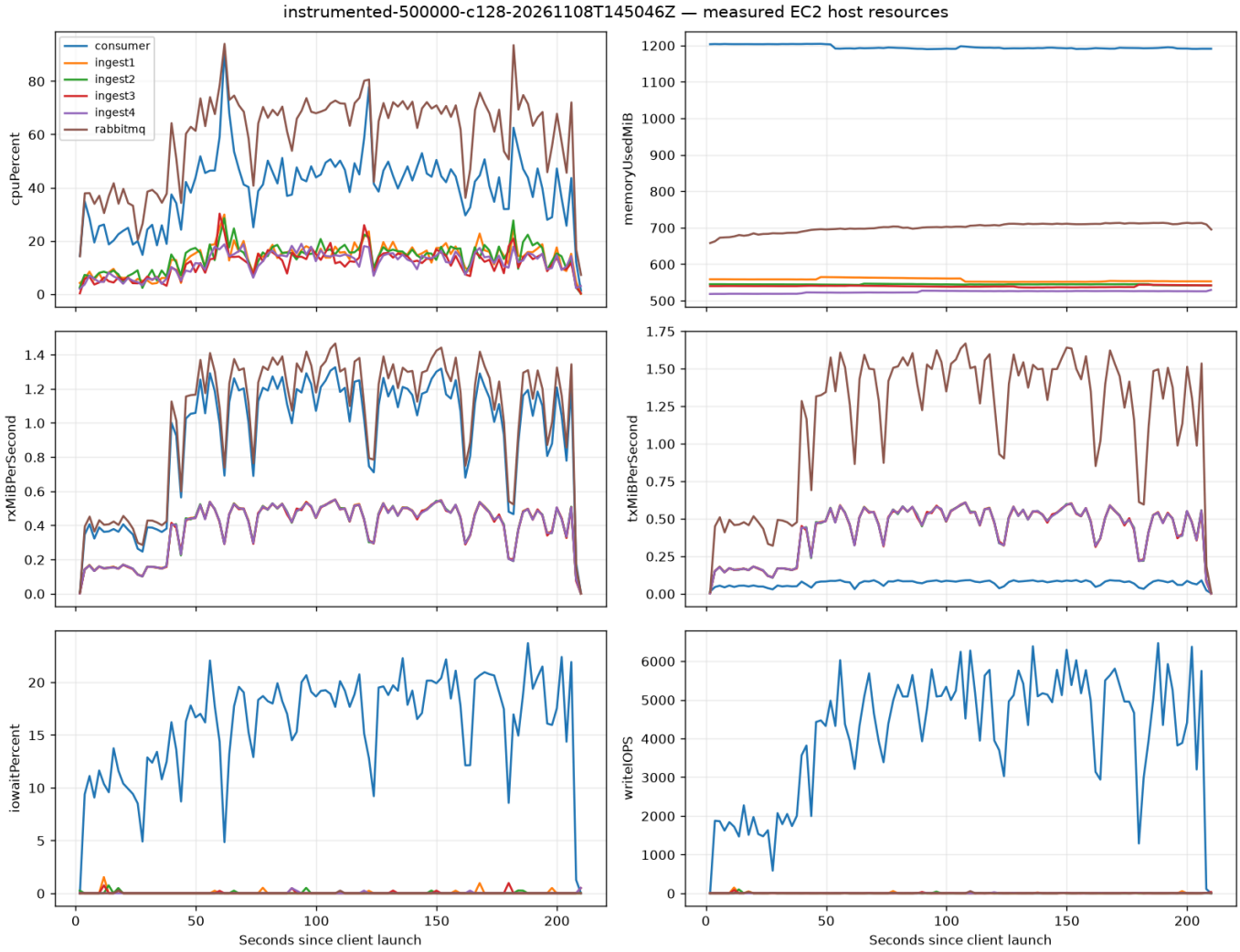
UTC: 2026-11-08T14:51:01.754633+00:00 to 2026-11-08T14:54:31.871816+00:00. Ingests=4; clientThreads=128; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

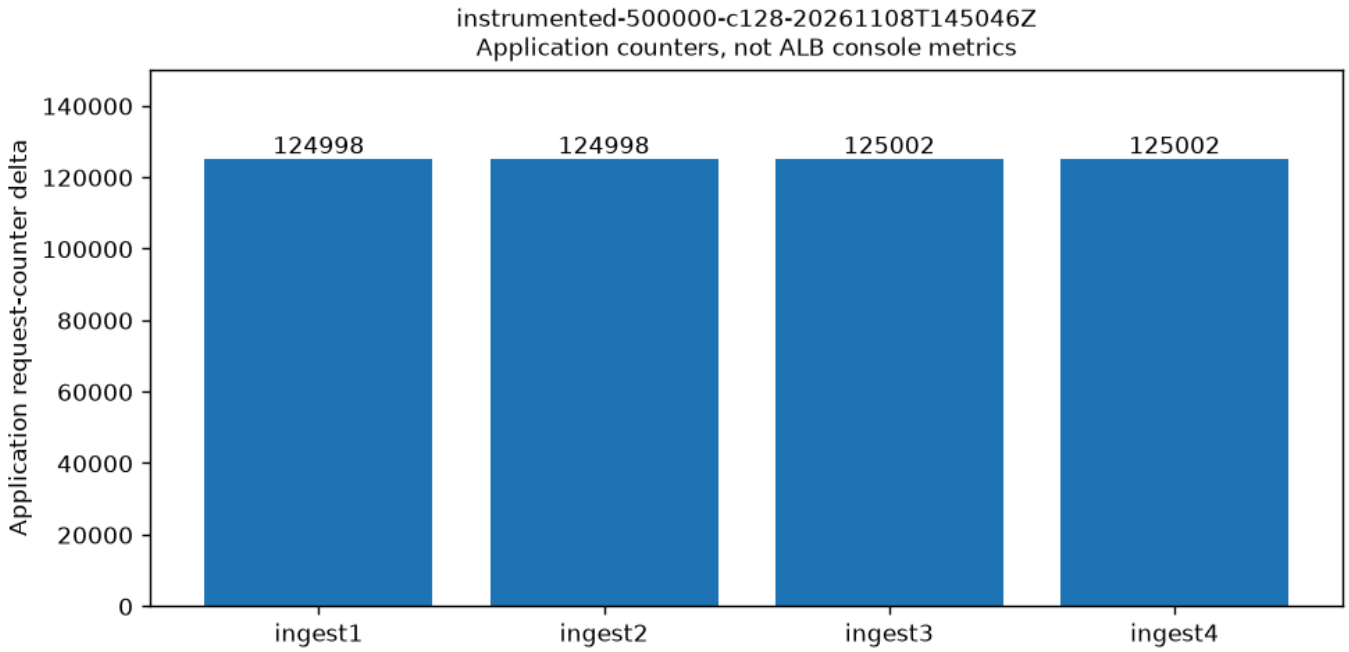
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	204.630
Program runtime seconds	208.626
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	45.35 / 71.79 / 202.31
Ready sampled mean / peak	0.74 / 26
Ready + Unacked peak	401
Mean broker publish / delivery / ACK rates per second	2,420.33 / 2,372.64 / 2,372.49
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	910 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

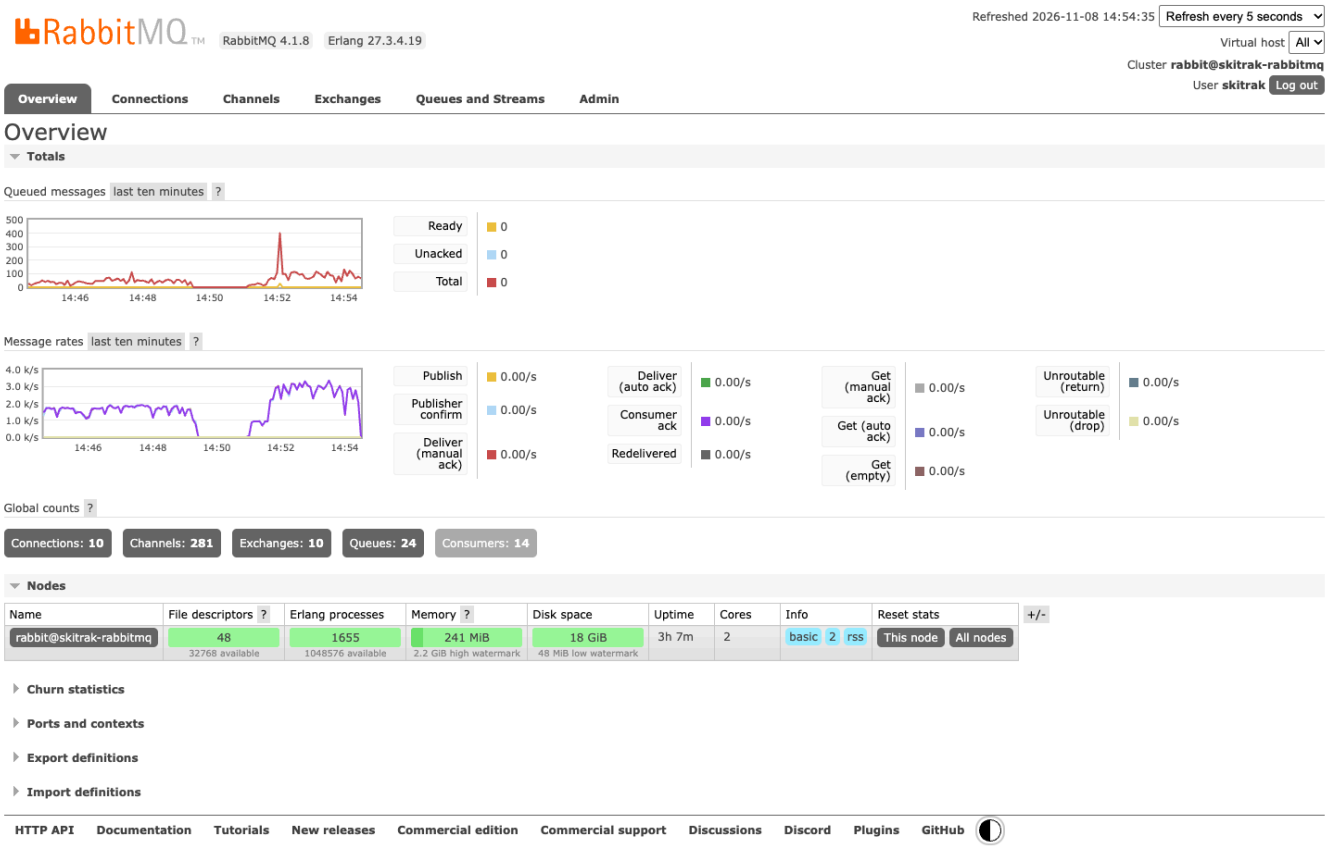






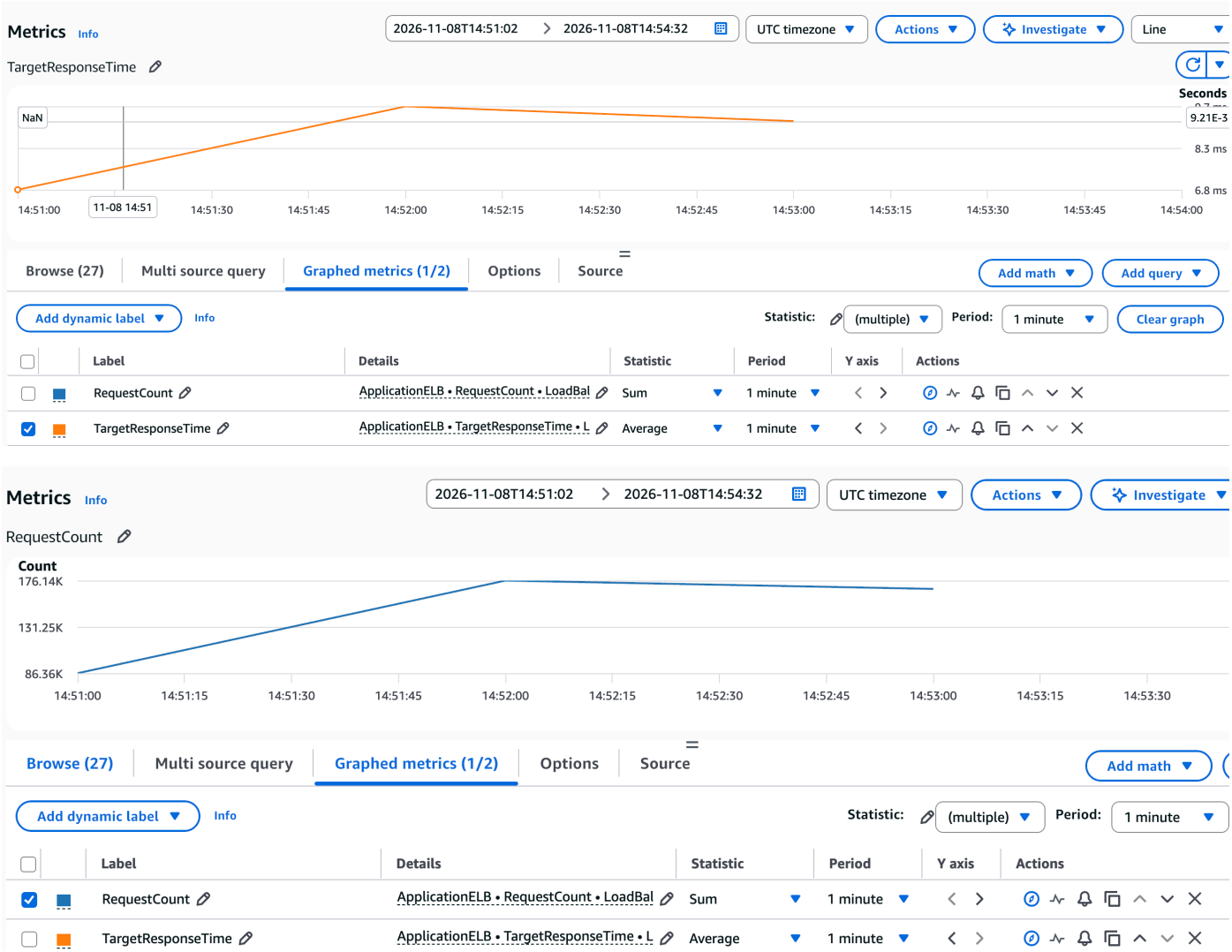
Actual RabbitMQ console captures:

- rabbitmq-20261108T145101352639Z.png
- rabbitmq-20261108T145146485707Z.png
- rabbitmq-20261108T145231691648Z.png
- rabbitmq-20261108T145317285102Z.png
- rabbitmq-20261108T145402547815Z.png
- rabbitmq-20261108T145438041284Z-final.png



Supplied AWS CloudWatch captures:

Verified LoadBalancer aggregate; 2026-11-08 14:51:02–14:54:32 UTC. RequestCount Sum, TargetResponseTime Average, period 60 seconds.



Collection gaps: None detected in the recorded collectors.

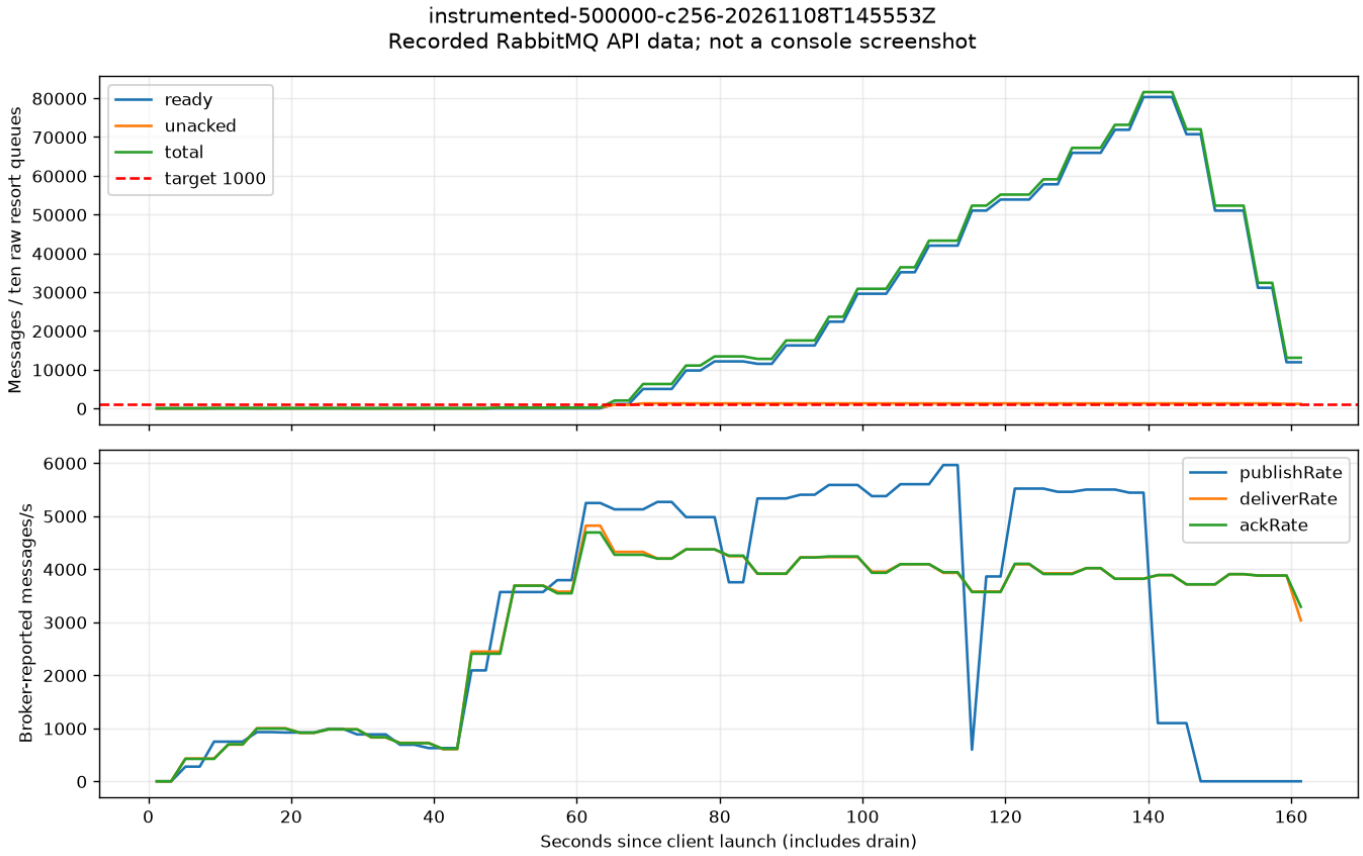
instrumented-500000-c256-20261108T145553Z

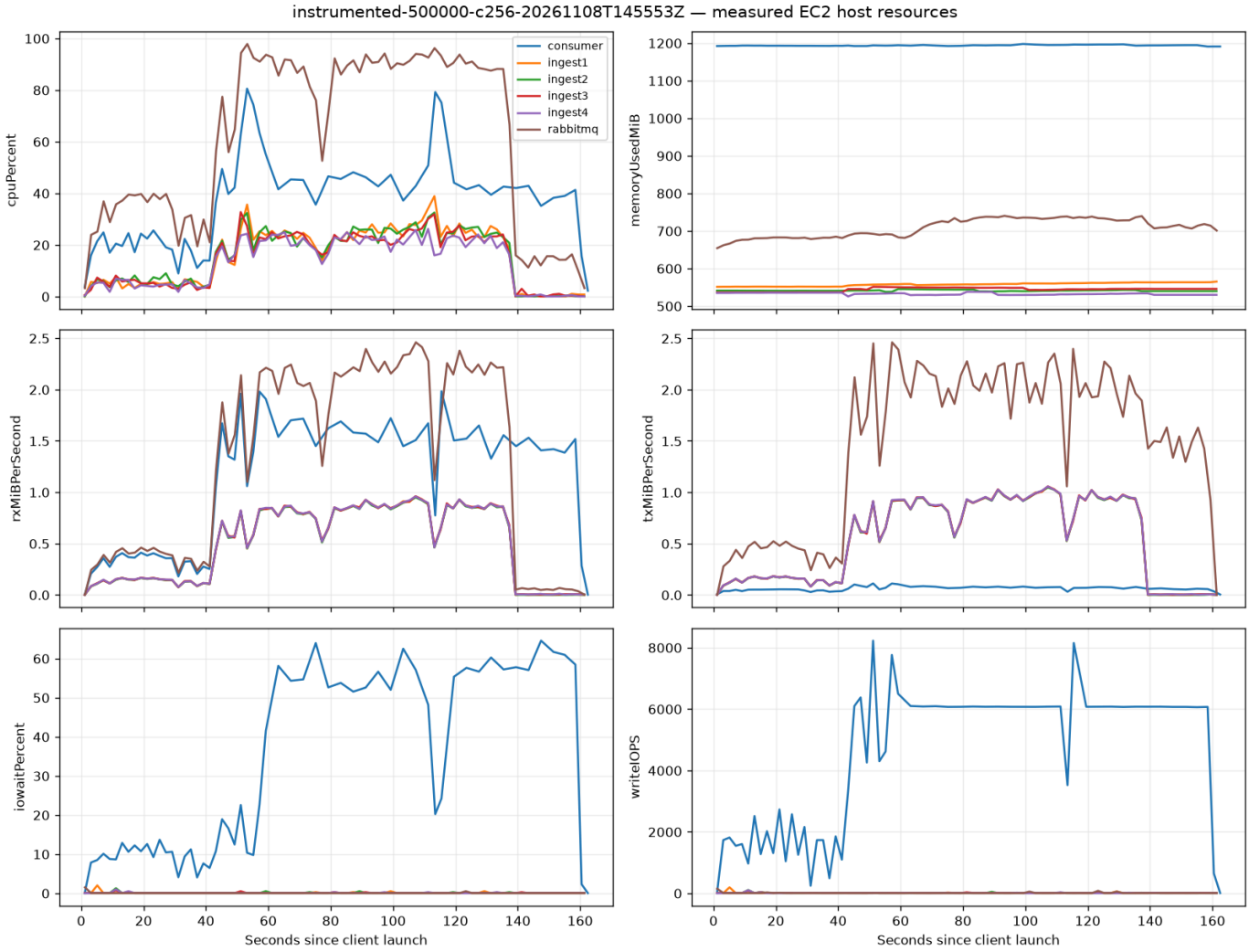
UTC: 2026-11-08T14:56:09.658997+00:00 to 2026-11-08T14:58:52.520694+00:00. Ingests=4; clientThreads=256; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

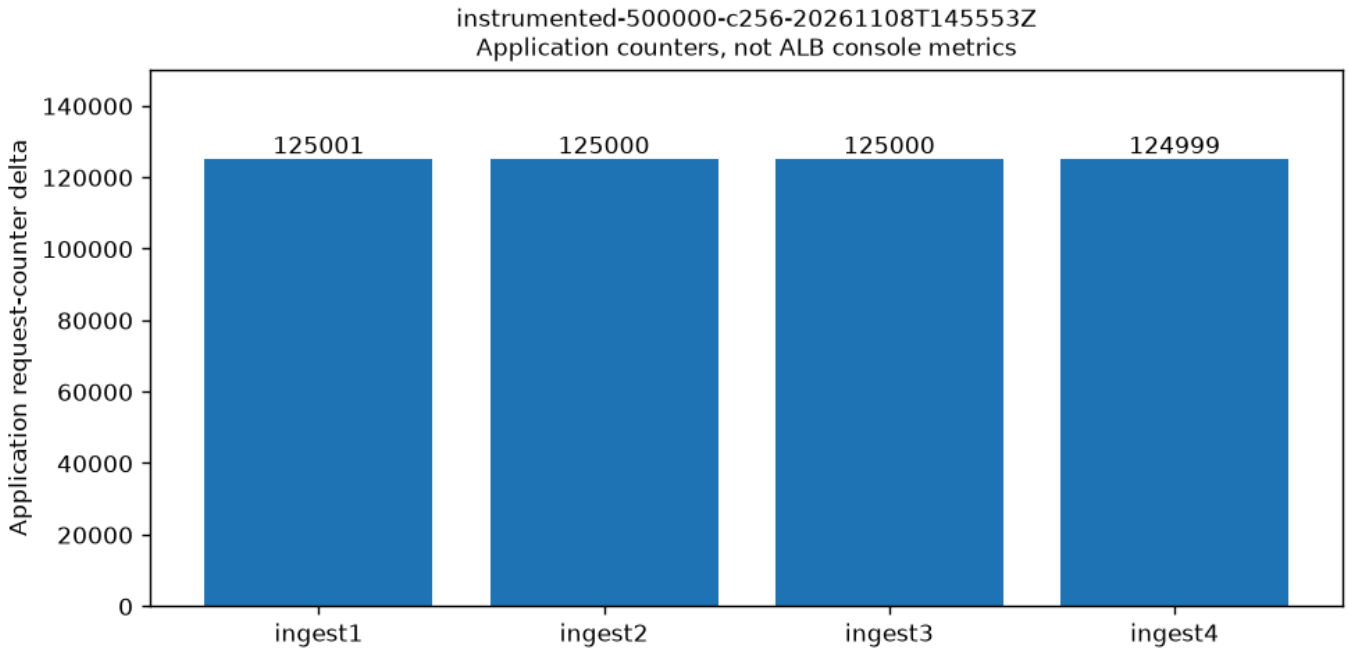
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	134.703
Program runtime seconds	161.983
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	51.12 / 82.12 / 194.77
Ready sampled mean / peak	22,422.90 / 80,309
Ready + Unacked peak	81,589
Mean broker publish / delivery / ACK rates per second	3,082.02 / 3,037.82 / 3,033.15
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	913 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

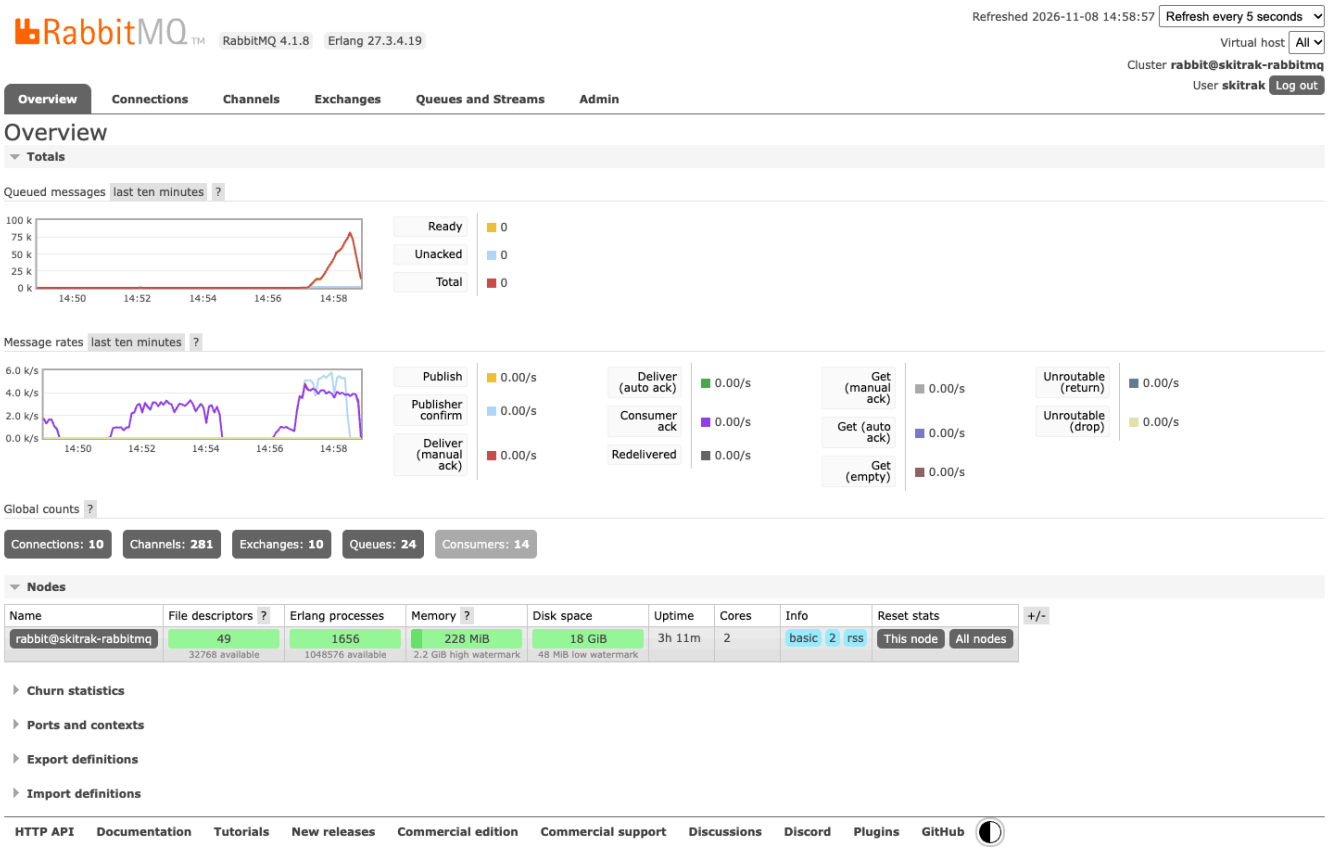






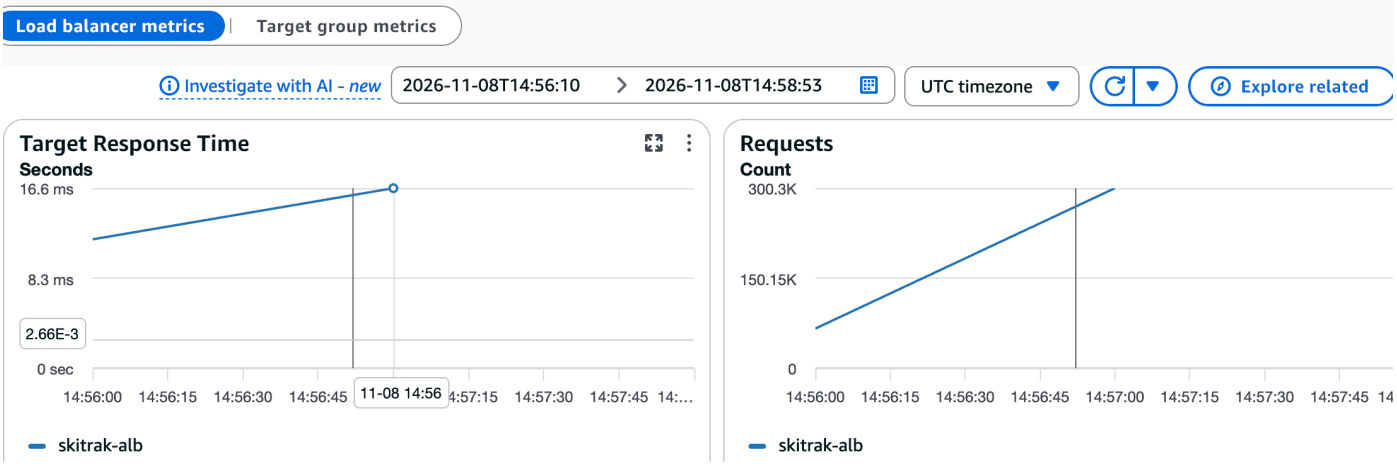
Actual RabbitMQ console captures:

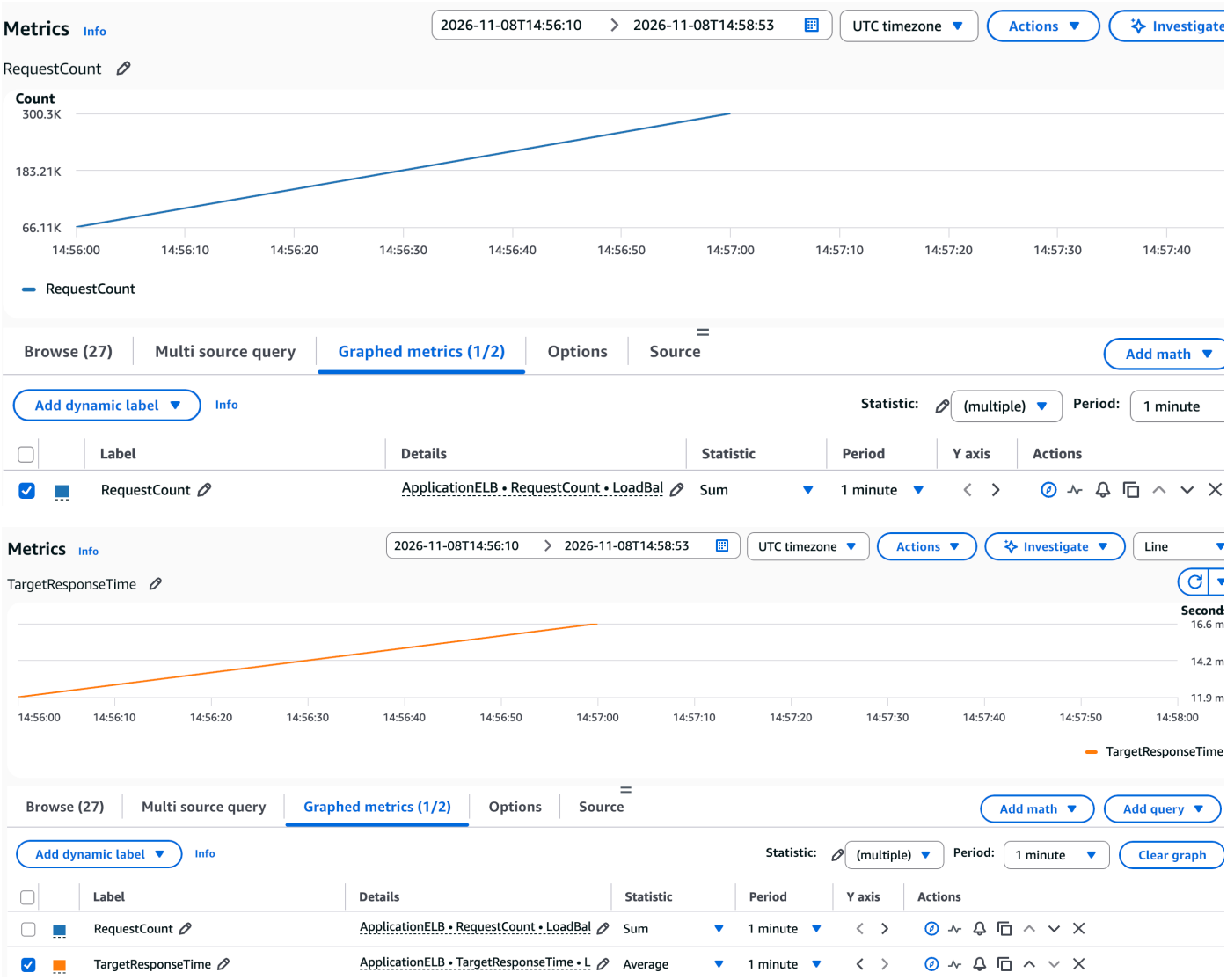
- rabbitmq-20261108T145609330066Z.png
- rabbitmq-20261108T145654462744Z.png
- rabbitmq-20261108T145739597113Z.png
- rabbitmq-20261108T145825092760Z.png
- rabbitmq-20261108T145858916926Z-final.png



Supplied AWS CloudWatch captures:

LoadBalancer aggregate.





Collection gaps: consumer collection errors: {"timeout": 21}

instrumented-500000-c512-20261108T150014Z

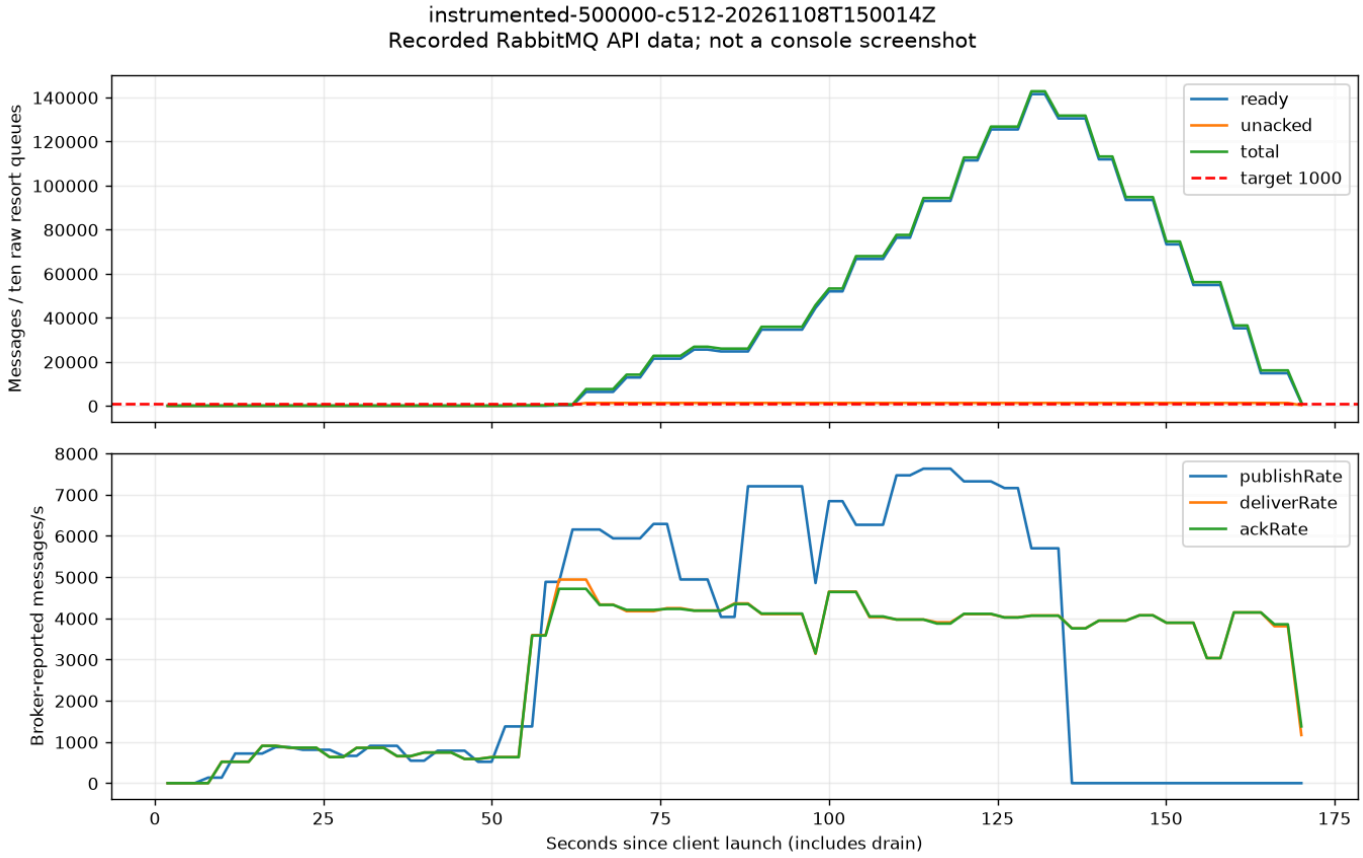
UTC: 2026-11-08T15:00:30.011623+00:00 to 2026-11-08T15:03:20.129150+00:00. Ingests=4; clientThreads=512; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

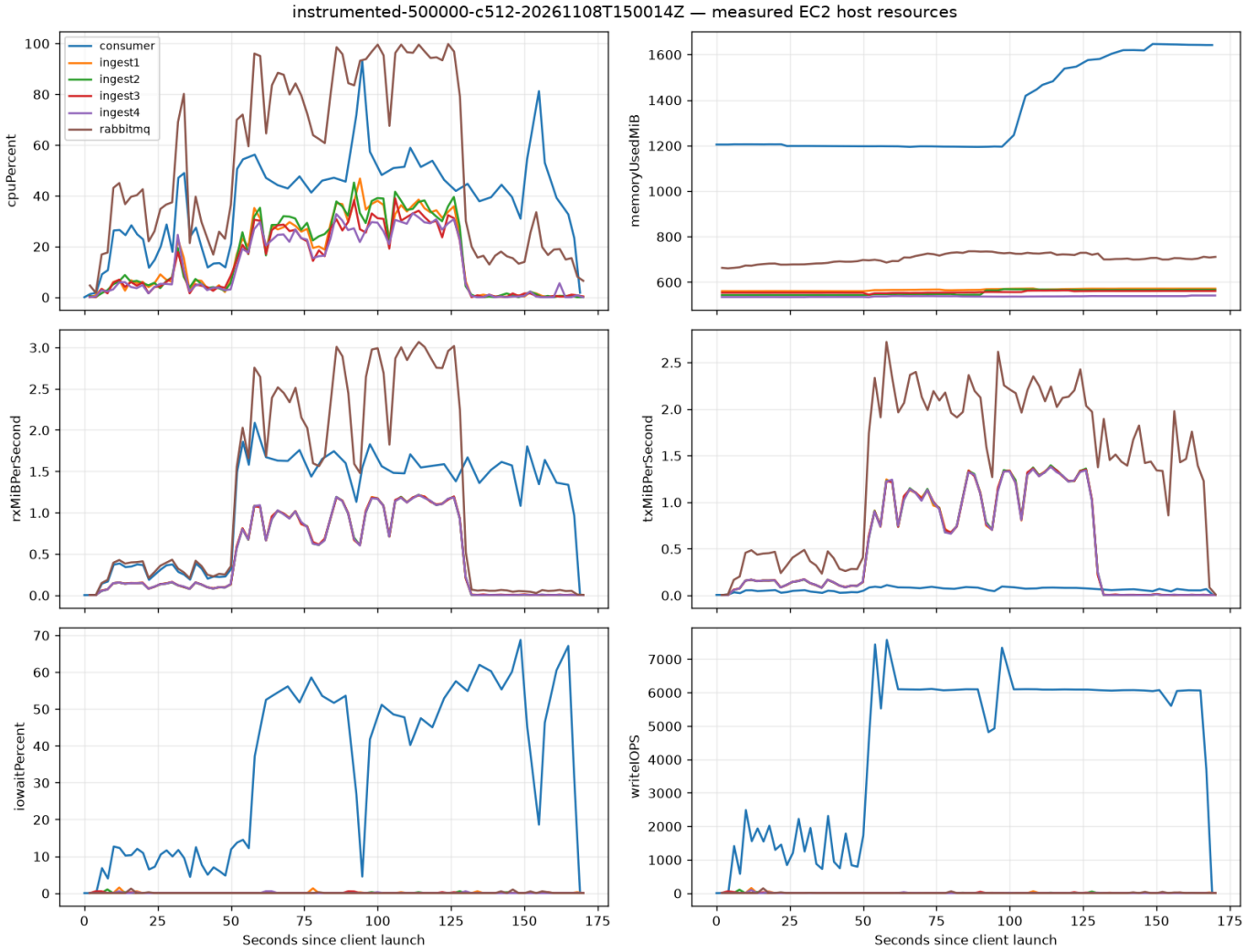
Complete client output | Full metrics, provenance and limits | Evidence page

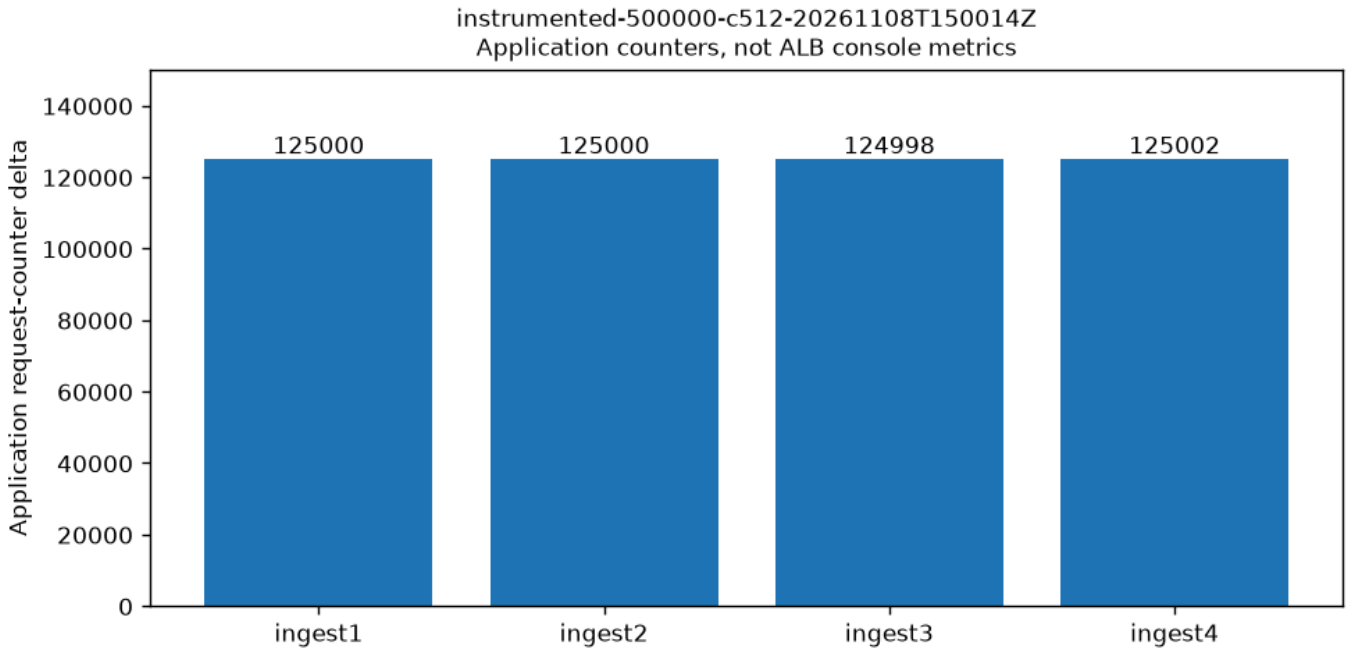
Measurement	Result
Ingestion runtime seconds	124.219
Program runtime seconds	167.748
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	83.85 / 186.00 / 331.42
Ready sampled mean / peak	39,491.69 / 141,495
Ready + Unacked peak	142,775
Mean broker publish / delivery / ACK rates per second	3,121.07 / 2,936.10 / 2,931.31
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0

Measurement	Result
WebSocket updates / reconnections	909 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

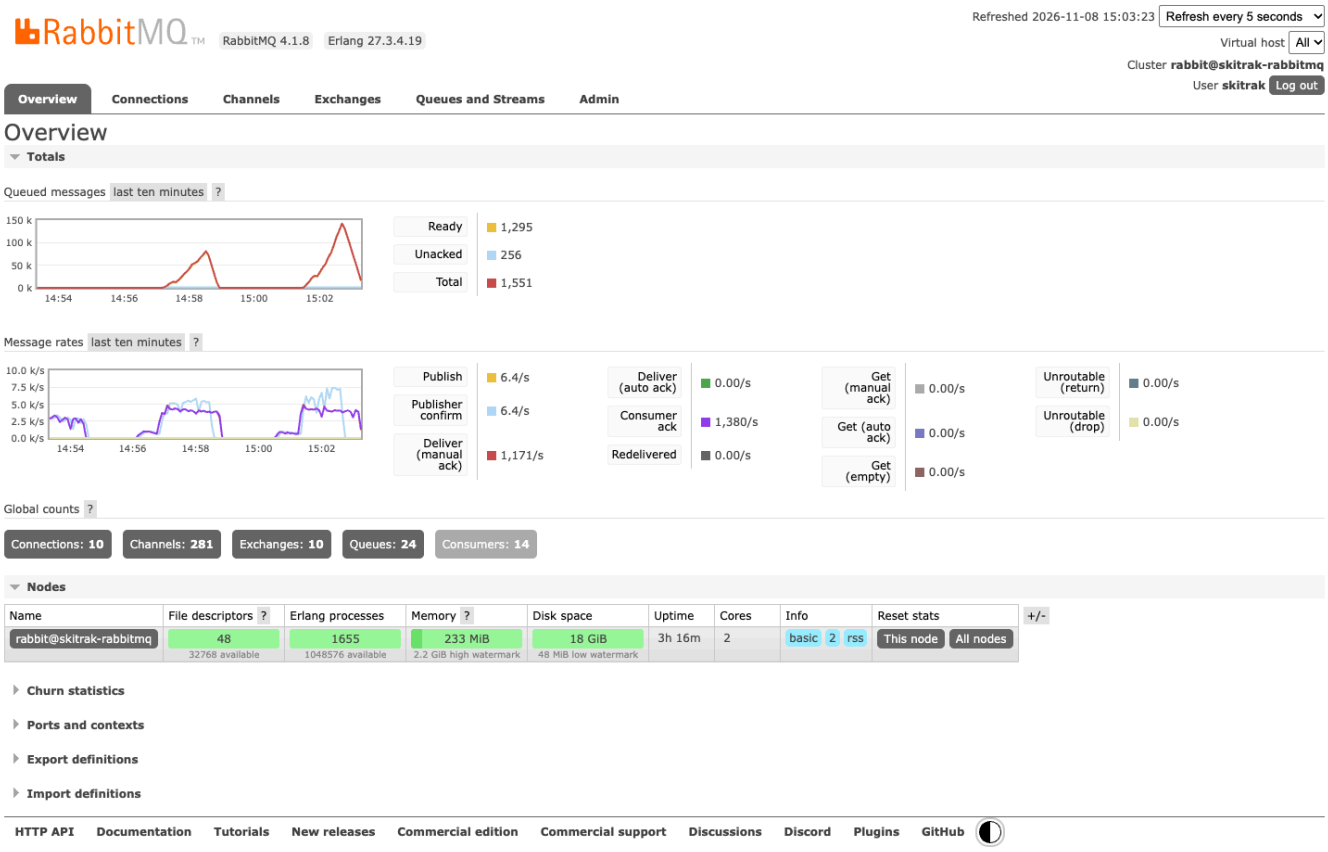






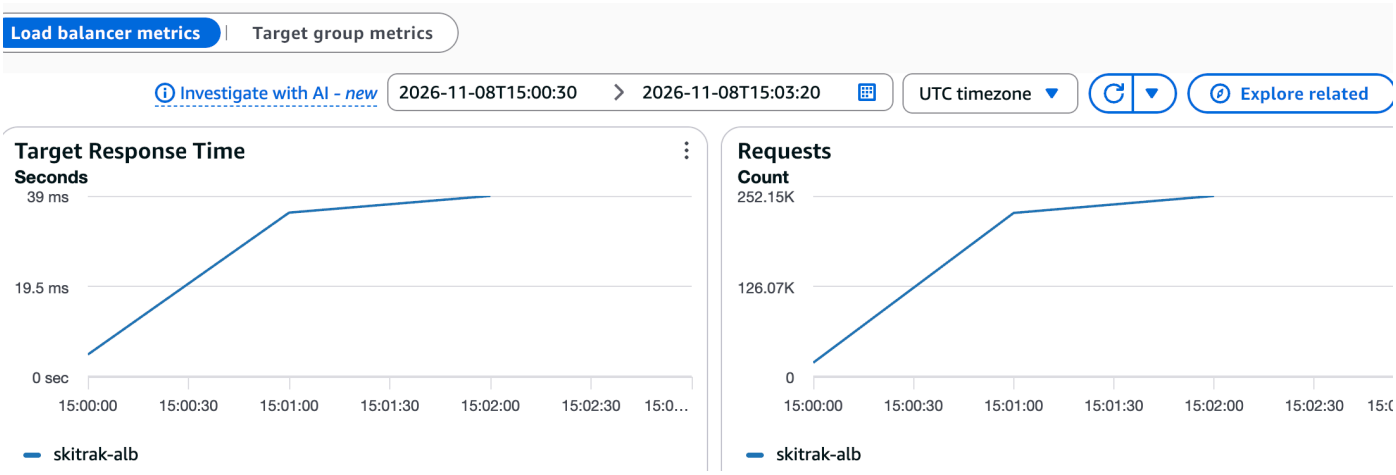
Actual RabbitMQ console captures:

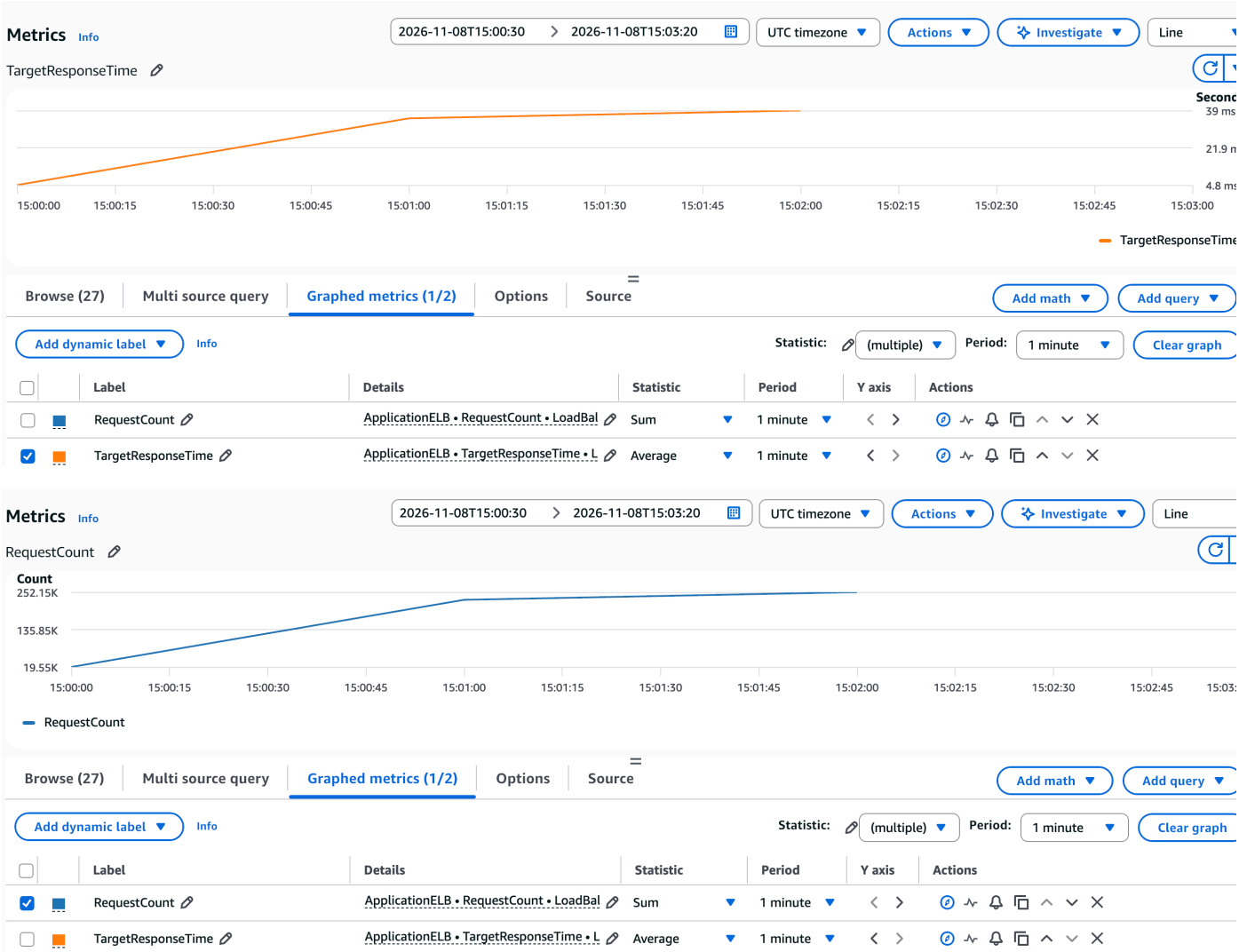
- rabbitmq-20261108T150029782808Z.png
- rabbitmq-20261108T150115543601Z.png
- rabbitmq-20261108T150200973877Z.png
- rabbitmq-20261108T150246370700Z.png
- rabbitmq-20261108T150326534269Z-final.png



Supplied AWS CloudWatch captures:

LoadBalancer aggregate.





Collection gaps: consumer collection errors: {"timeout": 13}

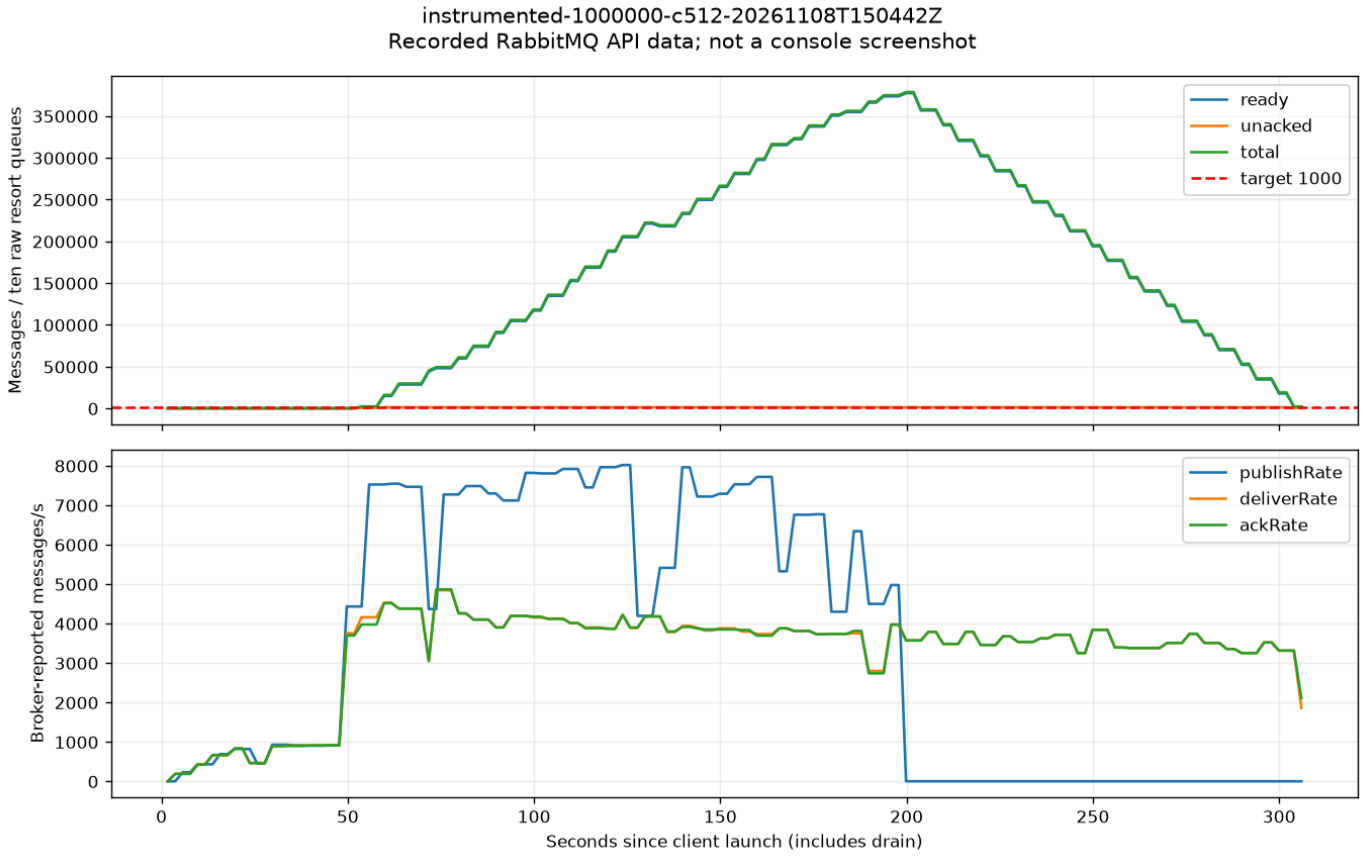
instrumented-1000000-c512-20261108T150442Z

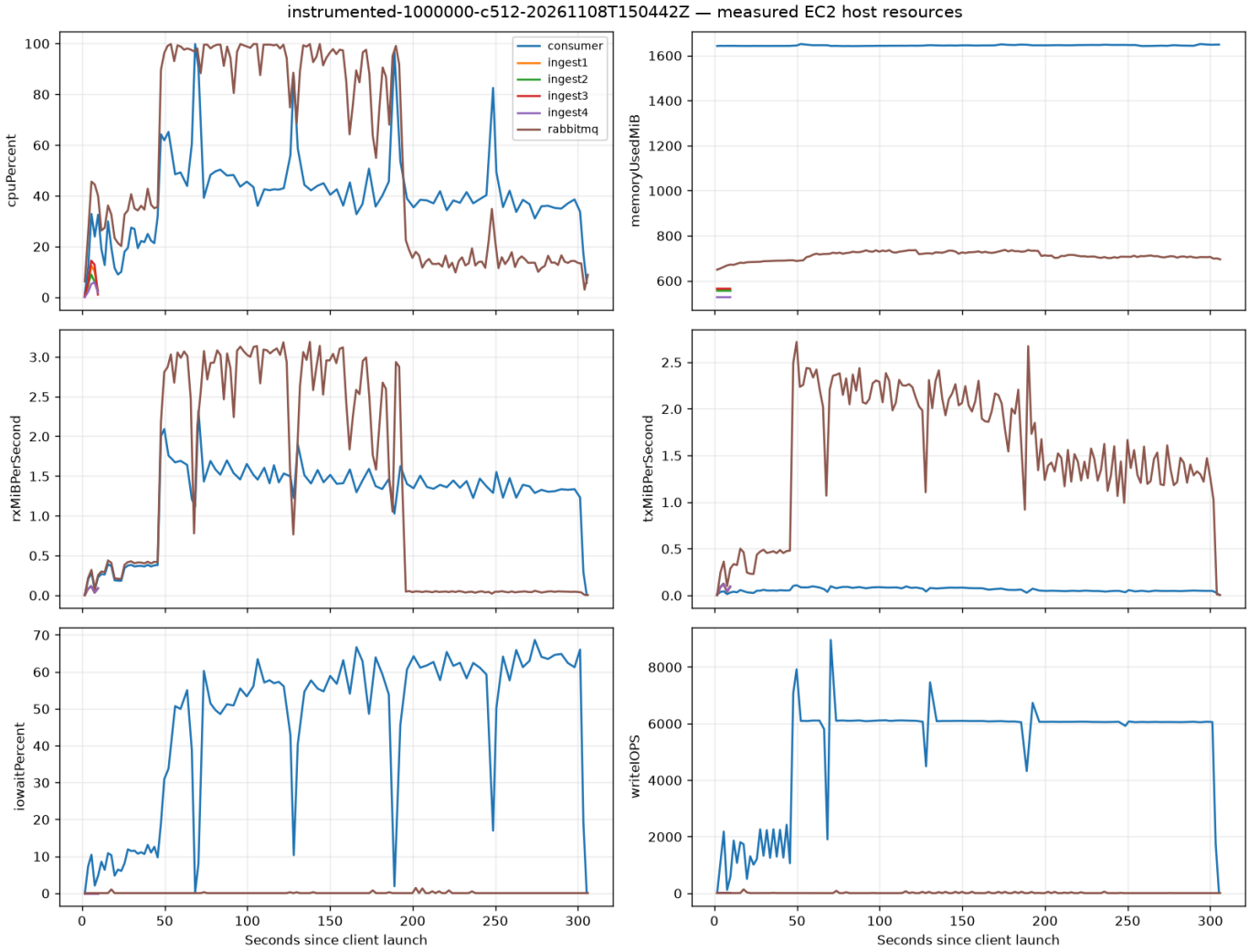
UTC: 2026-11-08T15:04:55.794833+00:00 to 2026-11-08T15:10:02.047298+00:00. Ingests=4; clientThreads=512; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

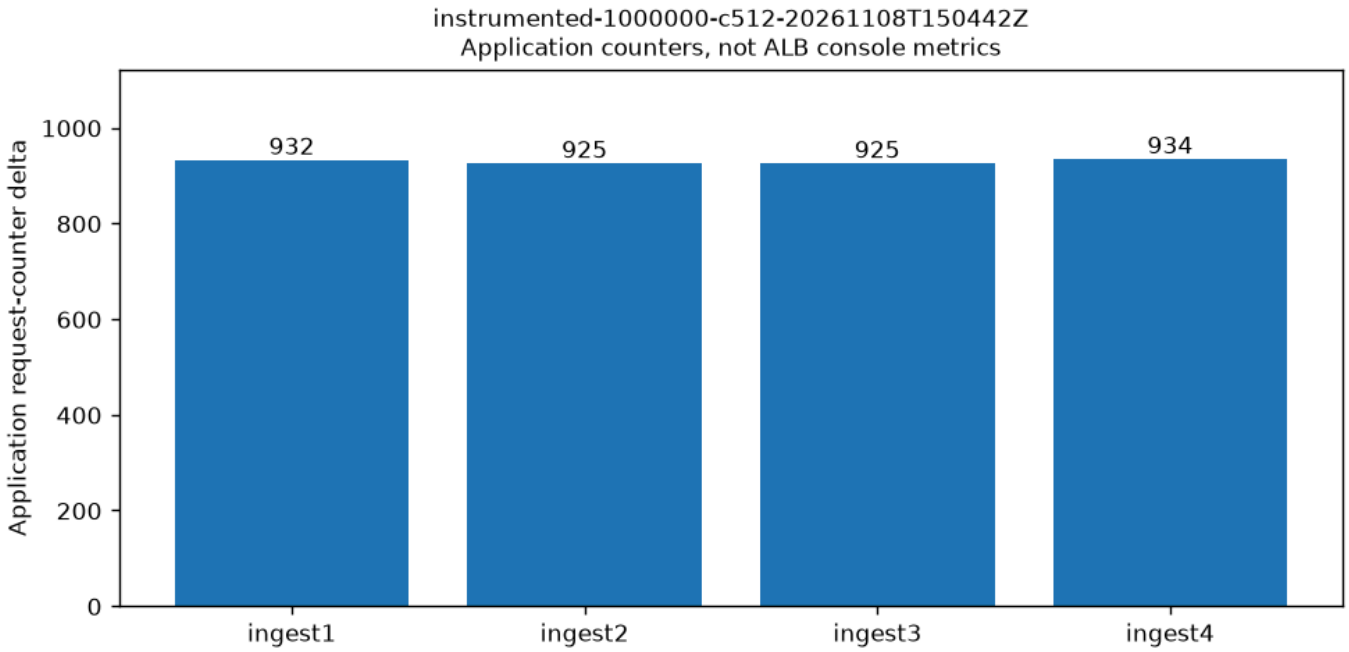
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	190.550
Program runtime seconds	304.875
Successful / final failed / retries	1,000,000 / 0 / 77
HTTP mean / p95 / p99 ms	76.88 / 142.22 / 312.63
Ready sampled mean / peak	158,344.33 / 377,327
Ready + Unacked peak	378,607
Mean broker publish / delivery / ACK rates per second	3,370.36 / 3,275.11 / 3,271.22
Additional processed / duplicate count	1000000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	1023 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

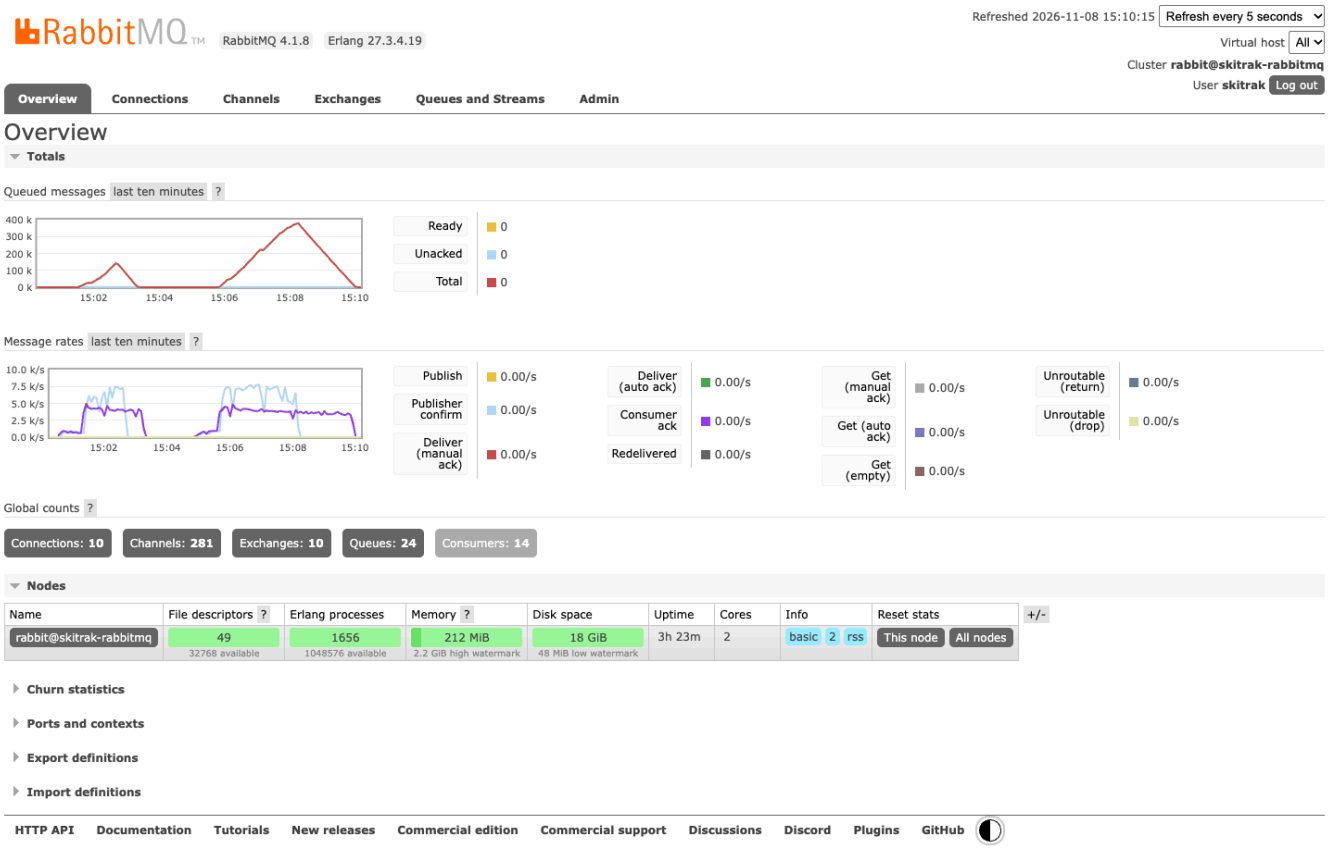






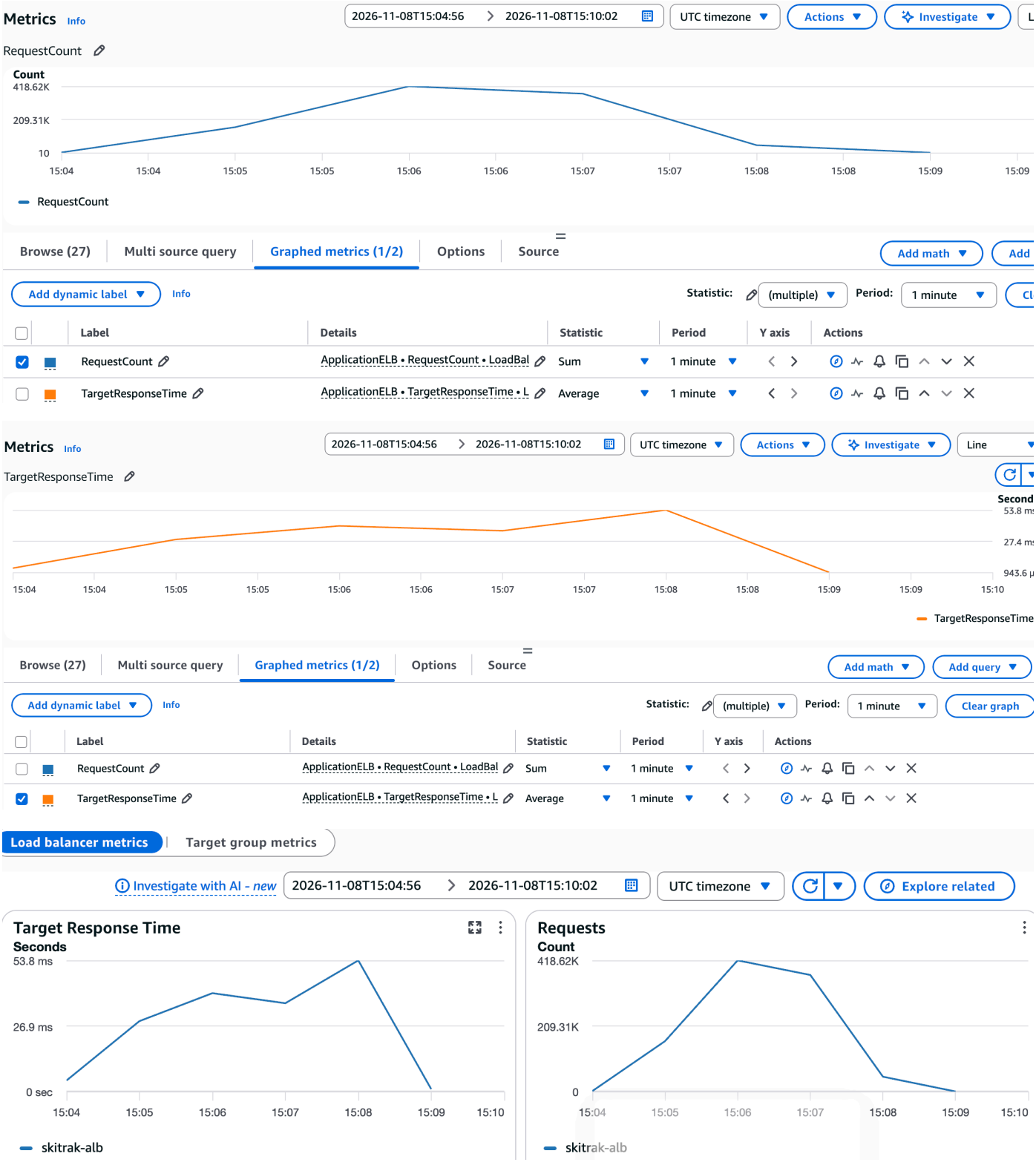
Actual RabbitMQ console captures:

- rabbitmq-20261108T150455745881Z.png
- rabbitmq-20261108T150540945189Z.png
- rabbitmq-20261108T150626224770Z.png
- rabbitmq-20261108T150711689935Z.png
- rabbitmq-20261108T150757300156Z.png
- rabbitmq-20261108T150842773045Z.png
- rabbitmq-20261108T150928269792Z.png
- rabbitmq-20261108T151016280059Z-final.png



Supplied AWS CloudWatch captures:

LoadBalancer aggregate.



Collection gaps: consumer collection errors: {"timeout": 44}

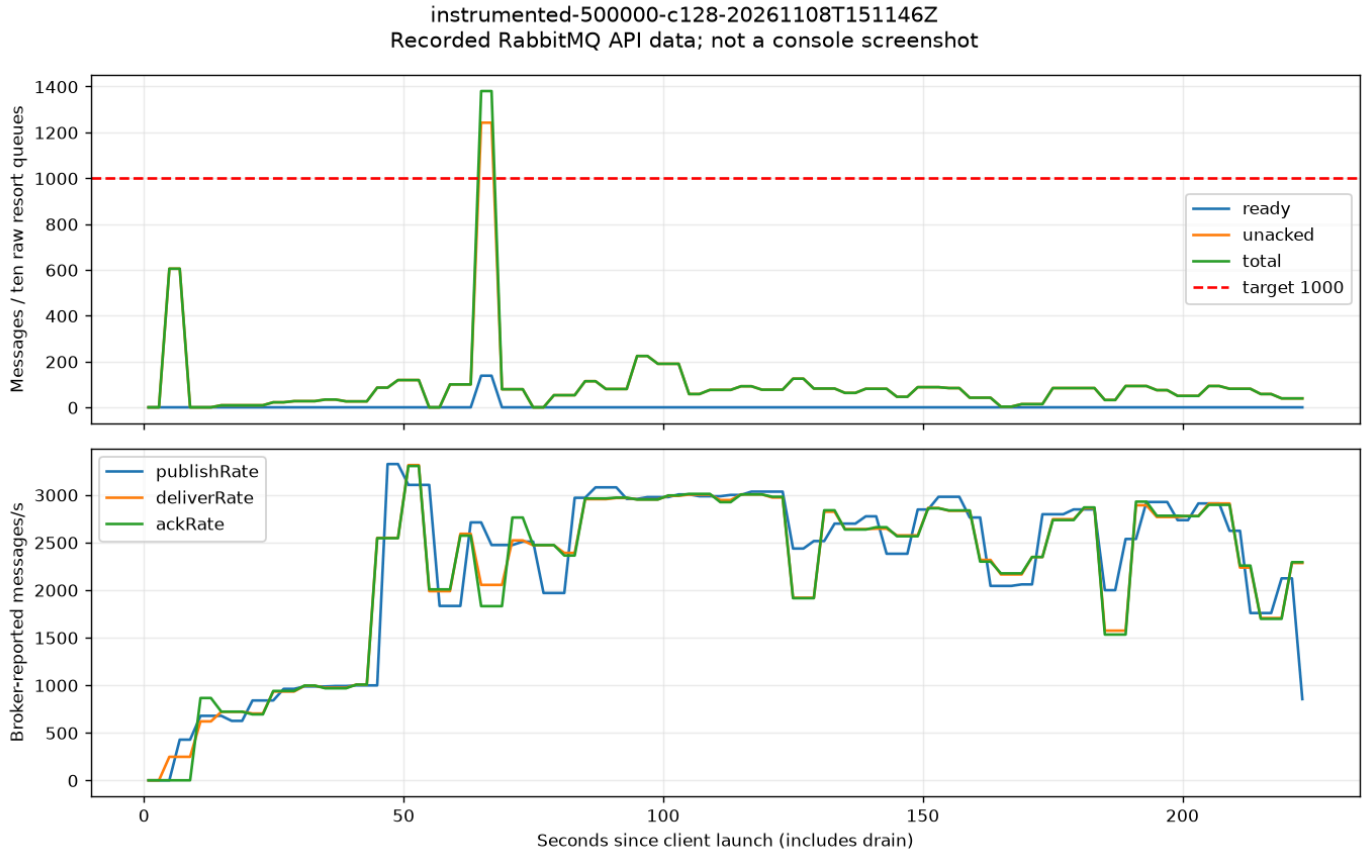
instrumented-500000-c128-20261108T151146Z

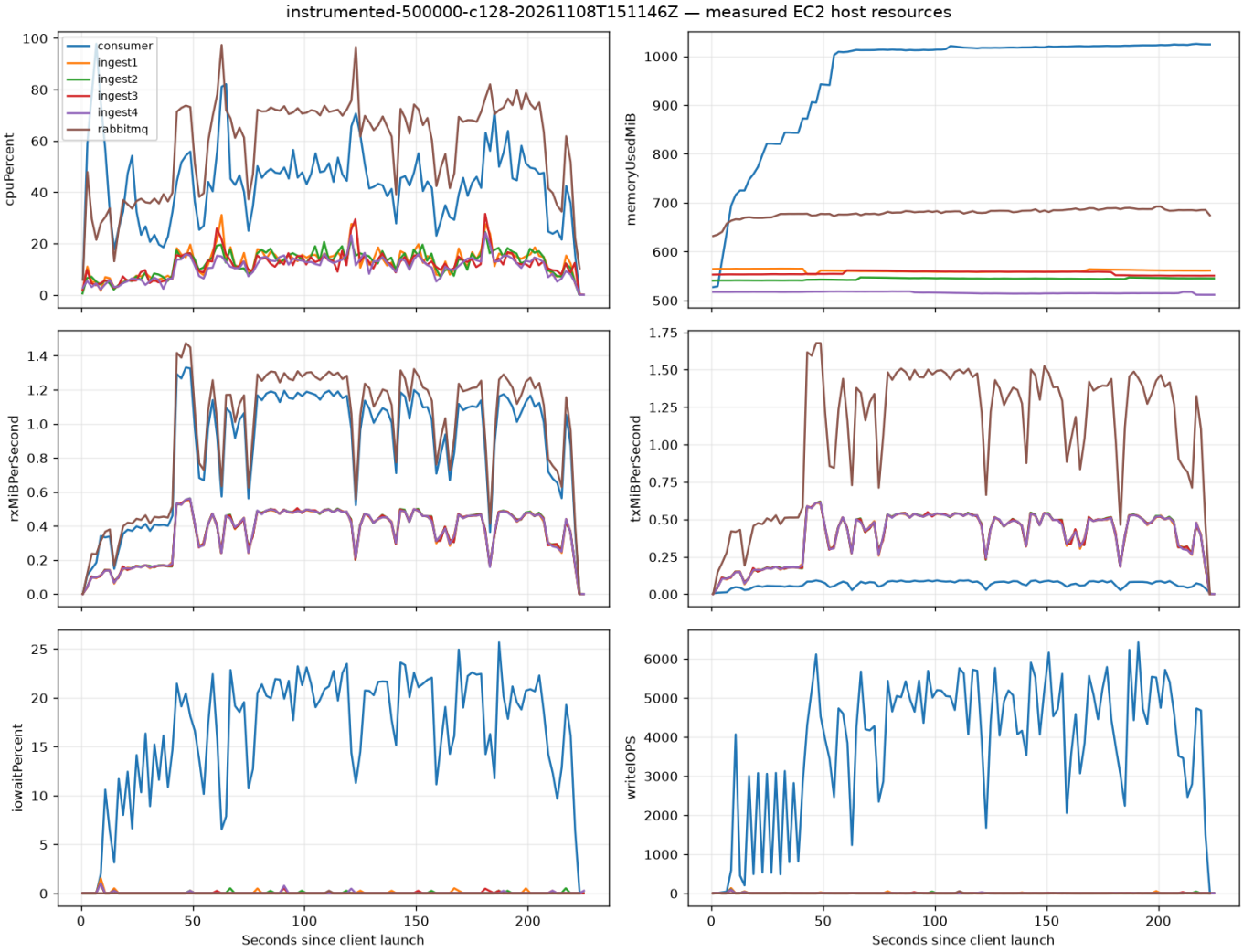
UTC: 2026-11-08T15:12:00.633818+00:00 to 2026-11-08T15:15:45.724134+00:00. Ingests=4; clientThreads=128; consumer={"configuredThreads": 5, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

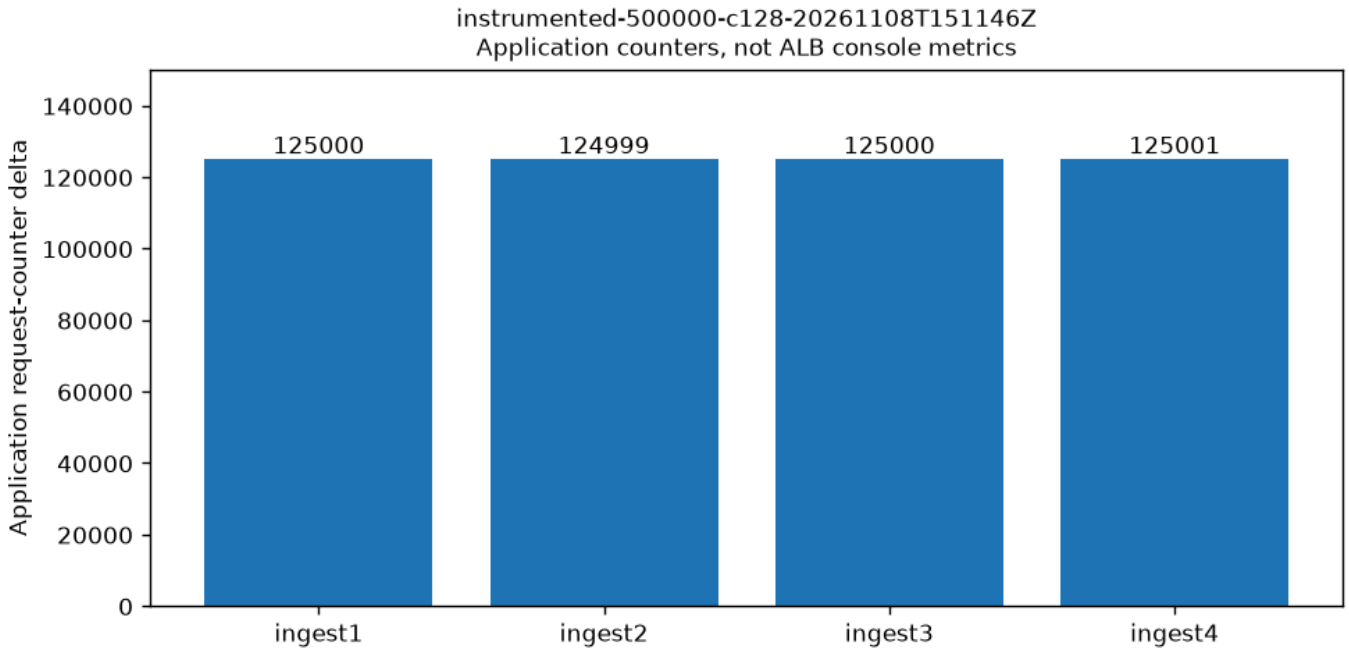
Complete client output | Full metrics, provenance and limits | Evidence page

Measurement	Result
Ingestion runtime seconds	218.414
Program runtime seconds	223.529
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	48.42 / 78.05 / 241.01
Ready sampled mean / peak	2.46 / 138
Ready + Unacked peak	1,380
Mean broker publish / delivery / ACK rates per second	2,236.23 / 2,205.92 / 2,201.43
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	918 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

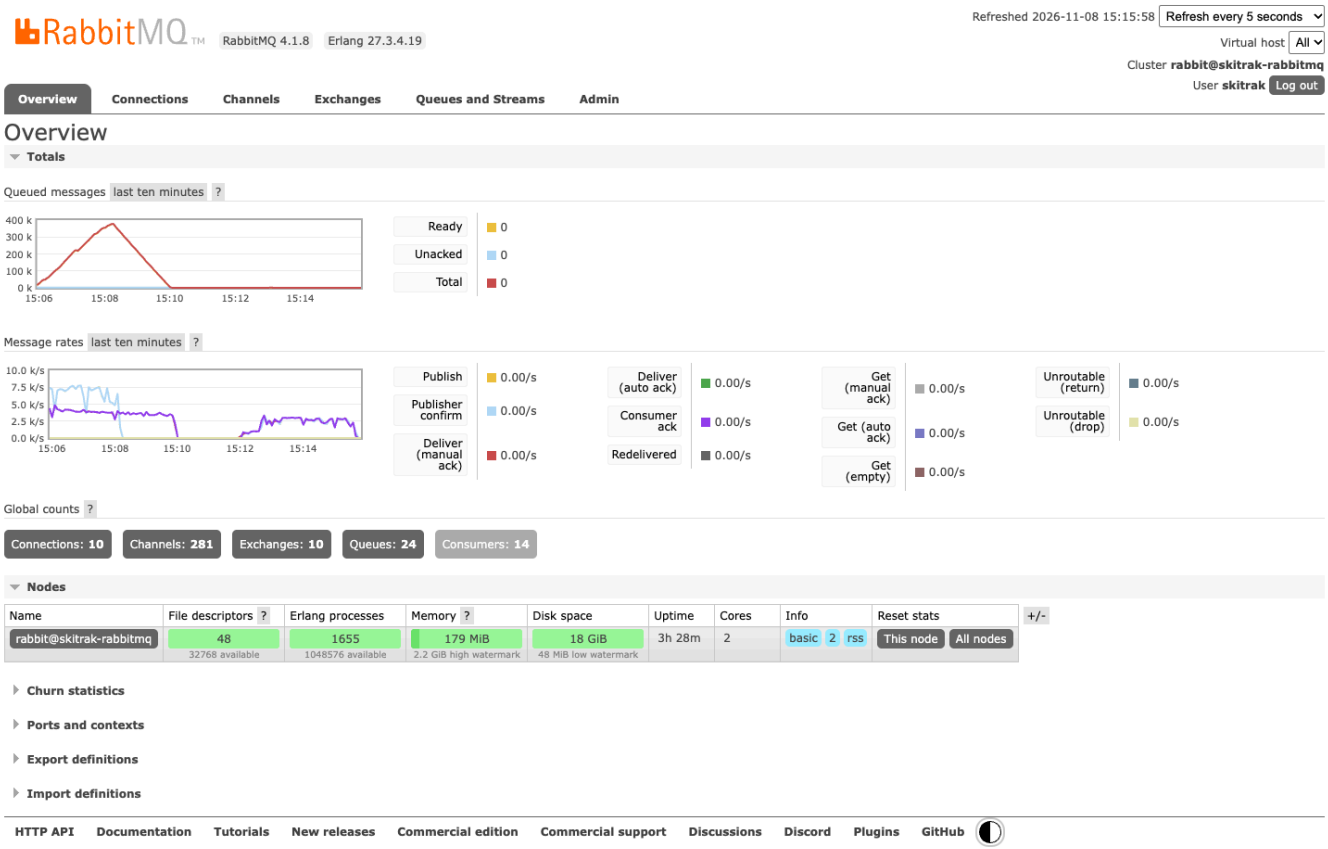






Actual RabbitMQ console captures:

- rabbitmq-20261108T151200561015Z.png
- rabbitmq-20261108T151245981478Z.png
- rabbitmq-20261108T151331142579Z.png
- rabbitmq-20261108T151416395564Z.png
- rabbitmq-20261108T151501633633Z.png
- rabbitmq-20261108T151547186523Z.png
- rabbitmq-20261108T151559637186Z-final.png



Collection gaps: consumer collection errors: {"timeout": 1}

instrumented-500000-c128-20261108T151722Z

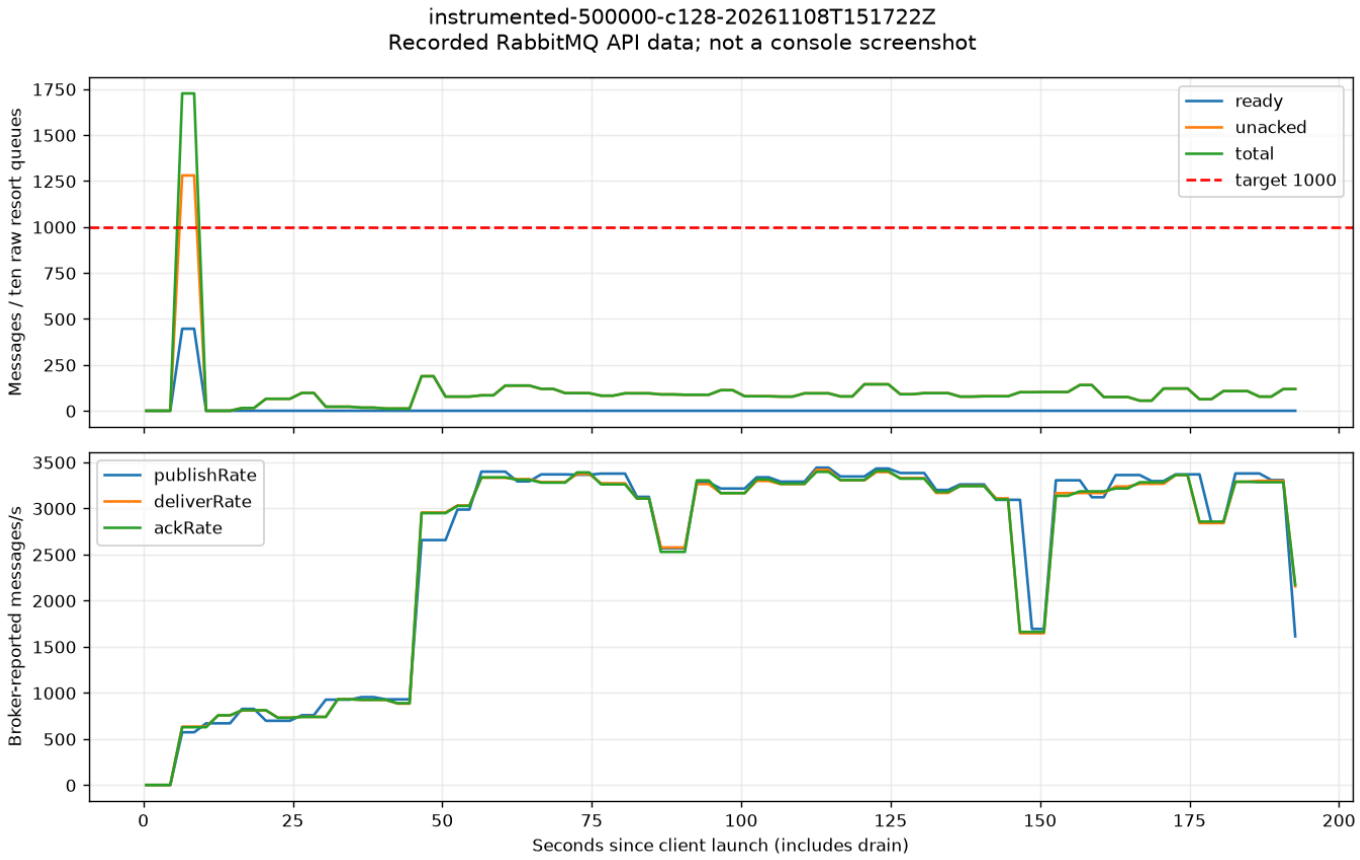
UTC: 2026-11-08T15:17:38.725229+00:00 to 2026-11-08T15:20:52.587022+00:00. Ingests=4; clientThreads=128; consumer={"configuredThreads": 10, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

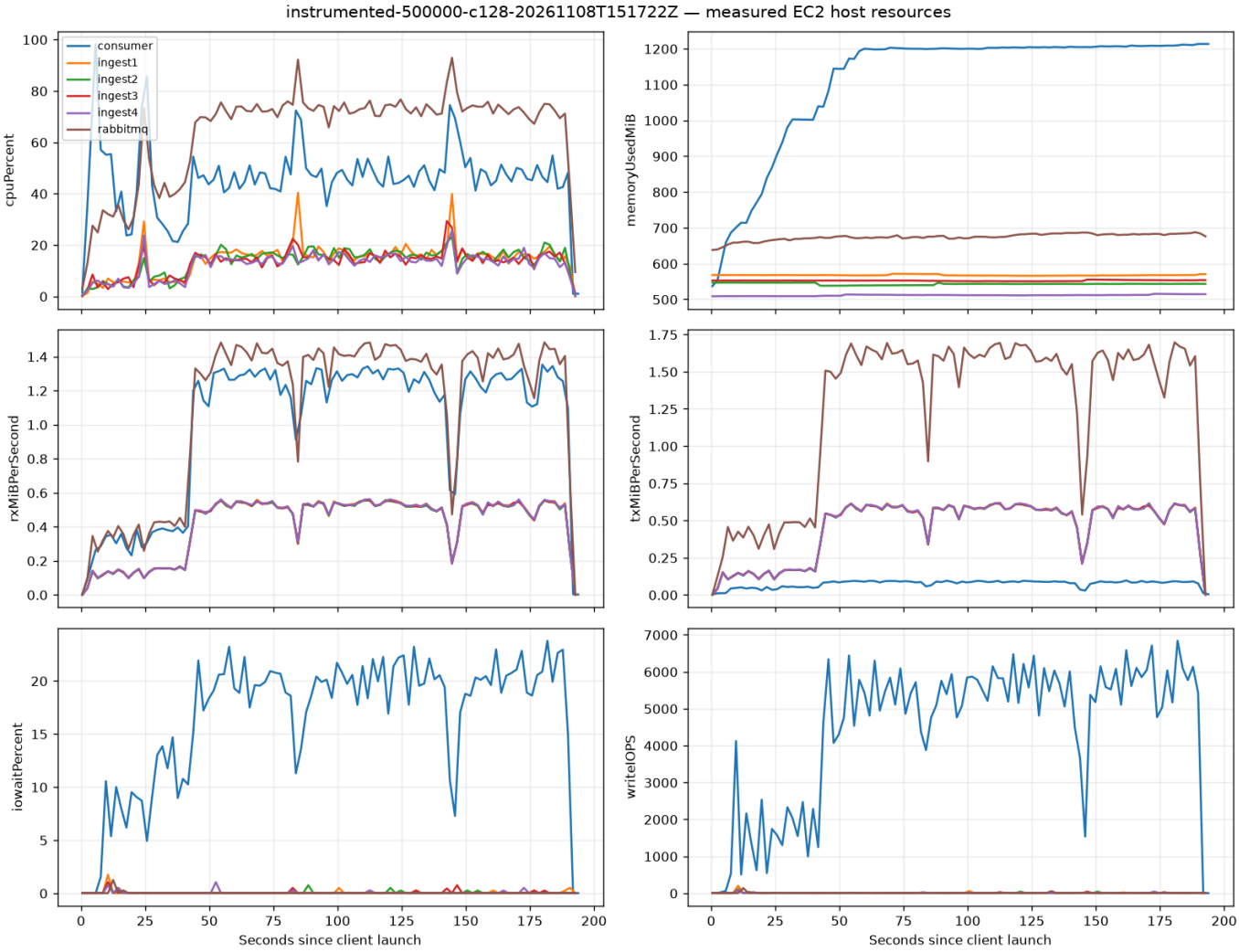
Complete client output | Full metrics, provenance and limits | Evidence page

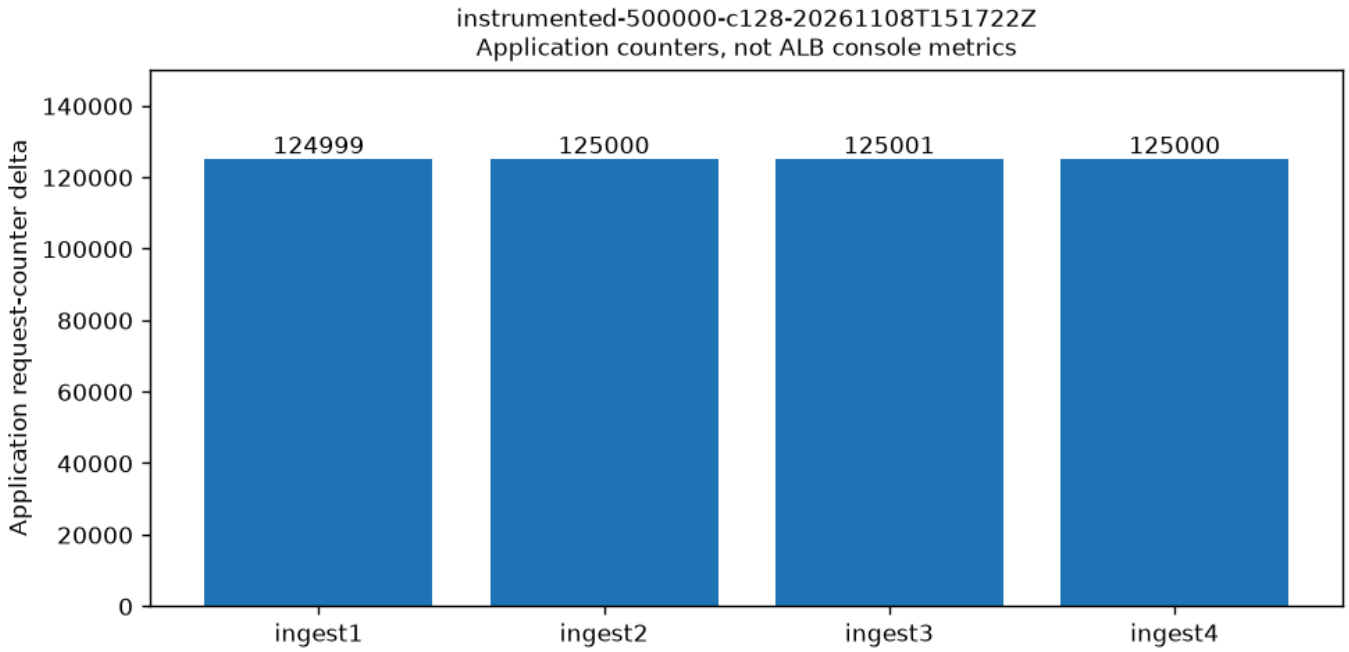
Measurement	Result
Ingestion runtime seconds	187.730
Program runtime seconds	192.907
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	40.43 / 55.58 / 92.10
Ready sampled mean / peak	9.20 / 446

Measurement	Result
Ready + Unacked peak	1,726
Mean broker publish / delivery / ACK rates per second	2,576.92 / 2,545.18 / 2,544.72
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	917 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

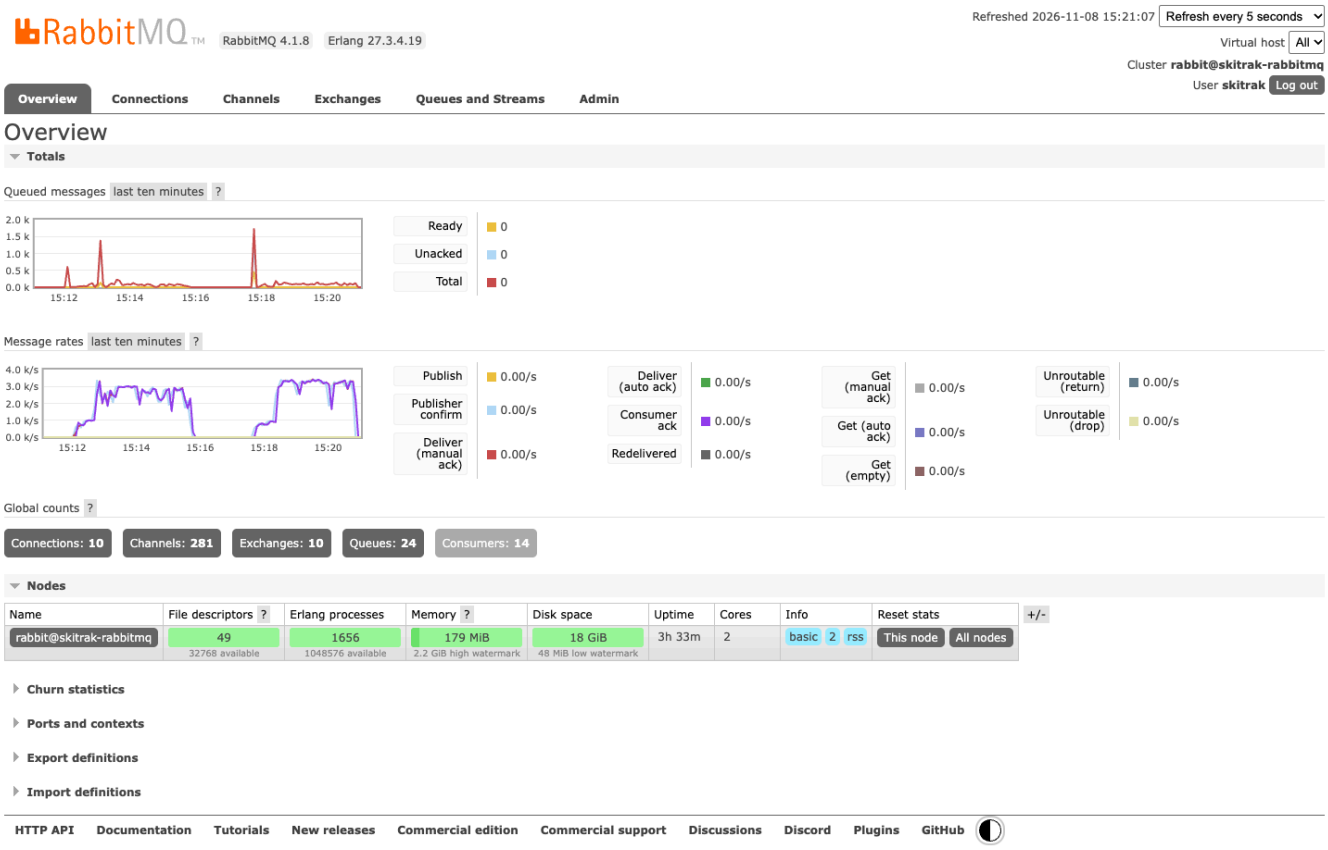






Actual RabbitMQ console captures:

- rabbitmq-20261108T151738291131Z.png
- rabbitmq-20261108T151823696833Z.png
- rabbitmq-20261108T151909185354Z.png
- rabbitmq-20261108T151954728312Z.png
- rabbitmq-20261108T152039898000Z.png
- rabbitmq-20261108T152108146701Z-final.png



Collection gaps: None detected in the recorded collectors.

instrumented-500000-c128-20261108T152227Z

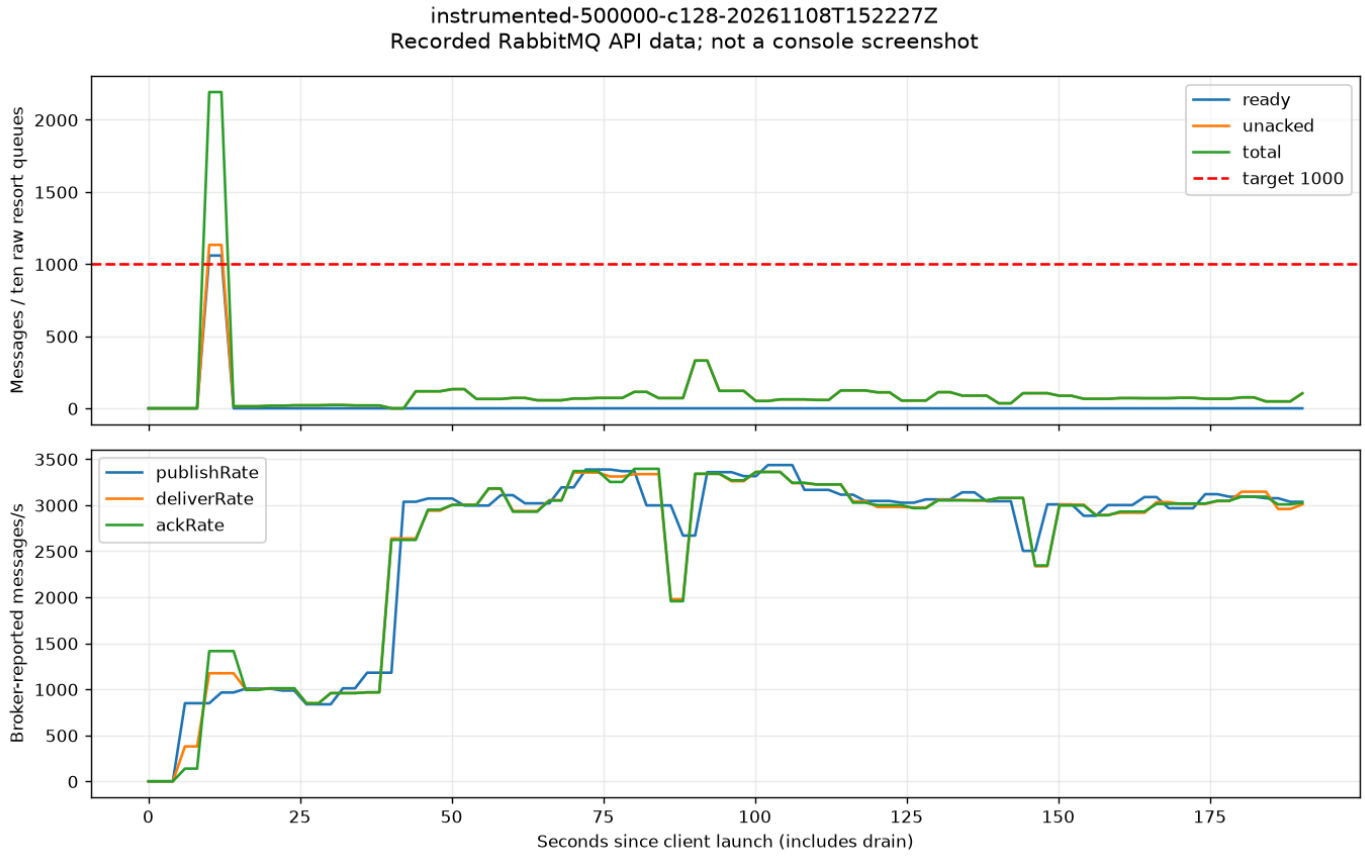
UTC: 2026-11-08T15:22:40.634625+00:00 to 2026-11-08T15:25:52.373649+00:00. Ingests=4; clientThreads=128; consumer={"configuredThreads": 15, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

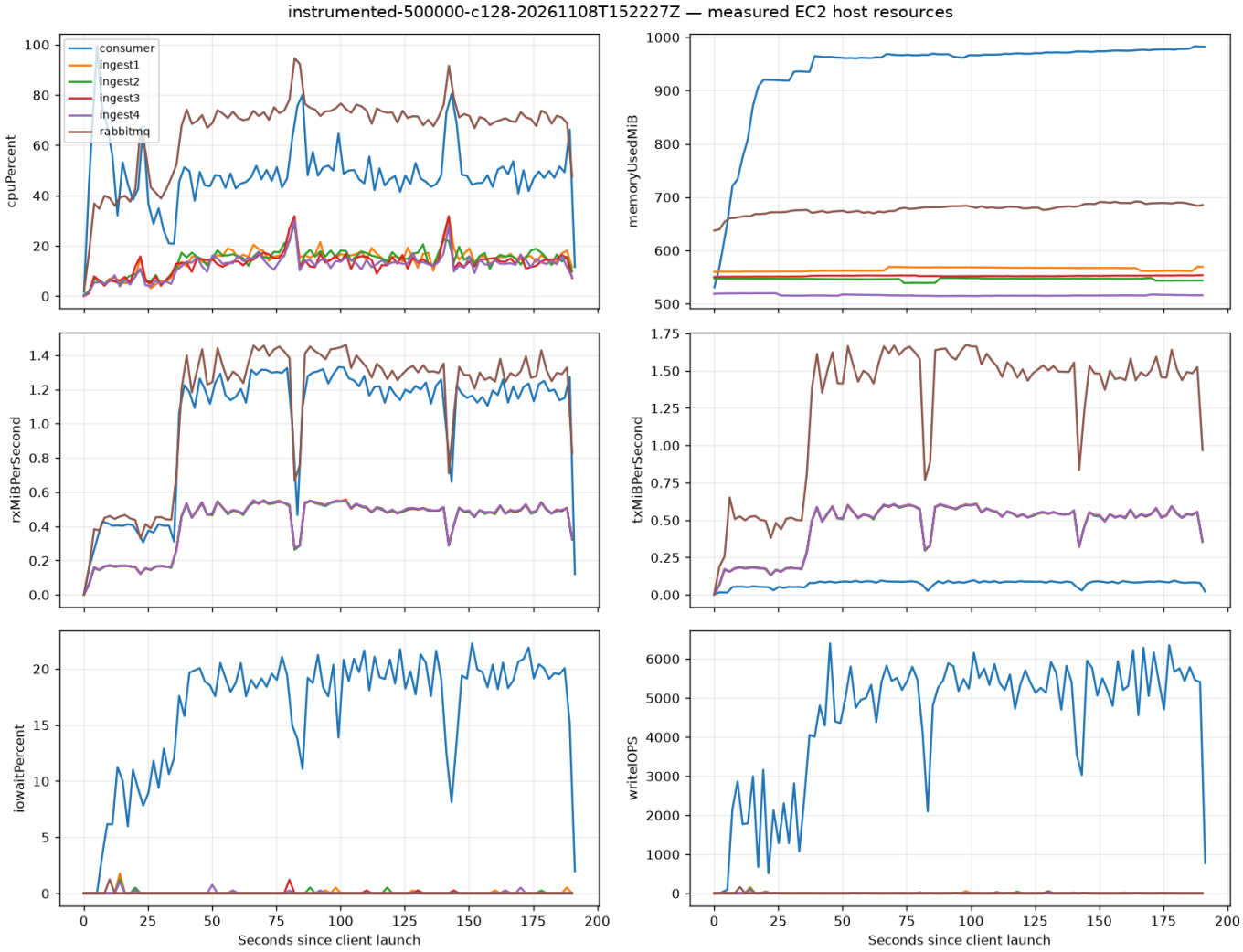
Complete client output | Full metrics, provenance and limits | Evidence page

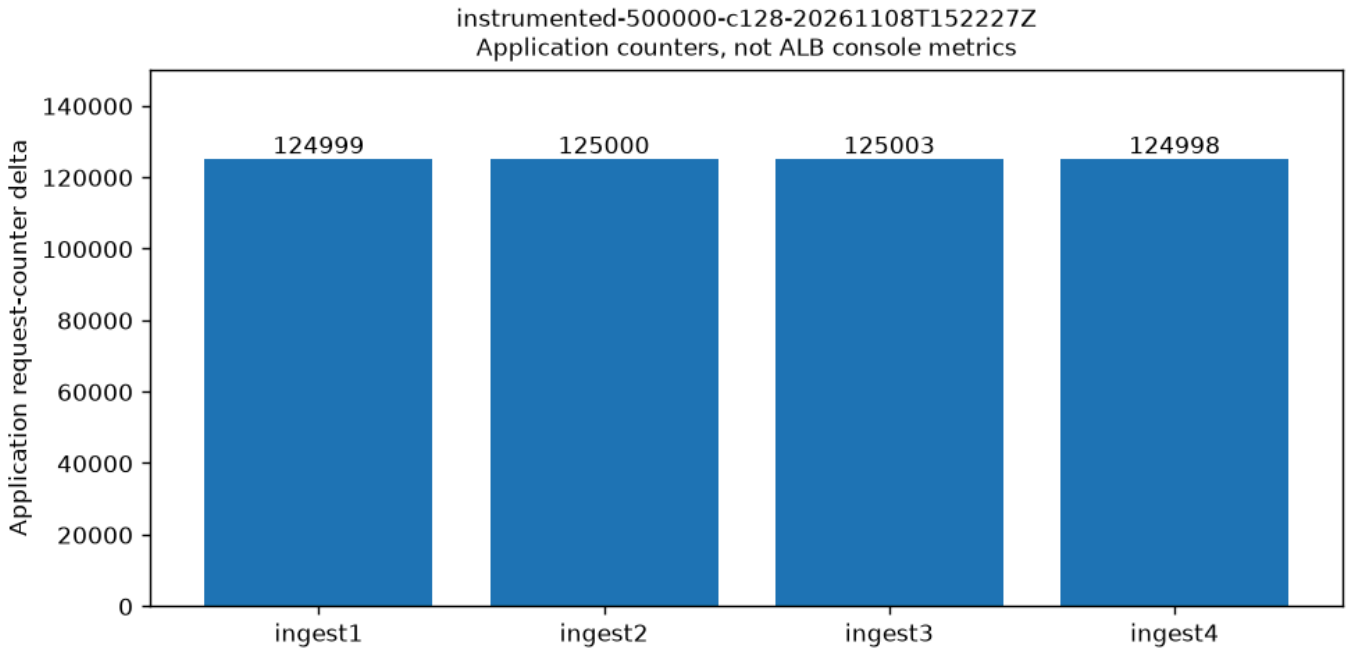
Measurement	Result
Ingestion runtime seconds	188.333
Program runtime seconds	190.874
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	41.66 / 59.26 / 93.09
Ready sampled mean / peak	22.06 / 1,059
Ready + Unacked peak	2,192

Measurement	Result
Mean broker publish / delivery / ACK rates per second	2,591.56 / 2,567.88 / 2,570.40
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	915 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

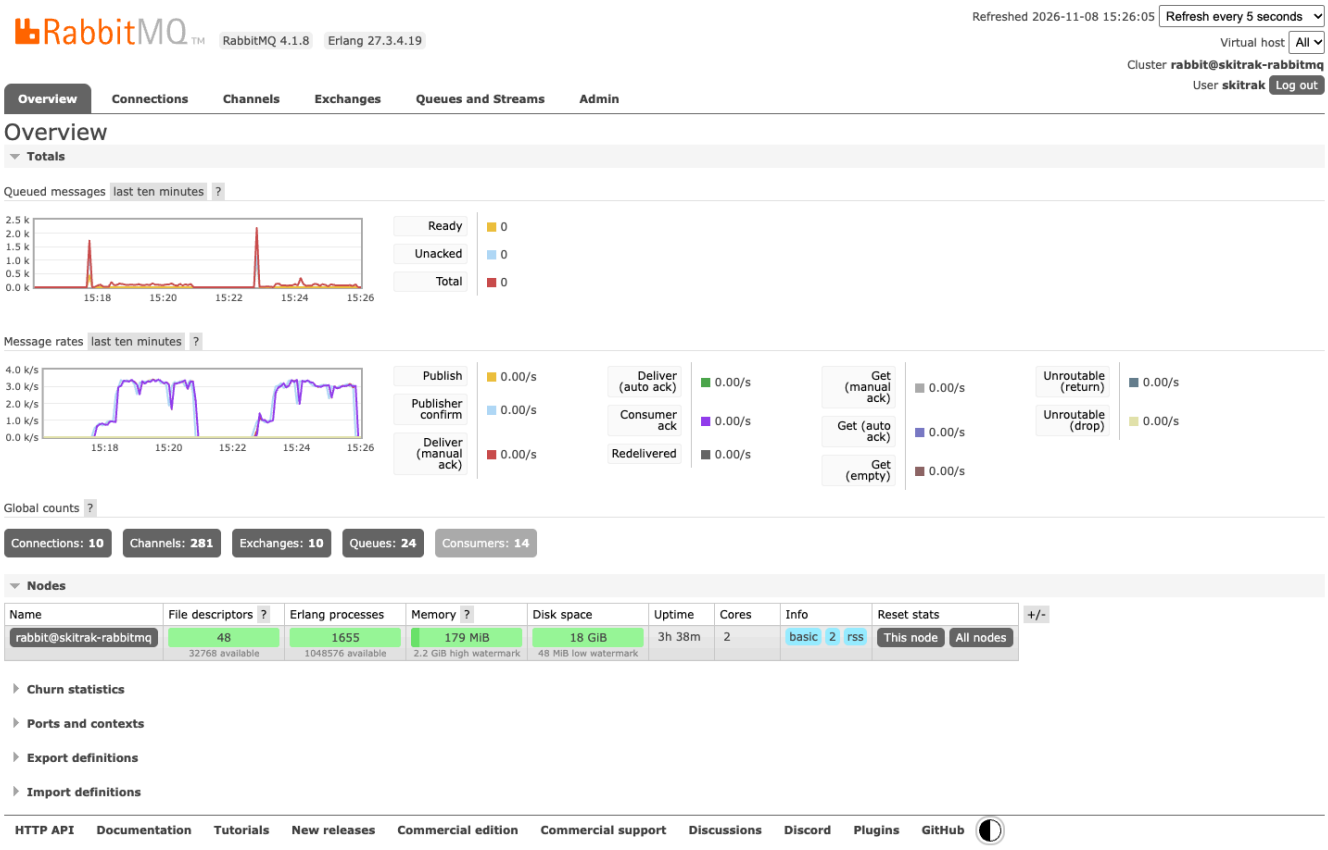






Actual RabbitMQ console captures:

- rabbitmq-20261108T152240430076Z.png
- rabbitmq-20261108T152325817680Z.png
- rabbitmq-20261108T152411184683Z.png
- rabbitmq-20261108T152456578021Z.png
- rabbitmq-20261108T152541975870Z.png
- rabbitmq-20261108T152606268242Z-final.png



Collection gaps: None detected in the recorded collectors.

instrumented-500000-c128-20261108T152726Z

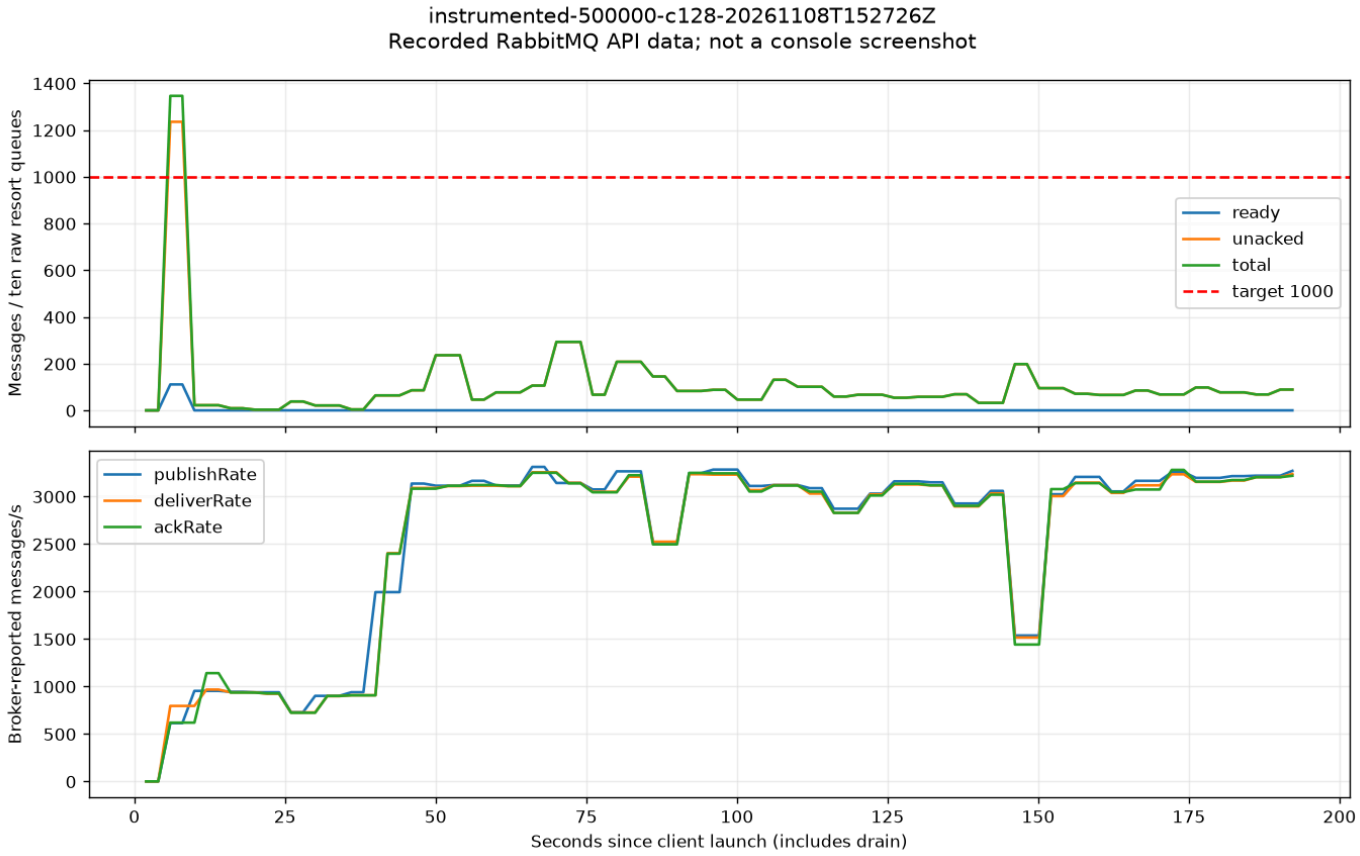
UTC: 2026-11-08T15:27:39.783122+00:00 to 2026-11-08T15:30:53.922037+00:00. Ingests=4; clientThreads=128; consumer={"configuredThreads": 20, "dynamicScaling": false, "prefetch": 128, "storeBatchSize": 64, "storeBatchWaitMs": 10, "ackBatchSize": 16}.

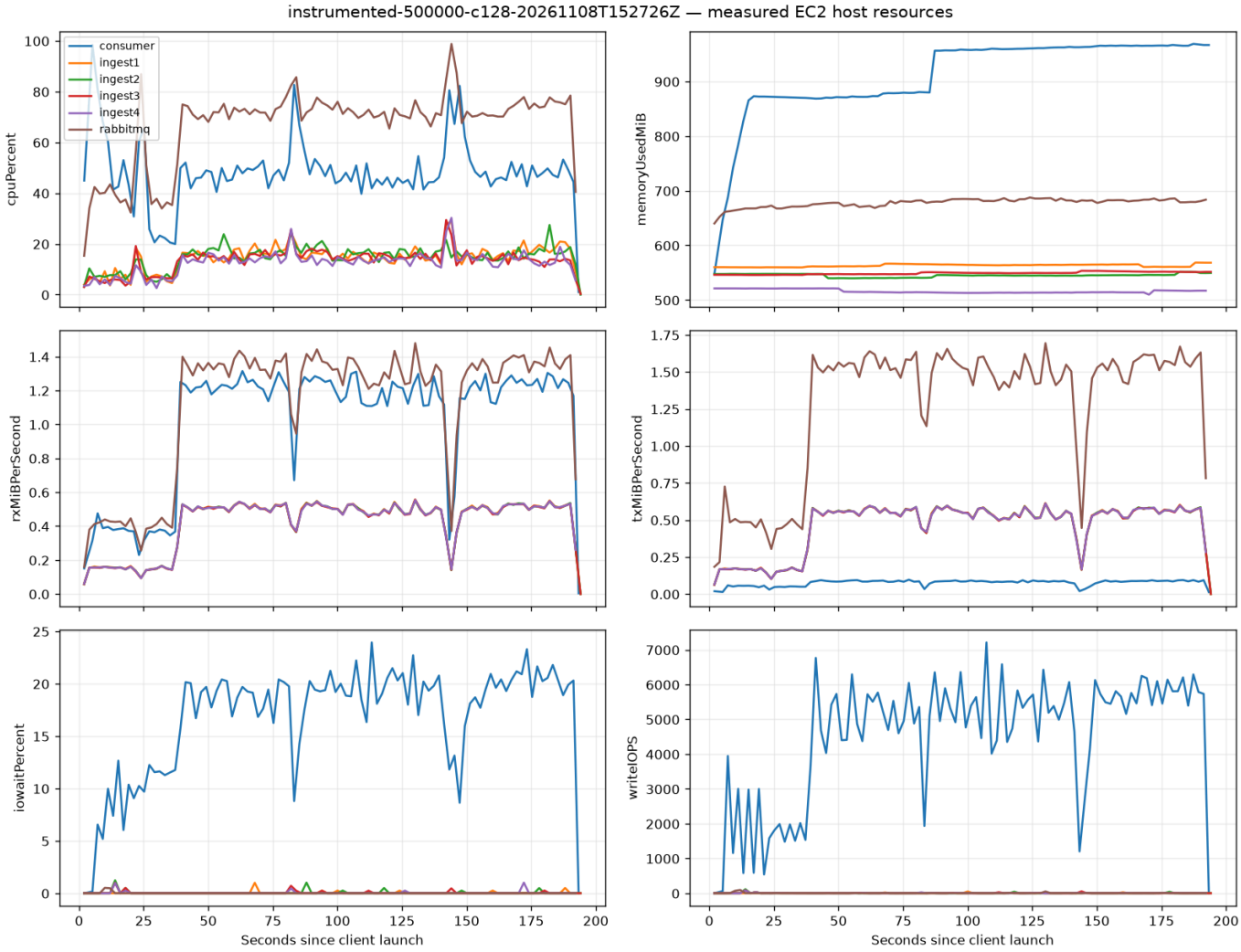
Complete client output | Full metrics, provenance and limits | Evidence page

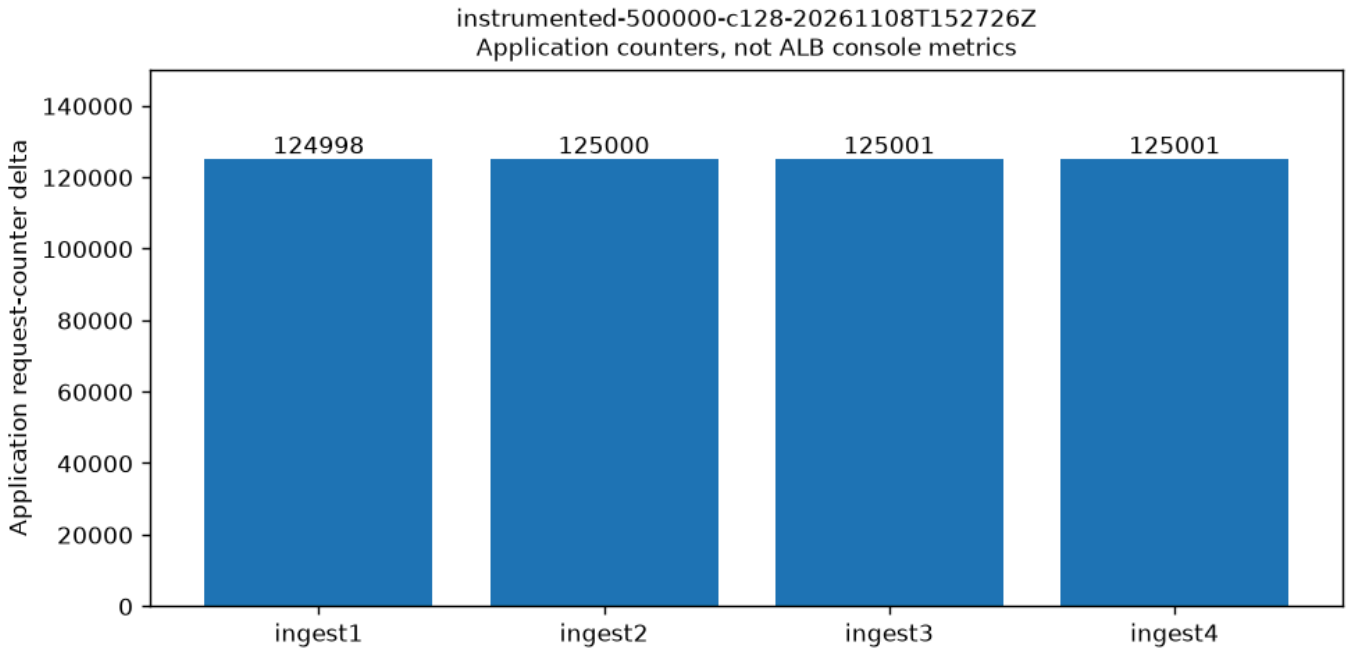
Measurement	Result
Ingestion runtime seconds	190.040
Program runtime seconds	192.782
Successful / final failed / retries	500,000 / 0 / 0
HTTP mean / p95 / p99 ms	41.73 / 58.24 / 99.31
Ready sampled mean / peak	2.31 / 111
Ready + Unacked peak	1,346

Measurement	Result
Mean broker publish / delivery / ACK rates per second	2,566.00 / 2,539.29 / 2,535.58
Additional processed / duplicate count	500000 / 0
Final Ready / in-flight / outbox	0 / 0 / 0
WebSocket updates / reconnections	915 / 0

Per-host CPU, memory, physical network and disk sample means/maxima are preserved in summary.json and per-host CSVs. Application counter distribution is not an ALB per-target console metric.

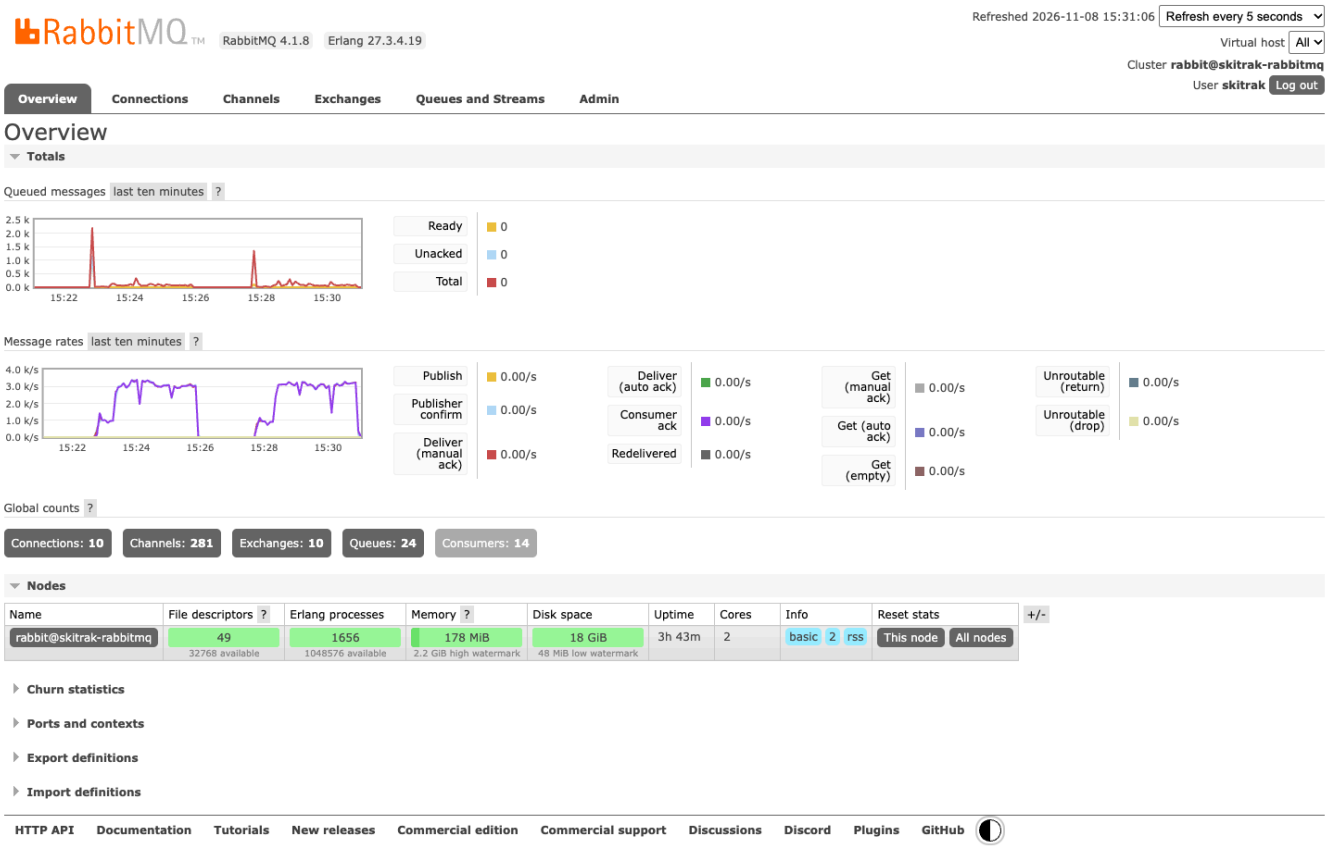






Actual RabbitMQ console captures:

- rabbitmq-20261108T152739462261Z.png
- rabbitmq-20261108T152824912813Z.png
- rabbitmq-20261108T152910364713Z.png
- rabbitmq-20261108T152955823075Z.png
- rabbitmq-20261108T153041273641Z.png
- rabbitmq-20261108T153107468189Z-final.png



Collection gaps: None detected in the recorded collectors.

Subscribers receive real, swipe-driven leaderboard updates

The load client holds one WebSocket subscription per resort (ten in all) through the ALB and records every pushed snapshot. The updates are not synthetic: the only producer of leaderboard snapshots is the Consumer, which publishes a snapshot to `leaderboard.topic` after committing the swipes that changed a resort's Top-N, and each ingest forwards it to its local subscribers. Four observations support that these updates reflect the swipes being sent.

1. Updates arrive during every run, with no reconnection. Each run received between 903 and 1,023 snapshot updates across the ten resorts (about 90 per resort for 500K events, 1,023 for the 1M run), with 0 reconnections. The client output lists each update with the server timestamp and the receipt time, for example (500K, 128 threads, 1 ingest, client output): `Resort 4, update 2, server time 14:03:25.848739Z`, then `update 3` at `14:03:26.061891Z`. The update number increases per resort. An update is a full snapshot published when a resort's displayed standings change, not one message per swipe.

2. The leaderboard totals grow by exactly the number of swipes sent. Each run ends with a snapshot per resort carrying a cumulative `processed` count (the same day ID is used in every run, so the counters accumulate). Summing the

ten final snapshots gives the table below. For all 16 consecutive pairs of runs the increase equals the events the client sent in that run (500,000 per run, 1,000,000 for the 1M run), and the Consumer's own processed counter delta matches. The counters do not depend on how many ingests or client threads were used.

Run	Ingests	Client threads	Workers	Events sent	WebSocket updates received	Reconnections	Consumer processed delta	Final snapshots: sum of `processed`	Increase over previous run
500000-c64-20261108T135722Z	1	64	10	500,000	911	0	500,000	16,000,002	n/a (first run)
500000-c128-20261108T140311Z	1	128	10	500,000	911	0	500,000	16,500,002	500,000
500000-c256-20261108T140642Z	1	256	10	500,000	914	0	500,000	17,000,002	500,000
500000-c512-20261108T140951Z	1	512	10	500,000	913	0	500,000	17,500,002	500,000
500000-c64-20261108T141701Z	2	64	10	500,000	911	0	500,000	18,000,002	500,000
500000-c128-20261108T142410Z	2	128	10	500,000	910	0	500,000	18,500,002	500,000
500000-c256-20261108T142912Z	2	256	10	500,000	904	0	500,000	19,000,002	500,000
500000-c512-20261108T143329Z	2	512	10	500,000	903	0	500,000	19,500,002	500,000
500000-c64-20261108T144338Z	4	64	10	500,000	905	0	500,000	20,000,002	500,000
500000-c128-20261108T145046Z	4	128	10	500,000	910	0	500,000	20,500,002	500,000
500000-c256-20261108T145553Z	4	256	10	500,000	913	0	500,000	21,000,002	500,000
500000-c512-20261108T150014Z	4	512	10	500,000	909	0	500,000	21,500,002	500,000
1000000-c512-20261108T150442Z	4	512	10	1,000,000	1,023	0	1,000,000	22,500,002	1,000,000
500000-c128-20261108T151146Z	4	128	5	500,000	918	0	500,000	23,000,002	500,000
500000-c128-20261108T151722Z	4	128	10	500,000	917	0	500,000	23,500,002	500,000
500000-c128-20261108T152227Z	4	128	15	500,000	915	0	500,000	24,000,002	500,000
500000-c128-20261108T152726Z	4	128	20	500,000	915	0	500,000	24,500,002	500,000

The `processed` totals start from 2 events written by an earlier functional check, hence the trailing 2. Each snapshot also carries the Top-N entries with skier IDs and vertical totals, stored in `final-snapshots-*.json` under each run's original results folder (path in each `summary.json`).

3. Standings match the official lift data, through the ALB and across ingests. The pipeline verification (`results/a2-fresh/20261108T135433Z/verification/pipeline.json`, 4 ingests behind the ALB) sent 100 unique swipes plus 20 repeated requests with the same Idempotency-Key; the requests were spread over the four servers (27 / 25 / 25 / 23), the duplicates were not double-counted (the two resorts used show 50 + 50 = 100 processed events), and the run passed against the official lift table (`demoLifts` false). This file is functional-correctness evidence, not performance evidence. The recovery run (`ec2-recovery.json`) verified the same totals after a Consumer restart with queued backlog and duplicate replay (23 events) and after a broker restart with cross-ingest replay (25 events), and the WebSocket failover run moved a subscriber from `ingest-4` to `ingest-1` in 7.199 s with the standings still correct.

4. Limits of this evidence. The WebSocket snapshot-to-receipt time differences in the client output include clock offset between the Mac and the EC2 hosts and are not calibrated latencies. The updates show that the leaderboard follows the swipes; they do not show that every intermediate version reaches every subscriber, because notifications are transient

snapshots and the Consumer's store is the source of truth. The failover harness reconnects with a Python client; the Java load client is not shown to reconnect automatically.

Functional and recovery evidence

Only result files generated under this fresh session are used; older functional/recovery results are excluded.

alb

Source: results/a2-fresh/20261108T135433Z/verification/alb.json


```
{
  "distribution": {
    "ingest-2": 51,
    "ingest-4": 49,
    "ingest-1": 50,
    "ingest-3": 50
  },
  "stickyPassed": true,
  "distributionTargetsPassed": true,
  "failoverTested": false
}
```


ec2-recovery

Source: results/a2-fresh/20261108T135433Z/verification/ec2-recovery.json


```
{
  "passed": true,
  "dayID": 339,
  "brokerDownPublishStatus": 503,
  "restoreErrors": []
}
```


- initial: 3 unique events verified against official lift totals and subscribed standings.
- consumer restart, queued backlog and duplicate replay: 23 unique events verified against official lift totals and subscribed standings.
- broker restart and cross-ingest replay: 25 unique events verified against official lift totals and subscribed standings.

pipeline

Source: results/a2-fresh/20261108T135433Z/verification/pipeline.json


```
{
  "passed": true,
  "demoLifts": false,
  "dayID": 340,
  "uniqueEvents": 100,
  "duplicateRequests": 20,
  "serverRequestCounts": {
    "ingest-1": 27,
    "ingest-4": 25,
    "ingest-2": 25,
    "ingest-3": 23
  }
}
```

websocket-failover

Source: results/a2-fresh/20261108T135433Z/verification/websocket-failover.json


```
{
  "passed": true,
  "dayID": 338,
  "originalServer": "ingest-4",
  "replacementServer": "ingest-1",
  "reconnectSecondsFromStopStart": 7.199,
  "restoredIngestHealthy": true
}
```


Reconnection mode: Python test harness retries a new WebSocket through ALB. This is not evidence of the Java load client automatically reconnecting.

Fresh local regression

{"tests": 61, "failures": 0, "errors": 0, "skipped": 0}. Build success: True.

Initial restricted attempt could not bind localhost sockets. It is preserved separately; the complete rerun with local networking allowed passed without test-code changes.

Logs and JUnit XML: results/a2-fresh/20261108T135433Z/local-regression.

Recorded RabbitMQ topology

Captured at 2026-11-08T14:41:20.790711+00:00. Raw queues: 10; raw routing bindings: 10.

Exchanges: leaderboard.topic (topic), skitrak.dlx (direct), skitrak.exchange (topic).

First raw queue arguments (the full record includes all ten): `{"x-dead-letter-exchange": "skitrak.dlx", "x-dead-letter-routing-key": "resort.1", "x-max-length": 100000, "x-message-ttl": 86400000, "x-overflow": "reject-publish", "x-queue-type": "classic", "x-single-active-consumer": true}`.

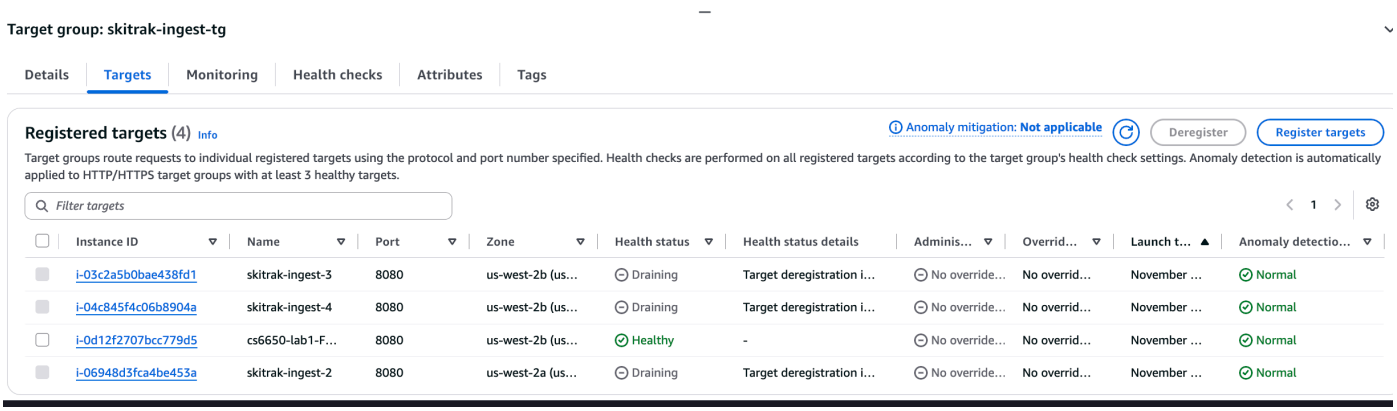
Included AWS console evidence

The supplied AWS console screenshots have been incorporated into this report and its PDF. They are not outstanding submission evidence.

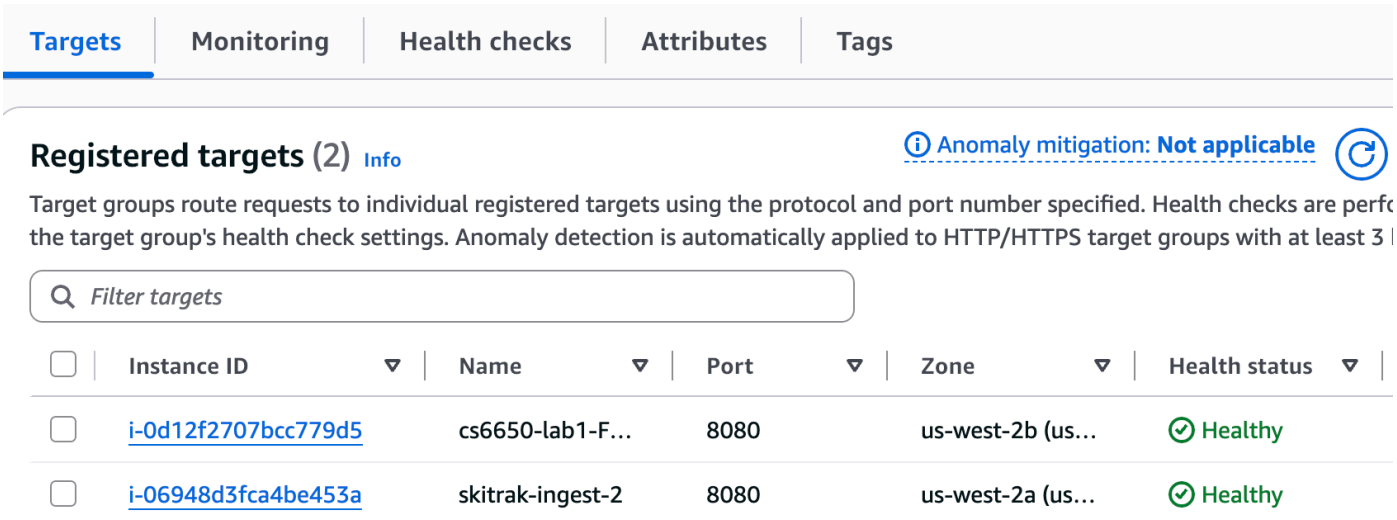
Stage	Included evidence
1 ingest(s), 500,000 events, 64 client threads	alb-request-count.png, alb-target-response-time.png
1 ingest(s), 500,000 events, 128 client threads	alb-target-response-time.png, alb-request-count.png
1 ingest(s), 500,000 events, 256 client threads	alb-request-count.png, alb-target-response-time.png
1 ingest(s), 500,000 events, 512 client threads	alb-target-response-time.png, alb-request-count.png
2 ingest(s), 500,000 events, 64 client threads	alb-request-count-whole-loadbalancer.png, alb-target-response-time-whole-loadbalancer.png
2 ingest(s), 500,000 events, 128 client threads	alb-request-count-whole-loadbalancer.png, alb-target-response-time-whole-loadbalancer.png
2 ingest(s), 500,000 events, 256 client threads	alb-request-count-whole-loadbalancer.png, alb-target-response-time-whole-loadbalancer.png
2 ingest(s), 500,000 events, 512 client threads	alb-request-count-whole-loadbalancer.png, alb-target-response-time-whole-loadbalancer.png
4 ingest(s), 500,000 events, 64 client threads	alb-response-time-whole-loadbalancer.png, alb-request-count-whole-loadbalancer.png
4 ingest(s), 500,000 events, 128 client threads	alb-response-time-whole-loadbalancer.png, alb-request-count-whole-loadbalancer.png
4 ingest(s), 500,000 events, 256 client threads	alb-overview-supplied.png, alb-request-count-supplied.png, alb-response-time-supplied.png

Stage	Included evidence
4 ingest(s), 500,000 events, 512 client threads	alb-overview-supplied.png, alb-response-time-supplied.png, alb-request-count-supplied.png
4 ingest(s), 1,000,000 events, 512 client threads	alb-request-count-supplied.png, alb-response-time-supplied.png, alb-overview-supplied.png

ALB registered targets



Single-ingest transition: ingest1 Healthy and three targets Draining. This is a transition capture, not a final one-target screenshot.



Two-ingest stage: two registered Healthy targets.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (4) Info

Anomaly mitigation: **Not applicable**

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are pe
the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least

Filter targets

Instance ID	Name	Port	Zone	Health status
i-03c2a5b0bae438fd1	skitrak-ingest-3	8080	us-west-2b (us…	Healthy
i-04c845f4c06b8904a	skitrak-ingest-4	8080	us-west-2b (us…	Healthy
i-0d12f2707bcc779d5	cs6650-lab1-F…	8080	us-west-2b (us…	Healthy
i-06948d3fca4be453a	skitrak-ingest-2	8080	us-west-2a (us…	Healthy

Four-ingest stage: four registered Healthy targets.

ALB configuration captures

These three captures show the ALB and target-group settings used by every run in this report (cookie stickiness 5 minutes, health check `/health` every 30 s with timeout 5 s and healthy/unhealthy thresholds 2/3, connection idle timeout 120 s). They were captured earlier on 2026-11-08 (about 05:18 to 05:25 UTC, before this session) from `skitrak-alb` and `skitrak-ingest-tg` and were not re-captured during this session; they document configuration and are not metrics for any run.

Target groups (1/1) Info | What's new?

Actions | Create targe

Filter target groups

1

Name	ARN	Port	Protocol	Target type	Load balancer
skitrak-ingest-tg	arn:aws:elasticloadbalancin…	8080	HTTP	Instance	skitrak-alb

Target group: skitrak-ingest-tg

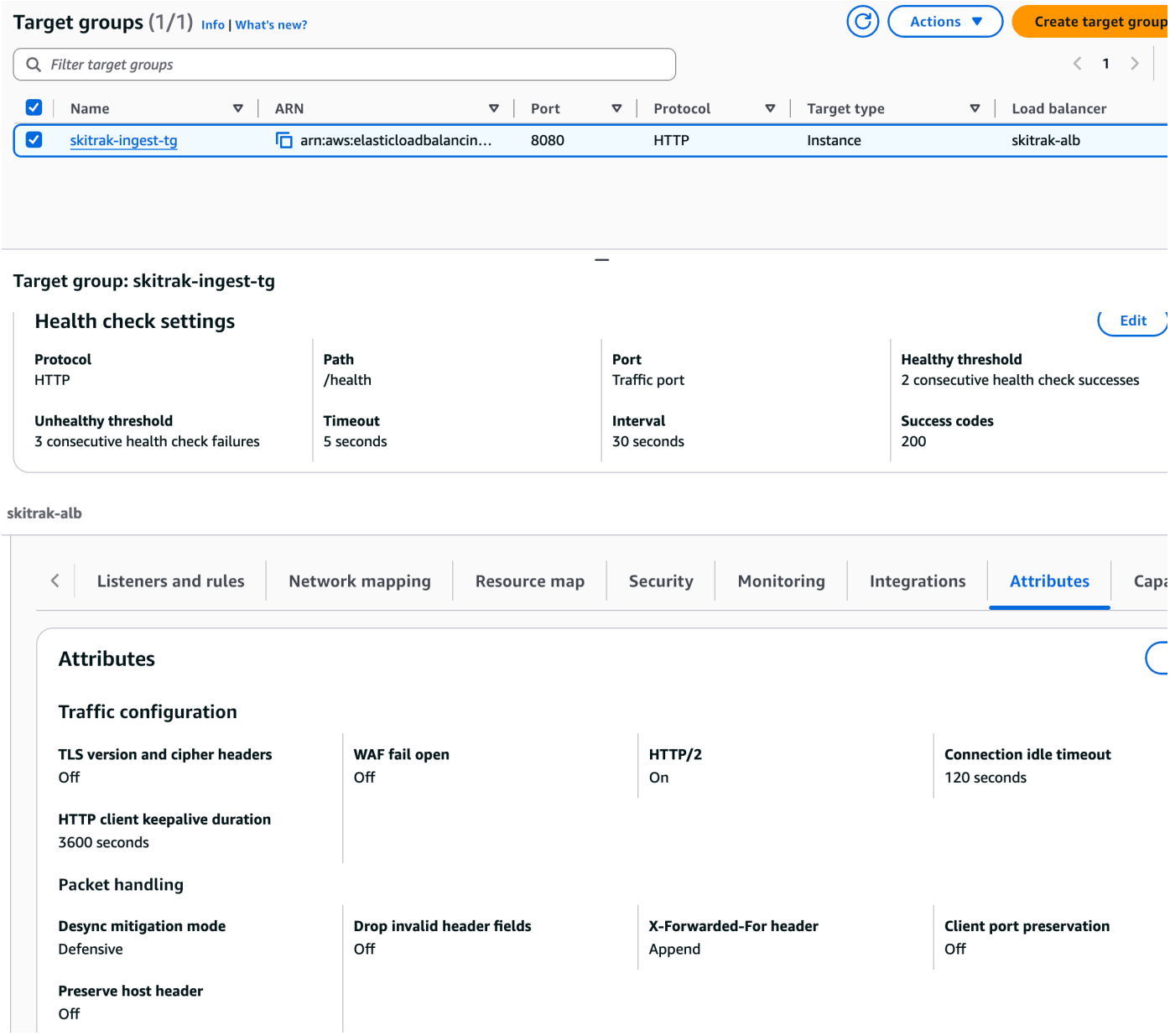
Target selection configuration

Stickiness
On

Stickiness type
Load balancer generated cookie

Stickiness duration
5 minutes

Cross-zone load balancing
Inherit settings from load balancer attributes



Superseded captures kept on disk

Twenty-one earlier ALB captures are stored under the run evidence folders but deliberately not linked above, because they were replaced by the whole-LoadBalancer captures: the zone-scoped `alb-*-az-us-west-2b.png` and the unscoped `alb-request-count.png`, `alb-target-response-time.png` and `alb-overview.png` for the three 2-ingest runs at 64, 256 and 512 threads, and the `alb-*-supplied.png` files for the 4-ingest runs at 64 and 128 threads. Every other screenshot recorded for this session (158 in the source folders) is included in this report.

The corrected two- and four-ingest metric captures use the verified LoadBalancer scope, RequestCount Sum, TargetResponseTime Average and 60-second periods. Historical windows correspond to the recorded scaling runs. The single-ingest transition image is described accurately; independent routing probes establish the responding server set. Consumer-worker sweep runs have broker/system evidence but no separately supplied CloudWatch captures; these are not listed as missing scaling-run screenshots.

Submission review items

The remaining actions are to review the student AI-use reflection, verify publication of the A2 source folders to the linked repository, and regenerate `test-report.pdf` from this file (the PDF was built before these additions). The report already discloses overload, retries, monitoring gaps and harness-managed reconnection. No further 1/2/4-ingest scaling screenshots or benchmark reruns are requested here.